

Depths of Induction, ML-Guided Proof Search, and Formalized Self-Awareness

Formal Systems, Inference Closure, Automated Theorem Proving,
Simulations, and Self-Aware Structures

Version 47

Brian Tenneson

M.S., Applied Data Science

Currently Unaffiliated

August 1, 2026

Part I continues Depths of a Simulation, Version 45.

The Version 47 supplement adds induction and ML-guided ATP analysis.

Part II preserves Formalized Self-Awareness and Self-Aware Structures intact.

Copyright 2026 Brian Tenneson. All Rights Reserved.

Abstract

This paper studies formal systems, semi-ideal computers (SICs), automated theorem provers (ATPs), simulation preorders, proof search, machine-learned search guidance, and formally self-aware structures. Version 47 retains Version 46’s systematic account of induction at four distinct levels. First, structural induction on well-formed formulas is available when the formula set is the least set generated by an explicit finitary grammar; it is not a consequence of merely declaring an arbitrary set of strings to be the well-formed formulas. Second, induction on finite derivations is equivalent to the least-inference-closure principle already underlying the consequence operator. Third, rule-driven ATPs admit stage induction, birth-time induction, and invariant induction, yielding uniform soundness principles and quantitative lower bounds on the first stage at which a target can appear. Fourth, the reflection tower used to grade formal self-awareness is an ordinary inductive sequence at finite levels, while its proposed transfinite extension requires genuinely separate limit-stage clauses.

The added results show that induction applies to ATPs in a mathematically substantive way: it proves preservation of formula invariants across a run, characterizes the union of exhaustive closure stages, propagates halting and freezing properties, and converts slowly changing formula ranks into lower bounds on ATP birth times. Version 47 additionally connects the proof-counting bound of Part I to leakage-controlled machine-learned ranking. In a controlled experiment, unguided Predator exhausted a 5,000-expansion budget on the Metamath theorem `prcom` without a certificate, whereas Predator 8.002, under the same formal environment and resource controls, found an independently verified certificate after 2,633 expansions. The experiment does not establish an asymptotic speedup theorem; it supplies a reproducible finite-instance proof of concept showing that learned ranking and controlled counterfactual creativity can concentrate finite search effort without weakening formal verification. Applied to the reflection-closure machine, the induction method still computes the exact proof layer of every finite reflection formula. The self-awareness theory, including tags, self-models, self-theories, the reflection sequence, the level function, the self-awareness preorder, the Discovery Thesis, the Picard Theorem, the reflection-closure machine, and the discussion of integrated information, is preserved without substantive alteration.

Version 47 changelog

- Added a formal connection between the proof-sequence counting bound of Part I, Section 8, and learned probability-like measures over legal proof-search applications.
- Documented Predator 8.002 as a leakage-controlled ML-guided ATP prototype whose Metamath verifier remains independent of the learned ranker.
- Added the controlled `prcom` result: unguided search returned bounded unknown after 5,000 expansions, while the ML-guided population produced an independently verified proof after 2,633 expansions under matched controls.
- Distinguished proof length, logical birth stage, search-state expansions, and the cardinal upper bound on proof sequences; ML changes search allocation, not theoremhood or the set of formally legal proofs.
- Identified the result as a finite-instance proof of concept and the HaloProof benchmark as the intended substantially harder transfer test, rather than claiming asymptotic superiority from one target.
- Added a formal distinction among formula-formation induction, proof or rule induction, ATP stage induction, and reflection-sequence induction.
- Added a grammatical extension of the xyz formal-system model. Structural induction on WFFs is stated only under the necessary least-generation hypothesis.
- Restated the consequence set as a least inference-closed set and proved the corresponding rule-induction scheme in predicate form.
- Added state-invariant, formula-invariant, and birth-time induction theorems for rule-driven ATPs.
- Added a quantitative ATP potential theorem: a bounded increase per inference layer forces a lower bound on the birth stage of a target.
- Added an induction theorem for persistent halting and frozen states.
- Connected the induction principles to the finite reflection tower and calculated the reflection-closure machine's exact layers $\text{bR}(\theta k) = k + 2$ on its designated base input.
- Clarified that the self-theory $\text{Self}(M)$ need not itself be inference-closed, even when the full consequence set is.
- Preserved the entirety of Part II, including all formal self-awareness definitions, results, objections, examples, conjectures, and philosophical scope restrictions.

Editorial and pagination note

Version 47 is assembled so that the established Parts I and II remain available in their original typeset form. The original Part I pages 9–53 appear first. A new Version 47 supplement is then inserted. The original Part II and appendices, pages 54–102, follow without substantive alteration. Printed page numbers inside those facsimile pages retain their earlier numbering; the inserted supplement uses page labels of the form V47-1, V47-2, and so on. This arrangement preserves every self-awareness page exactly while making the new induction and ML-guided proof-search material independently auditable.

Reader's map

Location	Content
Original Part I, pp. 9–53	Formal systems, consequence and inference closure; SICs and ATPs; simulation; self-reference and recoding; heuristic proof search; theorem graphs; ground-up ATPs; proof counting; computability; Cook-Reckhow proof systems; algebraic proof systems; conclusion.
Version 47 Supplement	Structural induction on WFFs; proof induction; ATP stage and birth-time induction; invariant and potential methods; proof-count bounds and ML-guided proof search; the Predator 8.002 controlled experiment; exact finite reflection layers; the boundary at transfinite limit stages.
Original Part II, pp. 54–94	Internal representations; tags; self-descriptive formulas; self-models and self-theories; levels of formal self-awareness; the self-awareness pre-order and quotient; Tegmark; the Discovery Thesis; Picard; settlement of conjectures; integrated information.
Original Appendices, pp. 94–100	Examples of ATPs; Turing machines, computation, and SICs.
Original closing matter, pp. 101–102	Acknowledgements and bibliography.

Part I

Computers, Formal Systems, Simulations, and Automated Theorem Proving

Global notation for functions

Throughout this paper, if A and B are sets, then $[A \rightarrow B]$ denotes the set of all functions with domain A and codomain B . Thus $f \in [A \rightarrow B]$ says what is often written as $f : A \rightarrow B$. When a partial function is intended, I write $f \in [A \rightarrow_{\text{part}} B]$. This notation is especially useful below because SICs, inference rules, coding maps, and graph-search policies are all treated as mathematical objects belonging to appropriate function spaces.

When a recoding map is used later, I write $t[A] := \{t(a) : a \in A\}$ for the image of a set and $t^{-1}[B] := \{a : t(a) \in B\}$ for the inverse image. If $M = (C, H)$ is a SIC, the notation M^t means the recoded or translated copy obtained by transporting its states through the same invertible map t .

The intersection of nonstandard analysis and proof theory

Short version: nonstandard analysis supplies infinitesimal-style proofs; proof theory analyzes their logical strength and, where possible, converts them into constructive or computationally explicit standard mathematics.

The first clause means that nonstandard analysis permits a mathematician to argue with infinitesimals, infinitely large indices, hyperfinite objects, transfer principles, standard-part maps, or predicates such as “standard” inside a rigorous formal environment. An *infinitesimal-style proof* is a proof whose main line of reasoning uses such nonstandard objects or principles in place of, or as a compressed substitute for, ordinary epsilon-delta, compactness, limiting, or approximation arguments. This is not merely historical metaphor. V  th’s *Nonstandard Analysis*, published by Birkhaeuser/Springer, describes nonstandard analysis as a rigorous justification of infinitesimals such as df and dx in Leibniz-style calculus and as a general method in analysis, topology, and functional analysis [29]. In the language of this paper, such a proof may be regarded as a proof inside a nonstandard extension F^{ns} of a standard formal system F .

Proof theory analyzes the strength of an infinitesimal-style proof by asking what ordinary resources were actually used. If a nonstandard proof of an ordinary, internal, or standard statement uses transfer, saturation, idealization, choice, comprehension, or a standardness predicate, proof theory asks whether those principles are conservative over the standard base theory, whether they raise the proof-theoretic strength, and whether the final ordinary theorem was already provable in the original system F . In SIC language, this is a comparison between a nonstandard proof-search machine over Y^{ns} and a standard proof-search machine over Y : the question is whether the nonstandard search has merely shortened the route to an already standard target, or whether it has used genuinely stronger deductive resources.

Proof theory converts infinitesimal-style proofs into standard mathematics by proof interpretations, conservation theorems, negative translations, term extraction, and proof mining. The schematic form is this: from a proof $\Gamma \vdash_{F^{\text{ns}}} \varphi$ of a standard conclusion φ , one attempts to extract a standard proof, witness, algorithm, rate of convergence, modulus of uniformity, or numerical bound in the base system F or in a controlled conservative extension of F . Kohlenbach’s applied proof theory program is explicitly organized around proof interpretations and the extraction of

effective data, such as bounds, from *prima facie* ineffective proofs [13]. Van den Berg, Briseid, and Safarik give a directly relevant nonstandard example: functional interpretations can rewrite proofs in constructive and classical systems of nonstandard arithmetic into standard ones, and the same rewriting is connected with term extraction [28].

Here *constructive standard mathematics* means ordinary mathematics stated without nonstandard vocabulary and proved in a constructive base theory, so that existence proofs carry witness information rather than relying only on classical excluded-middle reasoning. *Computationally explicit standard mathematics* means ordinary mathematics stated without nonstandard vocabulary in which the output of the proof is not just “there exists an object” but an algorithm, term, bound, modulus, finite search procedure, or other data that can in principle be computed.

This fits the present paper’s depth-of-search framework as follows. A nonstandard proof can be treated as a search path in F^{ns} , while a proof interpretation acts first as an extraction or abstraction map from that path to a standard path in F . It qualifies as a synchronous deterministic simulation only when the induced state map is injective and exactly preserves stage evolution and halting observations. It qualifies as a clocked deterministic simulation when the same preservation holds after an admissible time rescaling. In either case, the comparison can be measured by proof length, SIC stage, or transaction count. If the extracted standard proof is much longer, then the nonstandard proof has reduced visible proof-search depth but not necessarily theoremhood. If the extracted proof also supplies a bound or algorithm, then the nonstandard argument has been converted into computationally explicit standard mathematics.

There is an important contrast with the self-reference section below. A proof interpretation from F^{ns} to F may be a content-extracting translation that loses some nonstandard vocabulary and is not simply an isomorphism. A recoding map, by contrast, is deliberately invertible. It is a translator between two presentations that are mathematically the same language up to isomorphism. Thus proof extraction studies how much standard content hides in a nonstandard proof, while recoding studies how a system can recognize a coded, renamed, or quoted copy of itself without changing theoremhood.

1 Formal Systems, Consequence, and Inference Closure

The proof-theoretic background is standard; see, for example, [16, 25, 2]. Let $\mathbf{N}^+$ denote the positive integers. For each $m \in \mathbf{N}^+$, let $F_m := \{1, \dots, m\}$. If x is a set, then x^m denotes the set of all functions from F_m to x , equivalently the set of ordered m -tuples from x .

Definition 1.1. Let y be a set. A k -ary rule on y is a partial function $r \in [y^k \rightarrow_{\text{part}} y]$, where $k \in \mathbf{N}^+$. A *finitary rule* on y is a k -ary rule on y for some $k \in \mathbf{N}^+$. If r is a k -ary rule, then $\text{ar}(r) := k$ is the arity of r .

Definition 1.2. A formal system is an ordered triple $F = (x, y, z)$ such that:

1. x is a nonempty set;
2. y is a set of finite strings over the alphabet x ;
3. z is a set of finitary rules on y .

The elements of x are the symbols of F , the elements of y are the well-formed formulas of F , and the elements of z are the inference rules of F . For each $k \in \mathbf{N}^+$, write $z_k(F) := \{r \in z : \text{ar}(r) = k\}$.

Definition 1.3. Let $F = (x, y, z)$ be a formal system. Let $\beta \in y$ and let $A \subseteq y$. Then β is a direct consequence of A if there exist $k \in \mathbf{N}^+$, a rule $r \in z_k(F)$, and a tuple $(a_1, \dots, a_k) \in A^k$ such that $r(a_1, \dots, a_k) = \beta$.

Definition 1.4. Let $F = (x, y, z)$ be a formal system and let $\Gamma \subseteq y$. A proof from Γ is a finite sequence $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ of members of y such that for each $j \in F_n$, either $\pi(j) \in \Gamma$, or $\pi(j)$ is a direct consequence of $\{\pi(1), \dots, \pi(j-1)\}$. The proof length of π is $|\pi| := n$.

Definition 1.5. If $F = (x, y, z)$, $\Gamma \subseteq y$, and $\psi \in y$, write $\Gamma \vdash_F \psi$ if and only if there exists a proof π from Γ such that $\pi(|\pi|) = \psi$.

Definition 1.6. For $\Gamma \subseteq y$, define the consequence set $\text{Con}_F(\Gamma) := \{\psi \in y : \Gamma \vdash_F \psi\}$.

Definition 1.7. A subset $Y \subseteq y$ is inference-closed if whenever $A \subseteq Y$ and β is a direct consequence of A , one has $\beta \in Y$.

Definition 1.8. For $A \subseteq y$, define $D_F(A) := A \cup \{\beta \in y : \beta \text{ is a direct consequence of } A\}$.

Proposition 1.9. *The consequence operator is extensive, monotone, and idempotent. More precisely:*

1. $\Gamma \subseteq \text{Con}_F(\Gamma)$ for every $\Gamma \subseteq y$;
2. if $A \subseteq B \subseteq y$, then $\text{Con}_F(A) \subseteq \text{Con}_F(B)$;
3. $\text{Con}_F(\text{Con}_F(\Gamma)) = \text{Con}_F(\Gamma)$ for every $\Gamma \subseteq y$.

Proof. For (1), each member of Γ has a one-line proof from Γ .

For (2), every proof from A is also a proof from B .

For (3), extensiveness and monotonicity give $\text{Con}_F(\Gamma) \subseteq \text{Con}_F(\text{Con}_F(\Gamma))$. For the reverse inclusion, let $\psi \in \text{Con}_F(\text{Con}_F(\Gamma))$. Then there is a proof of ψ from $\text{Con}_F(\Gamma)$. Every hypothesis line in that proof already has a proof from Γ . Replacing each such hypothesis line by a proof from Γ and preserving the rule applications yields a proof of ψ from Γ . Hence $\psi \in \text{Con}_F(\Gamma)$. $\square$

Theorem 1.10 (Induction principle on proofs). *Let $\Gamma \subseteq y$ and let $W \subseteq y$ satisfy $\Gamma \subseteq W$. If W is inference-closed, then $\text{Con}_F(\Gamma) \subseteq W$.*

Proof. Let $\psi \in \text{Con}_F(\Gamma)$. Choose a proof $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ from Γ with $\pi(n) = \psi$. I claim that every line of π belongs to W .

Proceed by induction on the line number. If $j = 1$, then $\pi(1) \in \Gamma \subseteq W$. Suppose now that $\pi(1), \dots, \pi(j-1) \in W$. If $\pi(j) \in \Gamma$, then $\pi(j) \in W$. Otherwise $\pi(j)$ is a direct consequence of $\{\pi(1), \dots, \pi(j-1)\}$, and all of those premises lie in W by the induction hypothesis. Since W is inference-closed, $\pi(j) \in W$.

Thus every line of π lies in W , so in particular $\psi \in W$. $\square$

Theorem 1.11. *For every $\Gamma \subseteq y$, the set $\text{Con}_F(\Gamma)$ is the least inference-closed subset of y containing Γ .*

Proof. By extensiveness, $\Gamma \subseteq \text{Con}_F(\Gamma)$.

To prove inference-closure, let $A \subseteq \text{Con}_F(\Gamma)$ and let β be a direct consequence of A . Since the defining rule uses only finitely many premises, there is a finite subset $A_0 \subseteq A$ witnessing the derivation of β . For each $a \in A_0$ choose a proof of a from Γ . Concatenate those finitely many proofs and append the final rule application producing β . Hence $\Gamma \vdash_F \beta$, so $\beta \in \text{Con}_F(\Gamma)$.

Now let $W \subseteq y$ contain Γ and be inference-closed. By the preceding induction principle, $\text{Con}_F(\Gamma) \subseteq W$. Therefore $\text{Con}_F(\Gamma)$ is the least inference-closed set containing Γ . $\square$

Corollary 1.12. *Let $Y \subseteq y$ be inference-closed, and let $\Gamma \subseteq Y$. Then $\text{Con}_F(\Gamma)$ is the least inference-closed subset of Y containing Γ .*

Proof. Since Y is inference-closed and $\Gamma \subseteq Y$, every proof from Γ stays inside Y . Hence $\text{Con}_F(\Gamma) \subseteq Y$, and the preceding theorem applies inside Y . $\square$

Proposition 1.13. *Let $W \subseteq y$. Then W is inference-closed if and only if $D_F(W) = W$.*

Proof. If W is inference-closed, then every direct consequence of W already lies in W , so $D_F(W) \subseteq W$. The reverse inclusion is immediate.

Conversely, if $D_F(W) = W$ and β is a direct consequence of W , then $\beta \in D_F(W) = W$. Thus W is inference-closed. $\square$

Theorem 1.14 (Practical closure tests). *Let $Y \subseteq y$.*

1. *Y is inference-closed if and only if for every $k \in \mathbf{N}^+$, every $r \in z_k(F)$, and every tuple $(a_1, \dots, a_k) \in Y^k$ with $r(a_1, \dots, a_k)$ defined, one has $r(a_1, \dots, a_k) \in Y$.*
2. *If P is a property of formulas such that $Y = \{\psi \in y : P(\psi)\}$ and every rule in z preserves P , then Y is inference-closed.*
3. *If $\{Y_i : i \in I\}$ is any family of inference-closed subsets of y , then $\bigcap_{i \in I} Y_i$ is inference-closed.*

Proof. For (1), unwind the definition of inference-closure. A direct consequence of Y is exactly a value $r(a_1, \dots, a_k)$ with $r \in z_k(F)$ and $(a_1, \dots, a_k) \in Y^k$.

For (2), if $\beta = r(a_1, \dots, a_k)$ with all $a_i \in Y$, then $P(a_i)$ holds for each i , so preservation gives $P(\beta)$, hence $\beta \in Y$.

For (3), let β be a direct consequence of $\bigcap_{i \in I} Y_i$. Then β is a direct consequence of each Y_i , so $\beta \in Y_i$ for every i . Therefore $\beta \in \bigcap_{i \in I} Y_i$. $\square$

Remark 1.15. The point of Theorem 1.14 is practical. Even when Y is countably infinite, one does not verify closure by checking infinitely many premises at once. Every rule application is finitary, so one fixes an arbitrary rule and an arbitrary finite tuple of premises from Y , and one proves that the conclusion remains in Y .

2 Semi-Ideal Computers, Shortest Proofs, and Canonical Search

Fix a formal system $F = (x, y, z)$.

Definition 2.1. Let $\Gamma \subseteq y$ and $\varphi \in y$. Define $\|\varphi\|_F(\Gamma) := \min\{|\pi| : \pi \text{ is a proof from } \Gamma \text{ with final line } \varphi\}$, if $\Gamma \vdash_F \varphi$, and define $\|\varphi\|_F(\Gamma) := \infty$ otherwise.

Definition 2.2. For $\Gamma \subseteq y$ and $m \in \mathbf{N}^+$, define the exact shortest-proof layer $L_F(\Gamma, m) := \{\varphi \in y : \|\varphi\|_F(\Gamma) = m\}$.

Definition 2.3. Let $Y \subseteq y$. A semi-ideal computer over Y , abbreviated SIC over Y , is a pair $M = (C, H)$ such that:

1. $C \in [\mathcal{P}(Y) \times \mathbf{N}^+ \rightarrow \mathcal{P}(Y)]$;
2. $H \in [\mathcal{P}(Y) \times \mathbf{N}^+ \rightarrow \{0, 1\}]$;
3. $C(\Gamma, 1) = \Gamma$ for all $\Gamma \subseteq Y$;

4. $C(\Gamma, m) \subseteq C(\Gamma, m+1)$ for all $\Gamma \subseteq Y$ and $m \in \mathbf{N}^+$;
5. if $H(\Gamma, m) = 1$, then $H(\Gamma, m+1) = 1$;
6. if $H(\Gamma, m) = 1$, then $C(\Gamma, m+1) = C(\Gamma, m)$.

If $Y = y$, I simply say SIC.

Definition 2.4. For a SIC $M = (C, H)$ over Y and an input $\Gamma \subseteq Y$, the stage- m state of M on Γ is $C(\Gamma, m)$. The state space of M is $S_M := C[\mathcal{P}(Y) \times \mathbf{N}^+]$.

Definition 2.5. A SIC $M = (C, H)$ over Y is deductively effective if $\text{Con}_F(\Gamma) \cap Y = \bigcup_{m \in \mathbf{N}^+} C(\Gamma, m)$ for every $\Gamma \subseteq Y$.

Definition 2.6. A SIC $M = (C, H)$ over Y is deductively finitely effective if for every $\Gamma \subseteq Y$ there exists $m(\Gamma) \in \mathbf{N}^+$ such that $\text{Con}_F(\Gamma) \cap Y = \bigcup_{j=1}^{m(\Gamma)} C(\Gamma, j)$.

Definition 2.7. An automated theorem prover, abbreviated ATP, over Y is a SIC over Y intended to enumerate theorems or search for designated targets. An enumerating ATP is a deductively effective SIC.

Definition 2.8. Fix $\varphi \in Y$. A target-search ATP for φ is a SIC $M = (C, H)$ over Y such that:

1. if $H(\Gamma, m) = 1$, then $\varphi \in C(\Gamma, m)$;
2. if $\Gamma \vdash_F \varphi$, then $H(\Gamma, m) = 1$ for some $m \in \mathbf{N}^+$.

Definition 2.9. A SIC $M = (C, H)$ over Y is rule-driven if whenever $\beta \in C(\Gamma, m+1) \setminus C(\Gamma, m)$, the formula β is a direct consequence of $C(\Gamma, m)$.

Definition 2.10. Let $Y \subseteq y$. The canonical breadth-first SIC over Y is the SIC $B_{F,Y} = (C_{F,Y}^{\text{bf}}, H_{F,Y}^{\text{bf}})$ defined by $C_{F,Y}^{\text{bf}}(\Gamma, m) := \{\psi \in Y : \|\psi\|_F(\Gamma) \leq m\}$, and $H_{F,Y}^{\text{bf}}(\Gamma, m) := 0$ for all $\Gamma \subseteq Y$ and $m \in \mathbf{N}^+$.

Definition 2.11. Let $Y \subseteq y$ and let $\varphi \in Y$. The canonical breadth-first target-search SIC for φ is $B_{F,Y,\varphi} = (C_{F,Y,\varphi}^{\text{bf}}, H_{F,Y,\varphi}^{\text{bf}})$, where $C_{F,Y,\varphi}^{\text{bf}}(\Gamma, m) := \{\psi \in Y : \|\psi\|_F(\Gamma) \leq m\}$, and $H_{F,Y,\varphi}^{\text{bf}}(\Gamma, m) = 1 \iff \|\varphi\|_F(\Gamma) \leq m$.

Theorem 2.12 (Proof-Horizon Theorem). *Let $Y \subseteq y$, let $\Gamma \subseteq Y$, and let $\varphi \in Y$.*

1. *The SIC $B_{F,Y}$ is deductively effective, and for every $m \in \mathbf{N}^+$ one has $C_{F,Y}^{\text{bf}}(\Gamma, m) = \bigcup_{j=1}^m (L_F(\Gamma, j) \cap Y)$.*
2. *The target-search SIC $B_{F,Y,\varphi}$ halts on input Γ if and only if $\Gamma \vdash_F \varphi$, and in that case the first halting stage is exactly $\|\varphi\|_F(\Gamma)$.*

Proof. For (1), the displayed identity is immediate from the definition of $C_{F,Y}^{\text{bf}}$. Taking the union over all m yields $\bigcup_{m \in \mathbf{N}^+} C_{F,Y}^{\text{bf}}(\Gamma, m) = \{\psi \in Y : \|\psi\|_F(\Gamma) < \infty\} = \text{Con}_F(\Gamma) \cap Y$. Thus $B_{F,Y}$ is deductively effective.

For (2), by definition $H_{F,Y,\varphi}^{\text{bf}}(\Gamma, m) = 1$ exactly when $\|\varphi\|_F(\Gamma) \leq m$. Hence some halting stage exists if and only if $\|\varphi\|_F(\Gamma) < \infty$, that is, if and only if $\Gamma \vdash_F \varphi$. The least such stage is exactly $\|\varphi\|_F(\Gamma)$. $\square$

Corollary 2.13. *Let $Y \subseteq y$, let $\Gamma \subseteq Y$, and let $\varphi \in Y$ with $\Gamma \vdash_F \varphi$. Then the transaction count of the breadth-first target-search SIC satisfies $\tau_{B_{F,Y,\varphi}}(\Gamma, \varphi) = \|\varphi\|_F(\Gamma)$, where τ is defined in Section 7.*

Proof. This is just part (2) of Theorem 2.12 rewritten in the notation of target-search transaction counts. $\square$

Remark 2.14. Theorem 2.12 is the first of the paper's three main imaginative theorems. It turns shortest proof length into a directly observable SIC stage parameter. Later sections use this fact twice: first to compare heuristic branches with canonical breadth-first search, and then to interpret speedup as the deliberate lowering of proof horizons.

Proposition 2.15. *Let $Y \subseteq y$ be inference-closed. If $M = (C, H)$ is a rule-driven SIC over Y , then $C(\Gamma, m) \subseteq \text{Con}_F(\Gamma) \cap Y$ for every $\Gamma \subseteq Y$ and every $m \in \mathbf{N}^+$.*

Proof. Fix $\Gamma \subseteq Y$. Proceed by induction on m . The case $m = 1$ is immediate because $C(\Gamma, 1) = \Gamma \subseteq \text{Con}_F(\Gamma) \cap Y$.

Assume the claim at stage m and let $\beta \in C(\Gamma, m + 1)$. If $\beta \in C(\Gamma, m)$, use the induction hypothesis. Otherwise β is a direct consequence of $C(\Gamma, m)$ because M is rule-driven. Every premise used in that rule application already belongs to $\text{Con}_F(\Gamma)$ by the induction hypothesis, so one more inference step yields $\Gamma \vdash_F \beta$. Also $\beta \in Y$ because every stage state lies in $\mathcal{P}(Y)$. $\square$

Definition 2.16. Let $Y \subseteq y$ be inference-closed. The canonical rule-enumeration SIC over Y is the machine $E_{F,Y} = (C_{F,Y}, H_{F,Y})$ defined by $C_{F,Y}(\Gamma, 1) = \Gamma$, $C_{F,Y}(\Gamma, m + 1) = D_F(C_{F,Y}(\Gamma, m))$, and $H_{F,Y}(\Gamma, m) = 0$ for all $\Gamma \subseteq Y$ and $m \in \mathbf{N}^+$.

Theorem 2.17. *Let $Y \subseteq y$ be inference-closed. Then $E_{F,Y}$ is a rule-driven enumerating ATP over Y , and it is deductively effective.*

Proof. The SIC axioms are immediate: stage one is the input, monotonicity follows from $A \subseteq D_F(A)$, and all states stay inside Y because Y is inference-closed. The machine is rule-driven by construction.

Fix $\Gamma \subseteq Y$ and set $W := \bigcup_{m \in \mathbf{N}^+} C_{F,Y}(\Gamma, m)$. Clearly $\Gamma \subseteq W$. Also W is inference-closed: if β is a direct consequence of W , only finitely many premises are used, so all of those premises already occur in some common stage $C_{F,Y}(\Gamma, m)$. Hence $\beta \in D_F(C_{F,Y}(\Gamma, m)) = C_{F,Y}(\Gamma, m + 1) \subseteq W$. Therefore $\text{Con}_F(\Gamma) \subseteq W$ by the least-closure theorem of Section 1.

For the reverse inclusion, every stage lies in $\text{Con}_F(\Gamma) \cap Y$ by the preceding proposition. Taking unions yields $W \subseteq \text{Con}_F(\Gamma) \cap Y$. Hence $W = \text{Con}_F(\Gamma) \cap Y$, so $E_{F,Y}$ is deductively effective. $\square$

Remark 2.18. Definition 2.16 is, rhetorically, a formal description of brute-force proof search. The canonical rule-enumeration SIC repeatedly adjoins every direct consequence available at the current stage. It does not prioritize, prune, or guess; it simply expands by exhaustive local closure.

Definition 2.19. Let $Y \subseteq y$ be inference-closed and let $\varphi \in Y$. The canonical target-search SIC over Y for φ is $E_{F,Y,\varphi} = (C_{F,Y,\varphi}, H_{F,Y,\varphi})$, where $C_{F,Y,\varphi}(\Gamma, 1) = \Gamma$, and for $m \in \mathbf{N}^+$, if $\varphi \in C_{F,Y,\varphi}(\Gamma, m)$ then $C_{F,Y,\varphi}(\Gamma, m + 1) = C_{F,Y,\varphi}(\Gamma, m)$, while otherwise $C_{F,Y,\varphi}(\Gamma, m + 1) = D_F(C_{F,Y,\varphi}(\Gamma, m))$. Also, $H_{F,Y,\varphi}(\Gamma, m) = 1 \iff \varphi \in C_{F,Y,\varphi}(\Gamma, m)$.

Theorem 2.20. *Let $Y \subseteq y$ be inference-closed and let $\varphi \in Y$. Then $E_{F,Y,\varphi}$ is a target-search ATP for φ .*

Proof. The SIC axioms follow directly from the definition. If $H_{F,Y,\varphi}(\Gamma, m) = 1$, then $\varphi \in C_{F,Y,\varphi}(\Gamma, m)$ by definition.

Conversely, suppose $\Gamma \vdash_F \varphi$. By Theorem 2.17, the formula φ appears at some stage of the enumerating search from Γ . The target-search SIC follows the same search until the first stage at which φ appears and then freezes. Hence $H_{F,Y,\varphi}(\Gamma, m) = 1$ for some $m \in \mathbf{N}^+$. $\square$

3 Weak, Synchronous, Clocked, and Effective Simulation

The original relational notion is retained as a boundary case. The exact stage-for-stage functional notion is retained as a strong faithful embedding. The principal comparison relation is clocked deterministic simulation, which allows a simulator to run faster, slower, or with idle stages while preserving states and halting observations at corresponding times.

Definition 3.1. Let $M = (C_M, H_M)$ be a SIC over $Y_M \subseteq y$, and let $N = (C_N, H_N)$ be a SIC over $Y_N \subseteq y$. A relation $R \subseteq S_M \times S_N$ is a *weak SIC-simulation witness* from M to N if:

1. for every $(p, q) \in R$ and every $m \in \mathbf{N}^+$, there exists $n \in \mathbf{N}^+$ such that $(C_M(p, m), C_N(q, n)) \in R$;
2. for every $\Gamma \subseteq Y_M$, there exists $\Delta \subseteq Y_N$ such that $(C_M(\Gamma, 1), C_N(\Delta, 1)) \in R$.

If such a relation exists, write $M \preceq_w N$ and say that M *weakly simulates into* N .

Theorem 3.2 (Collapse of weak simulation). *Every SIC weakly simulates into every other SIC.*

Proof. Let M and N be SICs and take $R := S_M \times S_N$. If $(p, q) \in R$ and $m \in \mathbf{N}^+$, choose $n = 1$. Then $C_M(p, m) \in S_M$ and $C_N(q, 1) = q \in S_N$, so $(C_M(p, m), C_N(q, 1)) \in R$. The anchoring condition is also immediate because every pair of initial states belongs to R . $\square$

Corollary 3.3. *Weak simulation has a single equivalence class. It cannot support a substantive theory of simulation depth.*

Proposition 3.4 (State-space saturation). *If M is a SIC over Y , then $S_M = \mathcal{P}(Y)$.*

Proof. By definition $S_M \subseteq \mathcal{P}(Y)$. Conversely, for every $\Gamma \subseteq Y$, the stage-one axiom gives $C_M(\Gamma, 1) = \Gamma$. Hence every subset of Y belongs to S_M . $\square$

3.1 Synchronous deterministic simulation

Definition 3.5. A *synchronous deterministic simulation* from M to N is a function

$$f \in [S_M \rightarrow S_N]$$

such that:

1. f is injective;
2. for every $p \in S_M$ and every $m \in \mathbf{N}^+$,

$$f(C_M(p, m)) = C_N(f(p), m);$$

3. for every $p \in S_M$ and every $m \in \mathbf{N}^+$,

$$H_M(p, m) = H_N(f(p), m).$$

If such an f exists, write $M \preceq_{\text{syn}} N$.

Remark 3.6. This is a faithful synchronous state embedding. It is intentionally stronger than behavioral similarity: injectivity preserves every source-state distinction, and the same stage parameter appears on both sides. The earlier term “observable simulation” is avoided because the definition preserves the entire SIC state rather than merely a separately specified observation.

Proposition 3.7. *Every synchronous deterministic simulation gives a weak simulation. If $f \in [S_M \rightarrow S_N]$ witnesses $M \preceq_{\text{syn}} N$, then its graph $R_f := \{(p, f(p)) : p \in S_M\}$ is a weak simulation witness.*

Proof. If $(p, f(p)) \in R_f$, then for every m the synchronous evolution equation gives $(C_M(p, m), C_N(f(p), m)) = (C_M(p, m), f(C_M(p, m))) \in R_f$. For anchoring, use $(\Gamma, f(\Gamma))$ and Proposition 3.4. $\square$

Proposition 3.8. *For every SIC M , the identity function $\text{id}_{S_M} \in [S_M \rightarrow S_M]$ witnesses $M \preceq_{\text{syn}} M$.*

Proof. The identity is injective and preserves both C_M and H_M exactly. $\square$

Proposition 3.9. *If $M \preceq_{\text{syn}} N$ via $f \in [S_M \rightarrow S_N]$ and $N \preceq_{\text{syn}} P$ via $g \in [S_N \rightarrow S_P]$, then $M \preceq_{\text{syn}} P$ via $g \circ f$.*

Proof. The composite is injective. For every p and m ,

$$(g \circ f)(C_M(p, m)) = g(C_N(f(p), m)) = C_P(g(f(p)), m),$$

and halting preservation composes in the same way. $\square$

Corollary 3.10. *The relation $\preceq_{\text{syn}}$ is a preorder. Write $M \sim_{\text{syn}} N$ when both directions hold; the quotient by $\sim_{\text{syn}}$ is partially ordered.*

3.2 Tests of synchronous over-rigidity

Proposition 3.11 (Cardinality obstruction). *If $M \preceq_{\text{syn}} N$, then there is an injection $\mathcal{P}(Y_M) \rightarrow \mathcal{P}(Y_N)$. If Y_M and Y_N are finite, then $|Y_M| \leq |Y_N|$.*

Proof. By Proposition 3.4, the witness is an injection $\mathcal{P}(Y_M) \rightarrow \mathcal{P}(Y_N)$. In the finite case, $2^{|Y_M|} \leq 2^{|Y_N|}$, hence $|Y_M| \leq |Y_N|$. $\square$

Example 3.12. Let $Y_M = \{a, b\}$ and $Y_N = \{c\}$. Let both SICs be static and never halting:

$$C_M(p, m) = p, \quad H_M(p, m) = 0, \quad C_N(q, m) = q, \quad H_N(q, m) = 0.$$

Their evolution and halting laws have the same form, but $|S_M| = 4 > |S_N| = 2$. Thus $M \not\preceq_{\text{syn}} N$. Synchronous simulation distinguishes full internal state capacity, not merely the form of the dynamics.

Theorem 3.13 (One-stage-delay obstruction). *There are SICs performing the same update with a one-stage delay that are incomparable under synchronous deterministic simulation.*

Proof. Let $Y = \{a\}$ and let both halting maps be constantly zero. Define M by

$$C_M(\Gamma, 1) = \Gamma, \quad C_M(\Gamma, m) = \Gamma \cup \{a\} \quad (m \geq 2),$$

and define N by

$$C_N(\Gamma, 1) = \Gamma, \quad C_N(\Gamma, 2) = \Gamma, \quad C_N(\Gamma, m) = \Gamma \cup \{a\} \quad (m \geq 3).$$

Both satisfy the SIC axioms. Suppose first that $f \in [S_M \rightarrow S_N]$ witnesses $M \preceq_{\text{syn}} N$. At stage 2 and input $\emptyset$,

$$f(\{a\}) = f(C_M(\emptyset, 2)) = C_N(f(\emptyset), 2) = f(\emptyset),$$

contradicting injectivity. Conversely, if f witnessed $N \preceq_{\text{syn}} M$, then at stage 2,

$$f(p) = f(C_N(p, 2)) = C_M(f(p), 2) = f(p) \cup \{a\} = \{a\}$$

for every $p \in S_N$, where the last equality holds because $f(p) \subseteq \{a\}$; again this contradicts injectivity, since $|S_N| = 2$. $\square$

3.3 Clocked deterministic simulation

Definition 3.14. An *admissible clock* is a function

$$\rho \in [\mathbf{N}^+ \rightarrow \mathbf{N}^+]$$

such that $\rho(1) = 1$, ρ is nondecreasing, and ρ is unbounded. Repeated values permit the source to take an idle stage relative to the target; skipped values permit the target to take extra stages between represented source stages.

Definition 3.15. A *clocked deterministic simulation* from M to N is a pair (f, ρ) such that $f \in [S_M \rightarrow S_N]$ is injective, ρ is an admissible clock, and for every $p \in S_M$ and $m \in \mathbf{N}^+$,

$$f(C_M(p, m)) = C_N(f(p), \rho(m))$$

and

$$H_M(p, m) = H_N(f(p), \rho(m)).$$

If such a pair exists, write $M \preceq_{\text{clk}} N$. Because this is the principal simulation relation in the remainder of the paper, the unadorned notation $M \preceq N$ abbreviates $M \preceq_{\text{clk}} N$.

Remark 3.16. Clocked deterministic simulation retains faithful injective state preservation but removes the artificial requirement that corresponding computations use the same stage number. It repairs timing rigidity, not cardinality rigidity. Lossy many-to-one comparisons remain state abstractions rather than simulations.

Proposition 3.17. If $M \preceq_{\text{syn}} N$, then $M \preceq N$.

Proof. Use the identity clock $\rho(m) = m$. $\square$

Proposition 3.18. For every SIC M , the pair $(\text{id}_{S_M}, \text{id}_{\mathbf{N}^+})$ witnesses $M \preceq M$.

Proposition 3.19 (Composition of clocked simulations). Suppose $M \preceq N$ via (f, ρ) and $N \preceq P$ via (g, σ) . Then $M \preceq P$ via

$$(g \circ f, \sigma \circ \rho).$$

Proof. The composite state map is injective. The composite clock sends 1 to 1, is nondecreasing, and is unbounded. Moreover,

$$(g \circ f)(C_M(p, m)) = g(C_N(f(p), \rho(m))) = C_P(g(f(p)), \sigma(\rho(m))),$$

and the halting equations compose identically. $\square$

Corollary 3.20. *The principal clocked deterministic simulation relation $\preceq$ is a preorder on SICs.*

Definition 3.21. Write $M \sim N$ if and only if $M \preceq N$ and $N \preceq M$.

Corollary 3.22. *The relation $\sim$ is an equivalence relation, and $\preceq$ induces a partial order on the set of $\sim$ -equivalence classes.*

Theorem 3.23 (Clocked repair of the delay example). *The SICs M and N from Theorem 3.13 satisfy $M \sim N$.*

Proof. For $M \preceq N$, use $f = \text{id}$ and

$$\rho(1) = 1, \quad \rho(m) = m + 1 \quad (m \geq 2).$$

For $N \preceq M$, use $f = \text{id}$ and

$$\sigma(1) = 1, \quad \sigma(2) = 1, \quad \sigma(m) = m - 1 \quad (m \geq 3).$$

Both clocks are admissible, and direct substitution verifies the state and halting equations. $\square$

3.4 Effective witnesses

The notation $[A \rightarrow B]$ denotes all functions, so neither synchronous nor clocked simulation requires a computable witness. This is appropriate for a set-theoretic preorder, but executable comparison requires a refinement.

Definition 3.24. An *effective presentation* of a SIC M is an injective coding $e_M \in [S_M \rightarrow \mathbf{N}]$ with decidable image such that the coded stage and halting maps are total computable functions. Given effective presentations of M and N , a clocked simulation (f, ρ) is *effective* if ρ is computable and the induced map

$$e_N \circ f \circ e_M^{-1}$$

is computable on $e_M[S_M]$. Write $M \preceq_{\text{eff}} N$ when such a witness exists.

Remark 3.25. By state-space saturation, a full SIC over a countably infinite Y has state space $\mathcal{P}(Y)$ and therefore has no coding of all states into $\mathbf{N}$. The effective definition applies directly to finite SICs and to other countably presented state spaces. Infinite executable applications should specify a restricted effective state domain, such as finite or reachable states, rather than silently coding the full power set.

Proposition 3.26. *For fixed effective presentations, $\preceq_{\text{eff}}$ is a preorder and implies $\preceq$.*

Proof. The identity maps and identity clock are computable. Computable state maps and clocks are closed under composition, and Proposition 3.19 provides the underlying clocked simulation. $\square$

Theorem 3.27 (Noncomputable simulation witnesses exist). *The set-theoretic synchronous simulation definition permits witnesses that are not computable even on finitely described singleton states.*

Proof. Let $K \subseteq \mathbf{N}$ be noncomputable. Define a permutation π_K of $\mathbf{N}$ by swapping $2e$ and $2e + 1$ exactly when $e \in K$:

$$\pi_K(2e) = \begin{cases} 2e + 1, & e \in K, \\ 2e, & e \notin K, \end{cases} \quad \pi_K(2e + 1) = \begin{cases} 2e, & e \in K, \\ 2e + 1, & e \notin K. \end{cases}$$

Let I be the static never-halting SIC over $Y = \mathbf{N}$, and define $f_K \in [S_I \rightarrow S_I]$ by $f_K(A) = \pi_K[A]$. The map is bijective and satisfies

$$f_K(C_I(A, m)) = f_K(A) = C_I(f_K(A), m), \quad H_I(A, m) = H_I(f_K(A), m) = 0.$$

Thus f_K is a synchronous self-simulation witness. But from $f_K(\{2e\})$ one can determine whether $e \in K$, so no algorithm computes this witness on singleton inputs. $\square$

Remark 3.28. The identity is also a computable self-simulation of I . The theorem does not claim that the pair is related only by noncomputable witnesses. It shows that effectivity is not automatic at the witness level and must be stated when it matters.

3.5 Canonical state abstractions

Definition 3.29. A *state abstraction* from a SIC M to a SIC N is a function $a \in [S_M \rightarrow S_N]$ satisfying

$$a(C_M(p, m)) = C_N(a(p), m)$$

for every $p \in S_M$ and every $m \in \mathbf{N}^+$. A state abstraction need not be injective and need not preserve halting observations. It is therefore not called a simulation in this paper.

Theorem 3.30 (Canonical closure-abstraction theorem). *Let $Y \subseteq y$ be inference-closed, and let $M = (C, H)$ be a rule-driven SIC over Y . Define*

$$a_M \in [S_M \rightarrow S_{E_{F,Y}}] \quad \text{by} \quad a_M(p) := \text{Con}_F(p) \cap Y.$$

Then a_M is a state abstraction from M to $E_{F,Y}$.

Proof. The map is well-defined because Proposition 3.4 gives $S_{E_{F,Y}} = \mathcal{P}(Y)$. Fix $p \in S_M$ and $m \in \mathbf{N}^+$. Since $a_M(p)$ is inference-closed in Y , the canonical rule-enumeration SIC started at $a_M(p)$ remains fixed:

$$C_{F,Y}(a_M(p), m) = a_M(p).$$

Because M is rule-driven, its soundness proposition gives $C(p, m) \subseteq \text{Con}_F(p) \cap Y$. By monotonicity and idempotence,

$$\text{Con}_F(C(p, m)) \cap Y = \text{Con}_F(p) \cap Y.$$

Hence

$$a_M(C(p, m)) = C_{F,Y}(a_M(p), m).$$

$\square$

Remark 3.31. The map a_M can identify distinct states that have the same consequence closure, and $E_{F,Y}$ has a constantly zero halting bit. It is therefore a useful abstraction but need not be a synchronous, clocked, or effective simulation.

Theorem 3.32 (Target-search closure-abstraction theorem). *Let $Y \subseteq y$ be inference-closed, let $\varphi \in Y$, and let $M = (C, H)$ be a rule-driven target-search ATP over Y . Define*

$$a_{M,\varphi}(p) := \text{Con}_F(p) \cap Y.$$

Then $a_{M,\varphi} \in [S_M \rightarrow S_{E_{F,Y,\varphi}}]$ is a state abstraction.

Proof. If $q = \text{Con}_F(p) \cap Y$, then q is inference-closed. Whether or not $\varphi \in q$, the canonical target-search SIC remains at q . The same closure argument as in Theorem 3.30 gives

$$a_{M,\varphi}(C(p, m)) = C_{F,Y,\varphi}(a_{M,\varphi}(p), m).$$

□

Remark 3.33. The same distinction applies to proof extraction from nonstandard analysis. A proof interpretation is always an extraction or abstraction map. It becomes a synchronous deterministic simulation only when it preserves exact stages and halting through an injective state map, and it becomes a clocked deterministic simulation when those properties hold after an admissible time rescaling.

3.6 Finite-horizon comparison

Definition 3.34. Let $K \in \mathbf{N}^+$. A *depth- K clocked deterministic simulation* from M to N is an injective function $f \in [S_M \rightarrow S_N]$ together with a nondecreasing function

$$\rho_K \in [\{1, \dots, K\} \rightarrow \mathbf{N}^+]$$

such that $\rho_K(1) = 1$ and, for every $p \in S_M$ and $m \leq K$,

$$f(C_M(p, m)) = C_N(f(p), \rho_K(m)) \quad \text{and} \quad H_M(p, m) = H_N(f(p), \rho_K(m)).$$

It is *synchronous* when $\rho_K(m) = m$ for every $m \leq K$.

Remark 3.35. Finite-horizon simulation is useful for bounded experiments but is not used for the full preorder and quotient results, because composition requires compatible target horizons.

4 Self-reference, recoding, and translated copies of SICs

This section gives a formal treatment of the motivating question: can a SIC speak about, prove things about, or verify a translated copy of itself? The answer is yes, but only after separating three levels. First, there is the ordinary mathematical object $M = (C, H)$. Second, there is a code, name, or quotation of that object inside some formal language. Third, there is a proof-search SIC whose states contain formulas of that language. The safe form of self-reference is therefore not an unrestricted claim that a machine contains its own truth predicate. It is the more controlled claim that a formal system can code syntax, can define an invertible recoding map on that syntax, and can prove syntactic facts about the recoded code of a machine.

Definition 4.1. A coding universe is a set U whose elements are codes for the objects currently under discussion: symbols, finite strings, well-formed formulas, inference rules, finite proofs, SICs, ATPs, weak simulation witnesses, synchronous simulation maps, clocked simulation pairs, and effective simulation witnesses. A uniform recoding on U is a bijection $t \in [U \rightarrow U]$. For $A \subseteq U$, write $t[A] = \{t(a) : a \in A\}$.

Remark 4.2. The phrase “uniform” is meant to capture the idea that symbols, formulas, and rules are not recoded by unrelated tricks. There is one invertible operation at the level of codes, and the recoding of larger objects is induced by applying that same operation to their constituents or to their defining graphs.

Definition 4.3. Let $F = (x, y, z)$ be a formal system whose symbols, formulas, and rules are coded in U . A syntactic recoding for F is a uniform recoding t such that $t[x]$ is an alphabet, $t[y]$ is the corresponding set of recoded well-formed formulas, and each k -ary rule $r \in z$ has a recoded rule r^t defined by $r^t(t(a_1), \dots, t(a_k)) = t(r(a_1, \dots, a_k))$ whenever $r(a_1, \dots, a_k)$ is defined. The recoded formal system is $F^t := (x^t, y^t, z^t)$, where $x^t = t[x]$, $y^t = t[y]$, and $z^t = \{r^t : r \in z\}$.

Definition 4.4. The map t is a translator between F and F^t . Since t is invertible, F and F^t are not two essentially different languages. They are two presentations of one language up to isomorphism.

Proposition 4.5 (Where recoded copies live). *Let t be a syntactic recoding for $F = (x, y, z)$, and let $\varphi \in y$.*

1. *The recoded copy $t(\varphi)$ is an element of $y^t = t[y]$, the set of well-formed formulas of the recoded formal system F^t , and $y^t \subseteq U$. The existence of $t(\varphi)$ therefore requires nothing beyond the coding universe itself.*
2. *In general $t(\varphi) \notin y$: the formula sets y and y^t are two subsets of U related by the bijection t , and nothing in the definition places one inside the other.*
3. *One has $t(\varphi) \in y$ for every $\varphi \in y$ if and only if $t[y] \subseteq y$. Call such a t an endorecoding of F . If moreover y is finite, then $t[y] = y$, so an endorecoding of a finite system permutes its own formulas.*

Proof. Claims (1) and (2) restate the definition of a syntactic recoding: y^t is defined as the image $t[y]$, a subset of U . For (3), the displayed condition is exactly $t[y] \subseteq y$; if y is finite, injectivity of t gives $|t[y]| = |y|$, and a subset of y of full cardinality equals y . $\square$

Remark 4.6 (One mechanism, one vocabulary). The proposition answers a question every self-reference framework should be asked: a formula $\varphi \in y$ does not automatically have a recoded copy inside y . What always exists is $t(\varphi)$ inside U , and it lands back inside y exactly in the endorecoding case. This is also why, as of this version, the paper uses a single word family. A recoding is nothing over and above a change of coding: if the objects under discussion are coded by an injection e into U and t is a recoding, then $t \circ e$ is another coding of the very same objects, and the “recoded copy” of an object is that same object presented under the coding $t \circ e$. Objects never change; codes do. Through Version 43 recodings were called tags; the older word survives only in labels and in sentences describing the renaming.

Theorem 4.7 (Formal-system transport theorem). *If $F = (x, y, z)$ is a formal system and t is a syntactic recoding for F , then F^t is a formal system. Moreover, t is an isomorphism from F to F^t : it sends symbols to symbols, formulas to formulas, and rule applications in F exactly to rule applications in F^t .*

Proof. By definition, x^t is the recoded alphabet and y^t is the set of recoded well-formed formulas. If $r \in z$ is k -ary, then r^t is a partial function from $(y^t)^k$ to y^t . Thus z^t is a set of finitary rules on y^t , so F^t is a formal system. The defining equation for r^t says exactly that t preserves and reflects rule applications. $\square$

Theorem 4.8 (Proof transport and proof-length invariance). *Let F be a formal system, let t be a syntactic recoding for F , let $\Gamma \subseteq y$, and let $\varphi \in y$. Then $\Gamma \vdash_F \varphi$ if and only if $t[\Gamma] \vdash_{F^t} t(\varphi)$. When these equivalent conditions hold, shortest proof length is preserved: $\|t(\varphi)\|_{F^t}(t[\Gamma]) = \|\varphi\|_F(\Gamma)$.*

Proof. If $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ is a proof of φ from Γ , then $t(\pi) := \langle t(\pi(1)), \dots, t(\pi(n)) \rangle$ is a proof of $t(\varphi)$ from $t[\Gamma]$, because each hypothesis line and each rule application is transported by Theorem 4.7. Applying t^{-1} gives the converse. The two maps $\pi \mapsto t(\pi)$ and $\rho \mapsto t^{-1}(\rho)$ preserve the number of proof lines, so the minimum possible proof length is the same on both sides. $\square$

Definition 4.9. Let $M = (C, H)$ be a SIC over Y , and let t be a uniform recoding with $Y^t = t[Y]$. The recoded SIC $M^t = (C^t, H^t)$ over Y^t is defined by $C^t(\Delta, m) := t[C(t^{-1}[\Delta], m)]$ and $H^t(\Delta, m) := H(t^{-1}[\Delta], m)$ for every $\Delta \subseteq Y^t$ and every $m \in \mathbf{N}^+$.

Theorem 4.10 (SIC transport theorem). *If $M = (C, H)$ is a SIC over Y , then M^t is a SIC over Y^t . If M is deductively effective, deductively finitely effective, rule-driven, an enumerating ATP, or a target-search ATP for φ , then M^t has the corresponding property over Y^t , with target $t(\varphi)$.*

Proof. The stage-one condition is preserved because $C^t(\Delta, 1) = t[C(t^{-1}[\Delta], 1)] = t[t^{-1}[\Delta]] = \Delta$. Monotonicity is preserved because images under t preserve inclusion. The two halting axioms are preserved because H^t is defined by pulling the input back along t^{-1} , applying H , and then transporting the state forward again. Thus M^t is a SIC.

For deductive and ATP properties, use Theorem 4.8. For rule-driven behavior, a new recoded formula at stage $m + 1$ has the form $t(\beta)$, where β was a new formula in the original machine and hence a direct consequence of the previous original state. The recoded rule then makes $t(\beta)$ a direct consequence of the previous recoded state. $\square$

Corollary 4.11 (Transaction invariance under recoding). *If M is a target-search ATP for φ , then $\tau_{M^t}(t[\Gamma], t(\varphi)) = \tau_M(\Gamma, \varphi)$ whenever either side is defined, with both sides equal to ∞ when the target is not reached.*

Proof. For every stage m , one has $t(\varphi) \in C^t(t[\Gamma], m)$ if and only if $\varphi \in C(\Gamma, m)$. Therefore the first successful stage is the same. $\square$

Theorem 4.12 (Recoded copies are synchronously and clockedly simulation-equivalent). *Let M be a SIC over Y . Then $M \sim_{\text{syn}} M^t$. Consequently, $M \sim M^t$ under the principal clocked relation.*

Proof. Define

$$f_t \in [S_M \rightarrow S_{M^t}] \quad \text{by} \quad f_t(p) := t[p].$$

Because t is bijective, f_t is bijective and hence injective. For every $p \in S_M$ and every $m \in \mathbf{N}^+$,

$$f_t(C(p, m)) = t[C(p, m)] = C^t(t[p], m) = C^t(f_t(p), m),$$

and

$$H(p, m) = H^t(t[p], m) = H^t(f_t(p), m).$$

Thus f_t is a synchronous deterministic simulation. Its inverse $f_t^{-1}(q) = t^{-1}[q]$ is synchronous in the other direction. Using the identity clock in each direction gives clocked equivalence. $\square$

Definition 4.13. Let T be a formal system capable of coding finite syntax. Write $\langle A \rangle_{\text{code}}$ for the code of an object A . Let $\text{SIC}(u)$ be a formula of T meaning “ u codes a SIC,” let $\text{ATP}(u)$ mean “ u codes an ATP,” and let $\text{Recode}(u, v, w)$ mean “ w codes the recoded copy of the object coded by u under the recoding coded by v .”

Definition 4.14. A SIC M over a formula set Y is proof-bearing for T if members of Y include formulas of T , and if saying that M proves a formula θ from hypotheses Γ means that $\theta \in \bigcup_{m \in \mathbf{N}^+} C(\Gamma, m)$.

Definition 4.15. A proof-bearing SIC M is self-descriptive relative to T and t if T contains codes for M , t , and M^t and proves the relevant coding equations, for example $\text{Recode}(\langle M \rangle_{\text{code}}, \langle t \rangle_{\text{code}}, \langle M^t \rangle_{\text{code}})$.

Theorem 4.16 (Self-certification theorem). *Let $M = (C, H)$ be a deductively effective proof-bearing ATP for T over Y , and let Γ contain the axioms or background assumptions of T used for syntax coding. Suppose that T proves $\text{Recode}(\langle M \rangle_{\text{code}}, \langle t \rangle_{\text{code}}, \langle M^t \rangle_{\text{code}}) \wedge \text{SIC}(\langle M^t \rangle_{\text{code}})$. Then M eventually enumerates that formula from Γ . If M is run as a target-search ATP for that formula, then M halts on that target.*

Proof. The displayed formula is a theorem of T from the chosen background assumptions. Since M is deductively effective for T , every theorem of T from Γ appears in some stage of M on input Γ . The target-search claim is the same statement with that displayed formula chosen as the target. $\square$

Corollary 4.17. *Under the hypotheses of Theorem 4.16, a SIC can prove that a recoded copy of itself is a SIC. If T also proves the corresponding formula $\text{ATP}(\langle M^t \rangle_{\text{code}})$, then the SIC can prove that the recoded copy is an ATP.*

Proof. Apply Theorem 4.16 to the relevant coded verification statement. $\square$

Remark 4.18. This is the precise sense in which a SIC can “say” or “prove” things about a translated copy of itself. The object-level formula is not the machine itself. It is a coded assertion about the recoded code of the machine. This distinction prevents the section from confusing syntax, semantics, and truth.

4.1 The Picard Protocol and a minimal self-certifying ATP

Theorem 4.16 says that a sufficiently strong machine eventually certifies its recoded copy. This subsection turns that guarantee into a named success criterion and then exhibits the smallest machine meeting it.

Definition 4.19 (Picard Protocol). Let F be a formal system, let $Y \subseteq y$ be inference-closed, let t be a syntactic recoding, and let $\Gamma \subseteq Y$. A SIC M over Y satisfies the Picard Protocol for (t, Γ) if there is a target formula $\varphi^* \in Y \setminus \Gamma$ such that:

1. under a declared coding of the machines under discussion — one for which the code of a recoded object is the recoding of its code — φ^* asserts that M^t is an ATP;
2. every member of Γ is true, and every rule of F preserves truth, under that declared interpretation;
3. M is a rule-driven target-search ATP for φ^* , and M halts on input Γ .

The requirement $\varphi^* \notin \Gamma$ excludes the degenerate protocol in which the machine is handed its conclusion as a hypothesis, and rule-drivenness excludes machines that adjoin the conclusion without inference.

Example 4.20 (The Picard machine M_P). The construction proceeds syntax first, machine second, interpretation last.

Step 1: syntax. Let $F_P = (x_P, y_P, z_P)$ with $x_P = \{A, a, b\}$, with $y_P = \{Aa, Ab\}$ (two strings of length two), and with $z_P = \{r\}$, where r is the unary rule defined by $r(Aa) = Ab$ and $r(Ab) = Aa$. Set $Y := y_P$; it is inference-closed because both values of r lie in y_P .

Step 2: recoding. Let U consist of the finite strings over x_P . Let t swap $a \leftrightarrow b$ letterwise and fix A . Then t is an involutive bijection of U with $t[y_P] = y_P$, so t is an endorecoding in the sense of Proposition 4.5: every recoded formula lives inside y_P itself. Moreover $r^t = r$, hence $F_P^t = F_P$: the system is its own recoded copy.

Step 3: machine. Let $M_P := E_{F_P, Y, Ab}$ be the canonical target-search SIC for the target Ab (Definition 2.19).

Step 4: coding declaration. Declare $\text{code}(M_P) := a$. Since the code of a recoded object is the recoding of its code, $\text{code}(M_P^t) = t(a) = b$. Interpret the formula $A\beta$, for $\beta \in \{a, b\}$, as “the machine coded by β is an ATP.” Under this interpretation the target Ab asserts precisely that M_P^t is an ATP.

Step 5: input. Let $\Gamma := \{Aa\}$.

Theorem 4.21 (The Picard machine satisfies the Picard Protocol). *M_P satisfies the Picard Protocol for $(t, \{Aa\})$ with target Ab , and*

$$\tau_{M_P}(\{Aa\}, Ab) = 2.$$

Proof. Clause (3) of Definition 4.19: Y is inference-closed, so M_P is a rule-driven target-search ATP for Ab by Theorem 2.20. On input $\Gamma = \{Aa\}$ the run is

$$C(\Gamma, 1) = \{Aa\} \not\supseteq Ab, \quad C(\Gamma, 2) = D_{F_P}(\{Aa\}) = \{Aa, Ab\} \supseteq Ab,$$

so the machine halts at stage 2 and $\tau_{M_P}(\{Aa\}, Ab) = 2$.

Clause (2): the hypothesis Aa asserts that M_P is an ATP, and this is true by the theorem just cited. Note the order of construction: M_P was fully defined in Step 3, before the interpretation was declared in Step 4, so this is a verification, not a circle. The rule r sends $A\beta$ to $At(\beta)$, since $r(Aa) = Ab = At(a)$ and $r(Ab) = Aa = At(b)$. It is therefore the syntactic shadow of the SIC transport theorem (Theorem 4.10): if the machine coded by β is an ATP, then its recoded copy, coded by $t(\beta)$, is an ATP. Hence r preserves truth under the declared interpretation.

Clause (1) holds by Step 4. □

Proposition 4.22 (Minimality of the Picard machine). *M_P is extremal in three simultaneous senses.*

1. *Every SIC satisfying the Picard Protocol has transaction count at least 2.*
2. *No formal system with a single well-formed formula supports the protocol.*
3. *At least one inference rule is required.*

M_P attains all three bounds: two formulas, one rule, one hypothesis, and transaction count exactly 2.

Proof. For (1), $H(\Gamma, 1) = 1$ would force $\varphi^* \in C(\Gamma, 1) = \Gamma$, contradicting $\varphi^* \notin \Gamma$. For (2), if $y = \{\varphi^*\}$ and $\varphi^* \notin \Gamma$, then $\Gamma = \emptyset$; a proof from $\emptyset$ has no admissible first line, since a hypothesis line is impossible and every finitary rule needs at least one premise, so $\text{Con}_F(\emptyset) = \emptyset$, and by the rule-driven soundness proposition of Section 2 no stage of a rule-driven machine ever contains φ^* . For (3), with $z = \emptyset$ there are no direct consequences at all, so a rule-driven machine never grows past Γ , which excludes the target by hypothesis. □

Remark 4.23 (Mutual certification). Because t is an involution and $F_P^t = F_P$, the recoded machine M_P^t is, by the SIC transport theorem, a target-search ATP for $t(Ab) = Aa$ — and under the declared coding, Aa asserts that $(M_P^t)^t = M_P$ is an ATP. So the two machines certify each other: on the recoded input $t[\Gamma] = \{Ab\}$, the machine M_P^t proves that *its* recoded copy is an ATP, again at transaction count 2 by Corollary 4.11. The Picard Protocol is satisfied here in its strongest symmetric form.

Remark 4.24 (What the example does and does not show). In this micro-setting the arithmetization is externalized: the two formulas carry their meaning by declaration, and the soundness of the one rule is underwritten by a transport theorem proved in the metatheory. That is precisely what makes every step checkable by hand, and it isolates the logical skeleton of self-certification: code, recode, transport, search, halt. Machines that carry the verification internally rather than by declaration are covered by Theorem 4.16 under the hypotheses of Theorems 4.28 and 4.32: an effectively presented system interpreting enough arithmetic, a provably primitive recursive recoding, a canonical deductively effective machine, and an input containing the finitely many defining axioms. In both settings the Tarski–Gödel barrier (Theorem 4.26) marks the ceiling: ATP-hood of the recoded copy is certifiable, while a full truth predicate for recoded sentences is not.

The next two theorem schemes are standard consequences of the diagonal lemma and Tarski-style undefinability phenomena, now expressed in the language of recoded syntax; see, for example, [20, 21].

Theorem 4.25 (Equivariant diagonal lemma for recoded syntax). *Let T be a recursively axiomatized theory strong enough to represent primitive recursive functions and to prove the ordinary diagonal lemma. Let t be a primitive recursive syntactic recoding whose action on sentence codes is represented in T . For every one-variable formula $P(v)$ of T , there is a sentence θ such that $T \vdash \theta \leftrightarrow P(\overline{t(\langle \theta \rangle_{\text{code}})})$.*

Proof. This is the ordinary diagonal lemma applied after composing the usual self-substitution function with the represented recoding function. Informally, instead of making a sentence that talks about its own code, one makes a sentence that talks about the recoded version of its own code. The required fixed point exists because the recoding operation is effectively representable in T . $\square$

Theorem 4.26 (Tarski–Gödel barrier for recoded self-reference). *Let T be a consistent effectively axiomatized theory containing enough arithmetic for the diagonal lemma. Then no formula $\text{True}_t(v)$ of T can serve as a full internal truth predicate for all recoded sentences of T in the sense that T proves all biconditionals $\text{True}_t(\overline{t(\langle \theta \rangle_{\text{code}})}) \leftrightarrow \theta$ for sentences θ of T .*

Proof. If such a predicate existed, Theorem 4.25 applied to the negation of True_t would produce a recoded liar sentence. The usual Tarski–Gödel contradiction follows. The recoding does not remove the obstruction because it is invertible and effectively represented; it only renames the codes. $\square$

Remark 4.27. The previous theorem is not a defect in the recoding framework. It is a useful guardrail. A SIC may prove many syntactic facts about its recoded copy, including that the copy is a SIC or ATP. What it may not have, on pain of the classical semantic paradoxes and incompleteness phenomena, is a complete internally correct truth predicate for all recoded sentences of its own language.

The following theorem settles, positively, what Version 42 stated as the *Self-Audit Compression Conjecture*. The hypotheses (i)–(iii) replace the informal word “natural” of the conjecture with explicit conditions; they hold for the standard arithmetized metatheory of Peano arithmetic and of the usual weaker fragments with efficient coding of syntax.

Theorem 4.28 (Self-audit compression theorem). *Let T be a recursively axiomatized theory representing all primitive recursive functions, and let $\mathcal{F}$ be a family of effectively coded SICs together with a coded family of uniform recodings. Write $|u|$ for the length of a code u . Suppose:*

- (i) (uniform provable transport) *there is a function symbol or definable term tc of T representing the primitive recursive map $(\langle M \rangle_{\text{code}}, \langle t \rangle_{\text{code}}) \mapsto \langle M^t \rangle_{\text{code}}$, and T proves the single universal sentence*

$$\forall u \forall v (\text{SIC}(u) \wedge \text{Recoding}(v) \rightarrow \text{SIC}(\text{tc}(u, v))),$$

and likewise with ATP in place of SIC;

- (ii) (polynomial adequacy) *every true numerical instance $\text{tc}(\bar{a}, \bar{b}) = \bar{c}$ has a T -proof of length polynomial in $|a| + |b| + |c|$;*
- (iii) (polynomially certified membership) *for each M and t in the family, the statements $\text{SIC}(\langle M \rangle_{\text{code}})$, $\text{ATP}(\langle M \rangle_{\text{code}})$ where true, and $\text{Recoding}(\langle t \rangle_{\text{code}})$ have T -proofs of length polynomial in $|\langle M \rangle_{\text{code}}|$ and $|\langle t \rangle_{\text{code}}|$ respectively.*

Then there is a polynomial P such that for every M and t in the family, the verification statements $\text{SIC}(\langle M^t \rangle_{\text{code}})$ and, where applicable, $\text{ATP}(\langle M^t \rangle_{\text{code}})$ have T -proofs of length at most $P(|\langle M \rangle_{\text{code}}| + |\langle t \rangle_{\text{code}}|)$.

Proof. A uniform recoding acts constituent-wise on codes, so $|\langle M^t \rangle_{\text{code}}|$ is bounded by a polynomial in $|\langle M \rangle_{\text{code}}| + |\langle t \rangle_{\text{code}}|$. Assemble the verification proof from four parts. First, one fixed T -proof Π_0 of the universal transport sentence of (i); its length is a constant independent of M and t . Second, proofs of $\text{SIC}(\langle M \rangle_{\text{code}})$ and $\text{Recoding}(\langle t \rangle_{\text{code}})$, each polynomial in the code lengths by (iii). Third, universal instantiation of Π_0 at the numerals $\overline{\langle M \rangle_{\text{code}}}$ and $\overline{\langle t \rangle_{\text{code}}}$, followed by two applications of modus ponens; this contributes length proportional to the instantiated formulas, hence polynomial in the code lengths. This yields $\text{SIC}(\text{tc}(\overline{\langle M \rangle_{\text{code}}}, \overline{\langle t \rangle_{\text{code}}}))$. Fourth, by (ii) there is a polynomial-length proof of $\text{tc}(\overline{\langle M \rangle_{\text{code}}}, \overline{\langle t \rangle_{\text{code}}}) = \overline{\langle M^t \rangle_{\text{code}}}$, and one substitution of equals gives $\text{SIC}(\langle M^t \rangle_{\text{code}})$. The sum of the four contributions is bounded by a single polynomial P . The ATP case is identical with the second transport sentence of (i). $\square$

Remark 4.29. In Version 42 this statement was the Self-Audit Compression Conjecture. The content of the resolution is that self-audit never requires re-deriving the transport theory from scratch for each machine: the transport theorem is proved once, and each instance costs only the price of certifying the codes and running the recoded-code computation inside T .

Version 42 conjectured that the normalized family of target transaction counts, taken over all finite hypothesis sets and all targets, classifies deterministic rule-driven SICs up to clocked deterministic simulation equivalence after quotienting by syntactic recodings (the *Normalized Depth Spectrum Conjecture*). The next theorem refutes it. For any SIC $M = (C, H)$ over Y , extend the target transaction count of Section 7 by setting $\tau_M(\Gamma, \varphi) := \min\{m \in \mathbf{N}^+ : \varphi \in C(\Gamma, m)\}$ and $\tau_M(\Gamma, \varphi) := \infty$ when no such stage exists; the *depth spectrum* of M is the family $(\tau_M(\Gamma, \varphi))$ over all finite $\Gamma \subseteq Y$ and all $\varphi \in Y$.

Theorem 4.30 (Failure of the normalized depth spectrum). *The Normalized Depth Spectrum Conjecture of Version 42 is false, in both possible directions, already for deterministic rule-driven SICs over a fixed finite formal system:*

1. *the depth spectrum is not invariant under clocked deterministic simulation equivalence: there are rule-driven SICs $M \sim N$ with $\tau_M \neq \tau_N$;*

2. the depth spectrum is not complete: there are rule-driven SICs M_0 and M_1 over the same formal system with identical depth spectra such that $M_0 \not\sim M_1$, and this failure survives quotienting by syntactic recodings, since no recoded copy of M_0 is clocked-equivalent to M_1 .

Since any normalization of the spectrum is a function of the raw spectrum, part (2) refutes the conjecture for every choice of normalization.

Proof. Fix the formal system $F_0 = (x_0, y_0, z_0)$ with $x_0 = y_0 = \{a, b\}$ (strings of length one) and $z_0 = \{r\}$, where r is the unary rule defined only by $r(b) = a$. Then $D_{F_0}(\Gamma) = \Gamma \cup \{a\}$ if $b \in \Gamma$ and $D_{F_0}(\Gamma) = \Gamma$ otherwise.

For (1), define M and N over $Y = y_0$, both with constantly zero halting maps, by

$$C_M(\Gamma, 1) = \Gamma, \quad C_M(\Gamma, m) = D_{F_0}(\Gamma) \quad (m \geq 2),$$

$$C_N(\Gamma, 1) = C_N(\Gamma, 2) = \Gamma, \quad C_N(\Gamma, m) = D_{F_0}(\Gamma) \quad (m \geq 3).$$

Both are SICs, and both are rule-driven because the only formula ever added is a , which is a direct consequence of any state containing b . Exactly as in Theorem 3.23, the identity state map with the clock $\rho(1) = 1, \rho(m) = m + 1$ for $m \geq 2$ witnesses $M \preceq N$, and the identity state map with the clock $\sigma(1) = \sigma(2) = 1, \sigma(m) = m - 1$ for $m \geq 3$ witnesses $N \preceq M$. Hence $M \sim N$. But

$$\tau_M(\{b\}, a) = 2 \neq 3 = \tau_N(\{b\}, a),$$

so clocked-equivalent machines can have different spectra.

For (2), define the static machines $M_0 = (C_{\text{stat}}, H_0)$ and $M_1 = (C_{\text{stat}}, H_1)$ over $Y = y_0$ by

$$C_{\text{stat}}(\Gamma, m) = \Gamma, \quad H_0(\Gamma, m) = 0, \quad H_1(\Gamma, m) = 1.$$

Both satisfy the SIC axioms (the halting-freeze axiom is vacuous for a static stage map) and both are vacuously rule-driven because no formula is ever added. They share one stage map, so $\tau_{M_0}(\Gamma, \varphi) = \tau_{M_1}(\Gamma, \varphi)$ for every input and target: the spectra are identical. Yet any clocked deterministic simulation from M_0 to M_1 would give $0 = H_0(p, m) = H_1(f(p), \rho(m)) = 1$, which is impossible, so $M_0 \not\sim M_1$. Finally, recoding cannot repair this: for any uniform recoding t , the recoded machine M_0^t has halting map $H_0^t(\Delta, m) = H_0(t^{-1}[\Delta], m) = 0$, so the same halting contradiction shows $M_0^t \not\sim M_1$. $\square$

Remark 4.31. The refutation isolates exactly what the spectrum measures and what it ignores. Transaction counts record *when* formulas first appear, so they are rescaled by admissible clocks (part 1) and are blind to halting observations (part 2), while clocked equivalence preserves halting exactly and forgives rescaling. A classification invariant for clocked equivalence must therefore record halting data and must be stated in a clock-invariant form, for example as the spectrum's equivalence class under admissible reparametrization. Whether some such repaired invariant is complete for an interesting subclass remains a reasonable further question, but the conjecture as posed in Version 42 is settled in the negative.

The third Version 42 conjecture asserted that every effectively coded deductively effective SIC has a conservative finite metadata extension that lets it prove the basic self-certification statement for each primitive recursive recoding in a fixed recoding family, without changing the underlying theorem set. The following theorem settles it positively.

Theorem 4.32 (Finite metadata self-description theorem). *Let T be a recursively axiomatized theory representing all primitive recursive functions, let $M = (C, H)$ be an effectively coded, deductively effective, proof-bearing SIC for T over Y , and let $(t_i)_{i \in \mathbf{N}}$ be a uniformly primitive recursive family of syntactic recodings, given by a single primitive recursive function u with $u(i, \cdot) = t_i$. Suppose T proves the arithmetized SIC transport theorem in the uniform form of Theorem 4.28(i). Then there is a finite definitional extension T' of T , obtained by adjoining function symbols for u and tc and predicate symbols SIC , ATP , and Recode together with their finitely many defining axioms, such that:*

1. *T' is conservative over T : every formula in the original language provable in T' is already provable in T , so the underlying theorem set is unchanged;*
2. *for every $i \in \mathbf{N}$,*

$$T' \vdash \text{Recode}(\langle M \rangle_{\text{code}}, \langle t_i \rangle_{\text{code}}, \langle M^{t_i} \rangle_{\text{code}}) \wedge \text{SIC}(\langle M^{t_i} \rangle_{\text{code}});$$

3. *if Γ contains the background syntax-coding axioms of T together with the finitely many defining axioms of T' , then M eventually enumerates each of the self-certification formulas in (2) from Γ , and run as a target-search ATP for any one of them, M halts on that target.*

Proof. The extension T' adds finitely many symbols, each with a defining axiom that determines it uniquely from the original language. Extensions by definitions are conservative, which proves (1); in particular the theorem set of the original language is untouched, so the metadata are pure bookkeeping.

For (2), fix i . The equation asserting that applying the coded recoding $u(i, \cdot)$ to $\langle M \rangle_{\text{code}}$ yields $\langle M^{t_i} \rangle_{\text{code}}$ is a true instance of a primitive recursive computation. Since T represents all primitive recursive functions and T' names u and tc , that instance is provable in T' , and by the defining axiom of Recode so is the first conjunct. The second conjunct follows from the uniform transport sentence instantiated at $\langle M \rangle_{\text{code}}$ and $\langle t_i \rangle_{\text{code}}$, exactly as in the proof of Theorem 4.28, followed by the provable code computation.

For (3), apply Theorem 4.16: each formula in (2) is a theorem of T' from Γ , and M is deductively effective, so each such formula appears at some stage of M on input Γ ; the target-search claim is the same statement with that formula chosen as target. $\square$

Remark 4.33. In Version 42 this was the Finite Metadata Self-Description Conjecture. The resolution identifies the promised “finite metadata” concretely: the finitely many defining axioms of a definitional extension. One finite package of names suffices for the whole infinite recoding family because the family is uniformly primitive recursive, so each instance is recovered by a provable computation rather than by a new axiom.

Theorem 4.34 (No-free-recoding-speedup theorem). *Pure invertible recoding alone does not yield transaction-count speedup for any target family. More precisely, let $(M_i, \Gamma_i, \varphi_i)_{i \in I}$ be a family in which each M_i is a target-search ATP for φ_i , and let t be an invertible recoding. Then*

$$\tau_{M_i^t}(t[\Gamma_i], t(\varphi_i)) = \tau_{M_i}(\Gamma_i, \varphi_i)$$

for every $i \in I$, including the value ∞ .

Proof. This is Corollary 4.11 applied pointwise. Since the two cost families agree exactly, they have the same finite values, failures, asymptotic growth, and normalized depth spectrum. $\square$

Remark 4.35. The theorem explains why this section belongs between simulation and heuristic search. Recoding gives self-reference and isomorphism; it does not, by itself, make hard targets easy. Speedup enters later when the language is conservatively extended by derived rules, when a graph-search policy changes the explored frontier, or when proof-theoretic extraction supplies computational content not previously explicit in the search state.

5 Nondeterministic Search Policies and Heuristic Proof Search

Definition 5.1. Let $Y \subseteq y$ and let $p \subseteq Y$. An admissible one-step extension at p is a finite set $\Delta \subseteq Y$ such that every $\beta \in \Delta$ is a direct consequence of p .

Definition 5.2. A nondeterministic proof-search policy on Y is a function Σ defined on pairs (p, m) with $p \subseteq Y$ and $m \in \mathbf{N}^+$ such that $\Sigma(p, m)$ is a nonempty set of admissible one-step extensions at p .

Definition 5.3. Let $Y \subseteq y$ be inference-closed and let Σ be a nondeterministic proof-search policy on Y . A branch of the nondeterministic search generated by Σ from input $\Gamma \subseteq Y$ is a sequence $b = \langle p_1, p_2, \dots \rangle$ such that $p_1 = \Gamma$ and for each $m \in \mathbf{N}^+$ there exists $\Delta_m \in \Sigma(p_m, m)$ with $p_{m+1} = p_m \cup \Delta_m$. The deterministic branch SIC induced by b is the SIC $M_b = (C_b, H_b)$ over Y defined at every input by

$$C_b(\Delta, m) := \begin{cases} p_m, & \Delta = \Gamma, \\ \Delta, & \Delta \neq \Gamma, \end{cases} \quad H_b(\Delta, m) := 0.$$

The second clause is needed because Definition 2.3 requires C_b to be total on $\mathcal{P}(Y) \times \mathbf{N}^+$, whereas the branch b is generated from the single input Γ ; on every other input M_b simply idles, which is rule-driven vacuously.

Definition 5.4. Let $\varphi \in Y$. A nondeterministic target-search branch for φ is a branch $b = \langle p_1, p_2, \dots \rangle$ such that once $\varphi \in p_m$, all later states are frozen equal to p_m .

Definition 5.5. A nondeterministic policy Σ is proof-covering if for every input $\Gamma \subseteq Y$ and every finite proof $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ from Γ , there exists a branch $b = \langle p_1, p_2, \dots \rangle$ of the search from Γ such that $\{\pi(1), \dots, \pi(j)\} \subseteq p_j$ for every $j \in F_n$.

Theorem 5.6 (Branch soundness theorem). *Let $Y \subseteq y$ be inference-closed, let Σ be a nondeterministic proof-search policy on Y , and let $b = \langle p_1, p_2, \dots \rangle$ be a branch from input $\Gamma \subseteq Y$. Then the induced deterministic branch SIC M_b is rule-driven. Consequently, for every $m \in \mathbf{N}^+$, $p_m \subseteq \text{Con}_F(\Gamma) \cap Y$. Moreover the canonical map $a_{M_b}(p) = \text{Con}_F(p) \cap Y$ is a state abstraction from M_b to E_{FY} .*

Proof. By construction, every new formula adjoined at stage $m+1$ belongs to some admissible one-step extension at p_m , so every such formula is a direct consequence of p_m . Thus M_b is rule-driven. The soundness inclusion now follows from the rule-driven soundness proposition of Section 2. The state-abstraction statement is then an immediate application of Theorem 3.30. $\square$

Theorem 5.7 (Branch-Covering Theorem). *Let $Y \subseteq y$ be inference-closed, let Σ be a proof-covering nondeterministic proof-search policy on Y , let $\Gamma \subseteq Y$, and let $\varphi \in Y$. If $\Gamma \vdash_F \varphi$, then there exists a nondeterministic target-search branch for φ that reaches φ at stage at most $\|\varphi\|_F(\Gamma)$.*

Proof. Choose a shortest proof $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ of φ from Γ , so $n = \|\varphi\|_F(\Gamma)$. Because Σ is proof-covering, there exists a branch $b = \langle p_1, p_2, \dots \rangle$ such that $\{\pi(1), \dots, \pi(j)\} \subseteq p_j$ for every $j \in F_n$. In particular, $\varphi = \pi(n) \in p_n$. Freezing the branch once φ appears yields a nondeterministic target-search branch for φ that reaches φ no later than stage $n = \|\varphi\|_F(\Gamma)$. $\square$

Corollary 5.8. *Under the hypotheses of Theorem 5.7, heuristic nondeterminism can recover the shortest-proof horizon of a target whenever some branch tracks a shortest proof.*

Proof. The theorem gives a branch that reaches φ by stage $\|\varphi\|_F(\Gamma)$. The Proof-Horizon Theorem shows that no target-search SIC based solely on shortest proof length can reach φ earlier than that stage. $\square$

Remark 5.9. Theorem 5.7 is the second of the paper's three main imaginative theorems. It says that nondeterministic heuristics can be understood as branch generators, and completeness reduces to whether some branch can shadow each proof. Later, Theorem 7.12 will show how derived rules can be used to lower the horizons that those branches must meet.

6 Theorem Graphs, Maze Search, and Graph-Theoretic ATPs

This section adds a graph-theoretic layer to the SIC and ATP framework. The purpose is not merely metaphorical. If a proof search can be represented as a graph search, then existing algorithms from graph theory can be pulled back into automated theorem proving. Breadth-first search becomes shortest-proof search for an unweighted theorem graph. A predecessor map becomes an explicit proof certificate. A maze exit becomes a target formula. A stopping condition becomes the requirement that the ATP halt as soon as the target vertex is reached.

Definition 6.1. Let $F = (x, y, z)$ be a formal system. A conjunction-like connective on y is an operation, written informally as $\wedge$, that behaves like conjunction at least with respect to conjunction introduction, conjunction elimination, and symmetry. Thus from formulas α and β one may form a formula behaving like $\alpha \wedge \beta$; from $\alpha \wedge \beta$ one may recover α and β ; and $\alpha \wedge \beta$ and $\beta \wedge \alpha$ are equivalent for proof-search purposes. Associativity and idempotence may also be assumed when the intended vertex is really a finite set of available formulas rather than a syntactic string with a fixed parenthesization.

Definition 6.2. Let $\Gamma \subseteq y$. A formula-labeled theorem graph for Γ is a directed graph whose vertices are labeled by elements of y . The most initial vertex is labeled by the conjunction of the members of Γ when this conjunction is available as a formula. If Γ is finite, this is the finite conjunction of the hypotheses. If Γ is an infinite axiom scheme, one may either use an infinitary conjunction in an infinitary logic or replace the single initial formula by a proof state that records the available hypotheses as a set.

Definition 6.3. The state theorem graph for Γ is the directed graph whose vertices are proof states $p \subseteq y$ with $\Gamma \subseteq p$. There is a directed edge from p to q labeled by the inference data $(r, a_1, \dots, a_k)$ when $r \in z_k(F)$, each $a_i \in p$, $r(a_1, \dots, a_k)$ is defined, and $q = p \cup \{r(a_1, \dots, a_k)\}$. The edge is nontrivial when $q \neq p$.

Remark 6.4. The formula-labeled graph is closer to the usual picture of formulas connected by inference arrows. The state theorem graph is often better for ATP design because a state remembers all formulas already obtained. In SIC language, a vertex of the state theorem graph is a possible value of $C(\Gamma, m)$, and an edge is one admissible proof-search transaction.

Proposition 6.5 (A DAG version of the theorem graph). *Let $G_{F,\Gamma}^{\text{nov}}$ be the state theorem graph obtained by keeping only nontrivial edges $p \rightarrow q$, where $q = p \cup \{\beta\}$ and $\beta \notin p$. Then $G_{F,\Gamma}^{\text{nov}}$ is a directed acyclic graph.*

Proof. Along every edge of $G_{F,\Gamma}^{\text{nov}}$, the state strictly increases by set inclusion. A directed cycle would require a finite chain of strict inclusions that returns to its starting set. That is impossible. Hence the graph is a DAG. $\square$

Remark 6.6. The unmodified formula-level theorem graph need not be a DAG. There may be cycles caused by identity rules, reversible equivalences, conjunction introduction followed by conjunction elimination, double-negation transformations, commutativity or associativity conversions, or derived rules that return to an earlier formula. The state graph may also have loops if trivial edges $p \rightarrow p$ are retained. Three standard repairs are available. First, suppress trivial edges and use the novelty state graph just defined. Second, quotient formulas by a chosen equivalence relation and orient only proof-progressing edges. Third, unfold the search by time, so that a vertex is a pair (p, m) and every edge increases the stage number. The time-unfolded graph is automatically acyclic because the second coordinate strictly increases.

Definition 6.7. A target vertex for a formula $\varphi \in y$ is any vertex whose label is φ in the formula-labeled graph, or any state p with $\varphi \in p$ in the state theorem graph. A proof-search path from Γ to φ is a directed path from the initial vertex to a target vertex. In the informal maze analogy, the initial vertex is the entrance, the target vertex is the exit, and each labeled edge is one legal move through the maze.

Proposition 6.8 (Proofs as graph paths). *For a finitary formal system, every ordinary proof of φ from Γ determines a directed path in the state theorem graph from the initial state Γ to a target state containing φ . Conversely, every directed path from Γ to a state containing φ determines a proof of φ from Γ after redundant states and unused formulas are ignored.*

Proof. Given a proof, start at Γ and add each new formula when it first occurs as the conclusion of a rule application. Each addition is exactly a labeled edge in the state theorem graph. When the proof reaches φ , the corresponding path has reached a target state. Conversely, a path in the state theorem graph is a finite list of rule-labeled state transitions. Reading off the formulas added by those transitions, and keeping the recorded premises for each labeled edge, gives a line-by-line proof of every formula added along the path, in particular of φ once the target state is reached. $\square$

Theorem 6.9 (Shortest-path ATP principle). *Assume the relevant portion of the state theorem graph is finite or locally finite and effectively enumerable. If every edge has unit cost, then breadth-first search from the initial state finds a target state containing φ in the least possible number of graph edges, if such a target is reached. If a predecessor map is stored during the search, then the shortest path can be reconstructed by tracing predecessors backward from the target to the initial state.*

Proof. This is the standard breadth-first-search theorem for unweighted directed graphs. Breadth-first search explores vertices in nondecreasing distance from the initial vertex. Therefore the first time a target vertex is removed from the queue, its distance is minimal among all paths from the initial vertex. A predecessor map records, for each newly discovered vertex, the previous vertex and edge label by which it was discovered. Tracing those entries backward reconstructs a shortest path. Pulling the edge labels back to inference rules reconstructs a shortest proof in the corresponding graph metric. $\square$

Remark 6.10. In graph-theoretic language, the length of the shortest tour to the target is simply the graph distance from the initial vertex to the target vertex, provided “tour” is understood as a directed path or walk rather than a closed tour returning to the start. In ATP language, this graph distance is a transaction count. If vertices are proof states and each edge adds exactly one new formula, then the shortest path length counts the fewest one-formula state expansions needed to reach φ . If vertices are proof lines instead, the same idea becomes shortest proof length in the sense of Section 2.

Remark 6.11. For weighted theorem graphs, where edges may have different costs, Dijkstra’s algorithm replaces breadth-first search when all edge weights are nonnegative. If there is an admissible heuristic estimating remaining distance to the target, A* search can prioritize promising branches while preserving optimality under the usual hypotheses. On a finite DAG, shortest paths can also be computed by dynamic programming after a topological ordering. These are graph-theoretic algorithms first, but in the present framework they become ATP policies once vertices are proof states and edge labels are inference rules.

Remark 6.12. Depth-first maze solvers can find an exit using less memory, but they do not necessarily find the shortest proof. Breadth-first search is the canonical shortest-path method in the unweighted case. A practical ATP may therefore mix policies: breadth-first search supplies the benchmark, depth-first or heuristic search supplies memory control, and nondeterministic branch policies from Section 5 describe the possible guided paths through the theorem graph.

Corollary 6.13. *The canonical breadth-first SIC $B_{F,Y,\varphi}$ may be read as a graph-search ATP on a theorem graph. Its halting stage is the point at which the breadth-first frontier first intersects the target set of vertices containing φ .*

Proof. By the Proof-Horizon Theorem, $B_{F,Y,\varphi}$ halts exactly when the target formula has appeared at the least proof horizon. Breadth-first search on the corresponding unweighted theorem graph discovers vertices by increasing path length. Thus the SIC stage and the graph-search distance are two presentations of the same underlying search horizon. $\square$

Remark 6.14. This section suggests a practical implementation route; for the standard algorithmic background on implemented provers see [5, 10]. To build a small ATP, represent formulas as syntax trees or strings, represent proof states as finite sets of formulas, generate outgoing edges by applying inference rules to tuples from the current state, and run a graph-search algorithm with an early stopping test $\varphi \in p$. The output should not merely say “success.” It should return the predecessor chain and the edge labels, because those labels are the actual proof.

7 Ground-Up ATPs, Weak Set Theory, Python, and Speedup

This section addresses four practical questions. What is the right formal background for $\text{Succ}(\{0\}) = \{0, 1\}$? Can the framework be implemented concretely in Python? What is a sensible SIC notion of “transaction”? And in what sense can one automated theorem prover be provably better than another?

7.1 A weak set-theoretic setting for $\text{Succ}(\{0\}) = \{0, 1\}$

Definition 7.1. A weak successor-set theory is any first-order theory in a language containing $\in$, $\emptyset$, a pairing operation, a union operation, and a successor operation Succ , together with enough axioms to support extensional equality, empty set, singleton and pair formation, union, and the defining equation $\text{Succ}(a) = a \cup \{a\}$. Define $0 := \emptyset$, $1 := \{0\}$, $2 := \{0, 1\}$.

Proposition 7.2. *There is no absolute smallest formal system for proving $\text{Succ}(\{0\}) = \{0, 1\}$ unless one first fixes the language and the notion of admissible definitions. Nevertheless, a very weak set-theoretic background suffices: extensional equality together with empty set, pairing, union, and the definition $\text{Succ}(a) = a \cup \{a\}$ already proves the target.*

Proof. Once the language contains the displayed operations and equality behaves extensionally, one computes $\text{Succ}(\{0\}) = \{0\} \cup \{\{0\}\} = \{0, \{0\}\} = \{0, 1\}$. The statement does not require Infinity, Separation, Replacement, Foundation, or the full strength of ZF. On the other hand, the statement is not even formulable in pure Robinson arithmetic, because pure arithmetic lacks $\in$, $\emptyset$, pairing, and union. $\square$

Remark 7.3. So the right answer to “what is the smallest formal system?” is partly relative. If one insists on a set-theoretic language, then a very weak extensional theory with empty set, pairing, union, and successor already works. If one allows conservative definitional extensions, even weaker presentations are possible. But pure Robinson arithmetic by itself is not the right ambient system.

7.2 Executable ATPs and a Python point of view

Definition 7.4. A subset $Y \subseteq y$ is effectively coded if there is an injective map $\text{code}_Y \in [Y \rightarrow \mathbf{N}]$ such that the image $\text{code}_Y[Y]$ is decidable and the basic syntactic operations needed to evaluate the rules of F on coded tuples are computable.

Theorem 7.5. *Let $Y \subseteq y$ be finite and effectively coded, let $\Gamma \subseteq Y$, and let $\varphi \in Y$. If the direct-consequence relation of F restricted to Y is decidable, then there exists a concrete breadth-first Python procedure that halts with success if and only if $\Gamma \vdash_F \varphi$ inside Y .*

Proof. Because Y is finite, one can represent each state as a finite Python set of codes. Because direct consequence on Y is decidable, one can enumerate all tuples of premises from the current state, test each rule, and compute the next breadth-first layer. Starting from Γ , the algorithm repeatedly replaces the current state by $D_F(\cdot) \cap Y$ until either φ appears or no new formula is produced. Since Y is finite, the search terminates. By Theorem 2.17, the search succeeds exactly when $\varphi \in \text{Con}_F(\Gamma) \cap Y$. $\square$

Remark 7.6. In practice, formulas can be represented by strings, tuples, or syntax trees; inference rules can be represented by Python functions on tuples of formulas; and breadth-first search is just repeated closure under those functions. For toy targets such as $\text{Succ}(\{0\}) = \{0, 1\}$, a user can implement either rewriting rules or explicit inference rules and obtain a real executable prover, even though the present paper abstracts away from low-level programming detail.

7.3 Transactions, derived rules, and speedup

Definition 7.7. Let $Y \subseteq y$. A derived rule on Y is a finitary rule d on Y such that whenever $d(a_1, \dots, a_k)$ is defined, one has $\{a_1, \dots, a_k\} \vdash_F d(a_1, \dots, a_k)$.

Definition 7.8. If D is a set of derived rules on Y , let $F^D := (x, y, z \cup D)$ denote the macro-augmented formal system.

Definition 7.9. Let $M = (C, H)$ be a target-search ATP for φ over Y . For input $\Gamma \subseteq Y$, define the transaction count $\tau_M(\Gamma, \varphi) := \min\{m \in \mathbf{N}^+ : \varphi \in C(\Gamma, m)\}$, if such an m exists, and define $\tau_M(\Gamma, \varphi) := \infty$ otherwise.

Remark 7.10. This is the paper’s formalization of “transactions” in proof search. It counts stage advances until the target first appears.

Proposition 7.11. *Every theorem proved in the macro-augmented system F^D is already a theorem of F . More precisely, if $\Gamma \vdash_{F^D} \psi$, then $\Gamma \vdash_F \psi$.*

Proof. Take a proof in F^D . Replace each line produced by a derived rule from D by a proof of that line from the corresponding premises in F . Since the proof is finite, this replacement process yields a finite proof in F of the same conclusion from the same hypotheses. $\square$

Theorem 7.12 (Conservative Compilation Theorem). *Let $Y \subseteq y$ be inference-closed, and let $\mathcal{T} = \{(\Gamma_i, \varphi_i) : 1 \leq i \leq N\}$ be a finite family with each $\Gamma_i \subseteq Y$, each $\varphi_i \in Y$, and each $\Gamma_i \vdash_F \varphi_i$. Then there exists a finite set D of derived rules on Y such that for every i , $\tau_{E_{F^D, Y, \varphi_i}}(\Gamma_i, \varphi_i) \leq 2$.*

Proof. For each i , choose one finite proof of φ_i from Γ_i . Only finitely many premises from Γ_i are actually used; list them as a tuple $\bar{a}_i = (a_{i,1}, \dots, a_{i,k_i})$. Define a new k_i -ary rule d_i by $d_i(a_{i,1}, \dots, a_{i,k_i}) = \varphi_i$, and leave d_i undefined on all other tuples. Because $\{a_{i,1}, \dots, a_{i,k_i}\} \vdash_F \varphi_i$, each d_i is a derived rule on Y . Let $D := \{d_1, \dots, d_N\}$.

Now fix i . At stage 1, the canonical target-search SIC E_{F^D, Y, φ_i} contains Γ_i . At stage 2, it adjoins every direct consequence of Γ_i in the augmented system, including the value of d_i on the tuple $\bar{a}_i$. Hence φ_i appears by stage 2, so $\tau_{E_{F^D, Y, \varphi_i}}(\Gamma_i, \varphi_i) \leq 2$. $\square$

Corollary 7.13. *Any finite workload of already provable targets can be compiled into a conservative lemma cache that collapses their transaction counts to at most two stages.*

Proof. This is just Theorem 7.12 interpreted as lemma compilation. $\square$

Corollary 7.14. *Theorem 7.12 does not settle asymptotic proof complexity for infinite families. It explains why finite workloads admit drastic speedup without implying uniform speedup on all targets.*

Proof. A finite compilation theorem speaks only about finitely many designated targets. Asymptotic claims concern unbounded families. $\square$

Remark 7.15. This is also the right place to locate the computational content of infinitesimal-style proofs. A nonstandard argument that proves a standard target can sometimes be compiled, by functional interpretation or proof mining, into standard witnesses or derived rules. In the present notation, that compilation does not make the nonstandard proof unsound; rather, it asks whether the nonstandard proof determines a conservative standard object that lowers the proof horizon or makes the hidden bound explicit.

Remark 7.16. Theorem 7.12 is the third of the paper’s three main imaginative theorems. It turns “speedup” into a concrete compilation statement: finite collections of provable targets can be prepackaged into sound macro rules. Together with Theorem 2.12, this shows that speedup may be read as lowering proof horizons; together with Theorem 5.7, it shows how heuristics and compiled lemmas can cooperate, because branch generators have fewer critical steps to guess correctly.

Theorem 7.17 (Abstract asymptotic speedup principle). *Let $Y \subseteq y$ be inference-closed, let (Γ_n, φ_n) be a family with $\Gamma_n \subseteq Y$ and $\varphi_n \in Y$, and let D be a set of derived rules on Y . Suppose there are functions $p, q \in [\mathbf{N}^+ \rightarrow \mathbf{N}^+]$ such that:*

1. $\tau_{E_{F^D, Y, \varphi_n}}(\Gamma_n, \varphi_n) \leq p(n)$ for all n ;
2. every proof of φ_n from Γ_n in F has length at least $q(n)$ for all n ;

3. $q(n)/p(n)$ is unbounded.

Then the augmented canonical target-search ATP is asymptotically faster on the family (Γ_n, φ_n) than line-by-line search in the original system.

Proof. Hypothesis (1) bounds target-finding transactions in the augmented system by $p(n)$. Hypothesis (2) bounds from below the line length of every original-system proof by $q(n)$. If $q(n)/p(n)$ is unbounded, then the augmented search reaches the targets in asymptotically fewer stage transitions than any line-by-line realization in the original system. Soundness relative to the original system follows from conservativity of derived rules. $\square$

Remark 7.18. A speedup theorem compares search disciplines or proof systems on families of targets. It does not say that one prover is uniformly better on every input, and it does not say that a single dramatic example settles the matter. What it does provide is an asymptotic separation on a specified family.

Remark 7.19. The self-reference section also clarifies what speedup is not. Since M and M^t are isomorphic under an invertible recoding, Corollary 4.11 says that pure recoding preserves transaction counts. Therefore a recoded copy of a prover is not automatically a faster prover. Any real speedup must come from adding derived rules, changing the search policy, mining a proof for explicit data, or otherwise changing the proof-search structure in a way that is not merely a renaming.

8 Counting Proofs, Shortest Proofs, and the Password Analogy

Definition 8.1. For $\Gamma \subseteq y$, $\varphi \in y$, and $n \in \mathbf{N}^+$, let $P_F(\Gamma, \varphi, n)$ be the number of proofs of φ from Γ having length exactly n . Let $P_F^{\min}(\Gamma, \varphi) := P_F(\Gamma, \varphi, \|\varphi\|_F(\Gamma))$ when $\|\varphi\|_F(\Gamma) < \infty$, and define $P_F^{\min}(\Gamma, \varphi) := 0$ otherwise.

Proposition 8.2. *Exact counts of proofs and exact counts of shortest proofs are not determined by the cardinalities of Γ , y , and the rule sets $z_k(F)$ alone.*

Proof. For shortest proofs, let $y = \{a, b, c\}$, let $\Gamma = \{a, b\}$, and let both formal systems F_1 and F_2 have exactly one unary rule. In F_1 let the rule be defined only by $r(a) = c$. In F_2 let the rule be defined by $s(a) = c$ and $s(b) = c$. Then $|y|$, $|\Gamma|$, and $|z_1|$ agree in the two systems, but $P_{F_1}^{\min}(\Gamma, c) = 1$ while $P_{F_2}^{\min}(\Gamma, c) = 2$. Thus the exact number of shortest proofs is not determined by cardinal data alone.

For total proofs, use the same underlying cardinal data but choose systems F_3 and F_4 with one unary rule each such that in F_3 the rule maps a to b and never yields c , while in F_4 the rule maps a to c . Then c is unprovable from Γ in F_3 , so the total number of proofs of c is 0, whereas in F_4 there are infinitely many proofs of c because redundant hypothesis lines may be inserted before the final use of the rule. Hence exact proof counts are not determined by those cardinalities alone. $\square$

Theorem 8.3. *Assume that the maximum rule arity in F is $a \in \mathbf{N}^+$. For each $k \in F_a$ let $\kappa_k := |z_k(F)|$. Then for every $\Gamma \subseteq y$, every $\varphi \in y$, and every $n \in \mathbf{N}^+$, $P_F(\Gamma, \varphi, n) \leq \prod_{j=1}^n (|\Gamma| + \sum_{k=1}^a \kappa_k(j-1)^k)$. In particular, if $\|\varphi\|_F(\Gamma) = m < \infty$, then $P_F^{\min}(\Gamma, \varphi) \leq \prod_{j=1}^m (|\Gamma| + \sum_{k=1}^a \kappa_k(j-1)^k)$.*

Proof. Fix n . At line j of a proof sequence, one may either choose a hypothesis from Γ , which offers at most $|\Gamma|$ possibilities, or choose a rule application. For a k -ary rule there are at most $(j-1)^k$ ordered choices of previous proof lines from which to draw the premises, and there are κ_k

choices of such a rule. Therefore the number of possible local ways to generate line j is at most $|\Gamma| + \sum_{k=1}^a \kappa_k(j-1)^k$. Multiplying these bounds over $j = 1, \dots, n$ bounds the number of length- n proof sequences ending with any specified target, hence also the number ending with φ . The second displayed inequality is the first applied at $n = m$. $\square$

Remark 8.4. Theorem 8.3 answers the upper-bound question in the most direct way available from cardinal data alone. It is crude, because it counts many sequences that are not genuine proofs and many different construction plans that collapse to the same formulas. Still, it makes explicit where the dependence on $|\Gamma|$, on the rule-cardinalities κ_k , and on proof length enters.

Remark 8.5. The comparison with password finding is suggestive but not exact. A theorem is like a password candidate in one important respect: once a candidate proof is exhibited, local checking is usually much easier than search. A proof is a certificate, just as a password guess is a candidate string. But proof search differs from password search because proofs have reusable internal structure. Lemmas, macro rules, search order, breadth-first layers, and nondeterministic branching all matter. In password search there is usually no analogue of a useful intermediate lemma; a wrong partial password rarely helps the next guess. In theorem proving, by contrast, partial derivations can be retained and reused.

Remark 8.6. A speedup theorem can, in a broad asymptotic sense, play the same role for password finders that proof-complexity speedup plays for theorem provers: it can prove that one search discipline beats another on a family of search instances. The analogy should not be pushed too far, but the structural moral is the same. Search procedures can be compared mathematically by how quickly they reach certified solutions.

9 Universality, Finite Effectiveness, and Computability

Standard computability background may be found in [18]. The results in this section are most transparent on finite fragments, where every state can be coded as a finite object.

Definition 9.1. A SIC over $Y \subseteq y$ is state-restricted if $Y \neq y$.

Definition 9.2. Define the identity SIC on y by $U_{\text{all}} = (C_{\text{all}}, H_{\text{all}})$, where $C_{\text{all}}(\Gamma, m) = \Gamma$, $H_{\text{all}}(\Gamma, m) = 0$ for all $\Gamma \subseteq y$ and all $m \in \mathbf{N}^+$.

Proposition 9.3 (Characterization of simulation into the identity SIC). *Let $M = (C, H)$ be a SIC over y . Then $M \preceq U_{\text{all}}$ if and only if*

$$C(p, m) = p \quad \text{and} \quad H(p, m) = 0$$

for every $p \in S_M$ and every $m \in \mathbf{N}^+$.

Proof. Suppose (f, ρ) is a clocked deterministic simulation from M to U_{all} . Halting preservation gives $H(p, m) = 0$. Since U_{all} is static,

$$f(C(p, m)) = C_{\text{all}}(f(p), \rho(m)) = f(p).$$

Injectivity of f yields $C(p, m) = p$. Conversely, if M is static and never halts, then the identity state map together with the identity clock is a clocked deterministic simulation from M to U_{all} . $\square$

Corollary 9.4. U_{all} is not universal under the principal clocked deterministic simulation relation for the class of all SICs over y .

Proof. Any SIC with a nonconstant stage evolution or a nonzero halting observation fails the characterization in Proposition 9.3. $\square$

Theorem 9.5. *Let $Y \subseteq y$ be inference-closed. The canonical rule-enumeration SIC $E_{F,Y}$ is not universal under clocked deterministic simulation for the class of all rule-driven SICs over Y .*

Proof. Define the always-halted static SIC $Q_Y = (C_Q, H_Q)$ by $C_Q(\Gamma, m) = \Gamma$ and $H_Q(\Gamma, m) = 1$ for every input and stage. It is a rule-driven SIC because it never adds a new formula. If a clocked deterministic simulation from Q_Y into $E_{F,Y}$ existed, halting preservation would give $1 = 0$, since the halting bit of $E_{F,Y}$ is constantly zero. This is impossible. $\square$

Remark 9.6. Theorem 3.30 remains valuable: it supplies a canonical consequence-closure abstraction for every rule-driven SIC. It does not supply a faithful clocked deterministic embedding, and therefore it no longer implies a universality theorem.

Theorem 9.7. *If $Y \subseteq y$ is finite and inference-closed, then $E_{F,Y}$ is deductively finitely effective.*

Proof. For fixed $\Gamma \subseteq Y$, the chain $C_{F,Y}(\Gamma, 1) \subseteq C_{F,Y}(\Gamma, 2) \subseteq \dots$ is a monotone sequence of subsets of the finite set Y . Hence it stabilizes after finitely many steps. Since $E_{F,Y}$ is deductively effective, that eventual fixed point is exactly $\text{Con}_F(\Gamma) \cap Y$. $\square$

Definition 9.8. A finite register-machine program is a finite list of instructions operating on finitely many registers ranging over $\mathbf{N}$. The basic instructions are zeroing a register, incrementing a register, copying one register into another, conditional jump on whether a register is zero, and halt.

Definition 9.9. Assume that $Y \subseteq y$ is effectively coded. A SIC $M = (C, H)$ over Y is computable if there exist total computable functions c and h such that for every code a of a subset $A_a \subseteq Y$ and every $m \in \mathbf{N}^+$, the value $c(a, m)$ is a code for $C(A_a, m)$ and the value $h(a, m)$ is $H(A_a, m)$.

Definition 9.10. A SIC $M = (C, H)$ over an effectively coded domain is computationally effective if there exists a finite register-machine program that, on coded input (a, m) , halts and outputs a code for $C(A_a, m)$ together with the value $H(A_a, m)$.

Theorem 9.11. *Assume that $Y \subseteq y$ is finite, effectively coded, and inference-closed, and assume that the direct-consequence relation of F restricted to Y is decidable. Then $E_{F,Y}$ is computable.*

Proof. Because Y is finite, every subset of Y has a canonical finite code. Given a code for $C_{F,Y}(\Gamma, m)$, one may effectively inspect all relevant tuples from that finite set and decide which formulas of Y are direct consequences. Hence one can compute a code for $D_F(C_{F,Y}(\Gamma, m)) = C_{F,Y}(\Gamma, m + 1)$. Since $H_{F,Y}$ is constantly 0, the halting output is computable as well. $\square$

Definition 9.12. The consequence problem on Y is the decision problem: given $\Gamma \subseteq Y$ and $\psi \in Y$, decide whether $\Gamma \vdash_F \psi$.

Theorem 9.13. *Suppose $M = (C, H)$ is a computable deductively finitely effective SIC over $Y \subseteq y$ such that for every $\Gamma \subseteq Y$ there exists $m \in \mathbf{N}^+$ with $H(\Gamma, m) = 1$ and $C(\Gamma, m) = \text{Con}_F(\Gamma) \cap Y$. Then the consequence problem on Y is decidable.*

Proof. Given codes for Γ and ψ , compute the stages of M until the first m with $H(\Gamma, m) = 1$ appears. By hypothesis such an m exists. Since M is computable, membership in $C(\Gamma, m)$ is decidable. Therefore one can decide whether $\psi \in C(\Gamma, m)$, which is equivalent to $\Gamma \vdash_F \psi$. $\square$

Corollary 9.14. *If the consequence problem on Y is undecidable, then there is no computable deductively finitely effective SIC over Y with the preceding halting-to-completion property.*

Proof. Otherwise the theorem would make the consequence problem decidable. $\square$

Theorem 9.15 (Internal Church–Turing theorem for SICs). *Let $M = (C, H)$ be a SIC over an effectively coded domain. Then M is computationally effective if and only if M is computable.*

Proof. One direction is immediate because register-machine computability gives total computable output functions. For the other direction, if the stage and halting maps are total computable, then the standard equivalence of classical models of computation yields a finite register-machine program computing them [18]. Combining the two computations gives one finite program producing both outputs. $\square$

Remark 9.16. This theorem is internal to the SIC framework. It is not a proof of the historical Church–Turing thesis in its philosophical form.

10 Propositional Proof Systems, p -Simulation, and Feasible Interpolation

This section imports the Cook–Reckhow theory of propositional proof systems [3] into the $F = (x, y, z)$ formal-system vocabulary of Section 1. The organizing source for the results assembled here is Beame’s proof-complexity lecture notes, written up by Sabharwal for the 2000 IAS/Park City Mathematics Institute Graduate Summer School and published as [1]; individual theorems are additionally attributed to their original authors as in that source. The point of the section is a dictionary: a propositional proof system in the classical Cook–Reckhow sense is exactly a syntactically restricted formal system $F = (x, y, z)$ together with a size function on y , and the canonical breadth-first target-search SIC of Section 2 already computes, under that size function, exactly the classical notion of proof size. Several classical theorems then become instances or refinements of theorems already proved for SICs elsewhere in this paper: the Cook–Reckhow theorem becomes a statement about polynomially bounded shortest-proof functions (Theorem 10.6); p -simulation becomes a second, genuinely different preorder alongside those of Section 3 (Definition 10.8); and the resolution lower bound for the pigeonhole principle instantiates a generalization of the Abstract Asymptotic Speedup Principle with an unconditional separation (Corollary 10.21).

10.1 Proof size as a refinement of proof length

Definition 10.1. Let $F = (x, y, z)$ be a formal system. A *size function* on F is a function $|\cdot| \in [y \rightarrow \mathbf{N}^+]$ assigning to each formula the length of its string representation over x . For a finite sequence $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ of members of y , define $\text{size}(\pi) := \sum_{j=1}^n |\pi(j)|$. For $\Gamma \subseteq y$ and $\varphi \in y$, define the shortest-proof *size*

$$\|\varphi\|_F^{\text{size}}(\Gamma) := \min\{\text{size}(\pi) : \pi \text{ is a proof from } \Gamma \text{ with final line } \varphi\},$$

if $\Gamma \vdash_F \varphi$, and $\|\varphi\|_F^{\text{size}}(\Gamma) := \infty$ otherwise.

Remark 10.2. Since every formula has size at least 1, $\text{size}(\pi) \geq |\pi|$ for every proof π , so $\|\varphi\|_F^{\text{size}}(\Gamma) \geq \|\varphi\|_F(\Gamma)$. Concatenation of proofs adds both line counts and sizes, which is the only property of $|\pi|$ used anywhere in Section 2. Consequently the canonical breadth-first SIC of Definition 2.19, the Proof-Horizon Theorem (Theorem 2.12), and the transaction-count apparatus of Section 7

all relativize verbatim under the substitution $|\pi| \mapsto \text{size}(\pi)$; write $B_{F,Y,\varphi}^{\text{size}}$ for the size-relativized canonical breadth-first target-search SIC, whose halting stage on input Γ is exactly $\|\varphi\|_F^{\text{size}}(\Gamma)$.

Definition 10.3. A formal system $F = (x, y, z)$ is *polynomial-time checkable* if there is an algorithm that, given $k \in \mathbf{N}^+$, a rule $r \in z_k(F)$, and a tuple $(a_1, \dots, a_k, \beta) \in y^{k+1}$, decides in time polynomial in $|a_1| + \dots + |a_k| + |\beta|$ whether $r(a_1, \dots, a_k) = \beta$.

Definition 10.4. Let $F = (x, y, z)$ be polynomial-time checkable with a distinguished formula $\perp \in y$, and let Π be a set of finite strings over an auxiliary alphabet together with a relation $\text{Ver} \subseteq \Pi \times \mathcal{P}(y)$, decidable in time polynomial in the length of its first argument and the size of its second, whose intended reading is that π verifies the refutation of Γ . A proof in the sense of Definition 1.4 is the special case in which Π is the set of formula sequences and Ver is the condition of Definition 1.4 with final line $\perp$. Call (F, Π, Ver) a *Cook–Reckhow system* for UNSAT if, for every finite $\Gamma \subseteq y$ coding an unsatisfiable propositional formula, $\Gamma \vdash_F \perp$, and conversely every $\Gamma \vdash_F \perp$ codes an unsatisfiable formula. Define the *proof-complexity function* of F by

$$s_F(\Gamma) := \|\perp\|_F^{\text{size}}(\Gamma)$$

for finite unsatisfiable $\Gamma \subseteq y$. Say F is *polynomially bounded* if there is a polynomial q with $s_F(\Gamma) \leq q(|\Gamma|)$ for every such Γ , where $|\Gamma| := \sum_{a \in \Gamma} |a|$. Where no confusion results we write F for the whole system (F, Π, Ver) , as in s_F just above; every system named in the remainder of this section — resolution, Frege, extended Frege — is of the formula-sequence kind, for which Π and Ver are those of Definition 1.4 and may be left implicit.

Remark 10.5 (Why the auxiliary proof alphabet is needed). Definition 10.4 restates Cook and Reckhow’s original definitions of a proof system and its complexity [3]; see also [1], Definitions 1.1–1.4. Soundness and completeness of the proof system correspond exactly to the two displayed conditions on refutability of Γ .

The generality of (Π, Ver) over the formula sequences of Definition 1.4 is not cosmetic, and it is worth saying exactly where it is used. In a formal system $F = (x, y, z)$ every line of a proof is a well-formed formula that is either a hypothesis or a direct consequence of earlier lines; there is no line at which an arbitrary string may be guessed and then checked. Consequently the forward direction of Theorem 10.6 — which, from $\text{NP} = \text{coNP}$, must manufacture a proof system whose proofs are the polynomial-size NP-witnesses for UNSAT — cannot be carried out with formula sequences alone. Nor can one evade this by taking a unary rule r with $r(\gamma) = \perp$ exactly when γ is unsatisfiable: deciding whether $r(\gamma) = \perp$ is then deciding UNSAT, so such an F is not polynomial-time checkable in the sense of Definition 10.3. The auxiliary alphabet is what makes the classical theorem statable here. Everything else in this section — the size function, the proof-complexity function s_F , p -simulation, and the resolution and Frege examples — lives comfortably inside the formula-sequence special case, since in those systems a refutation *is* a sequence of clauses or formulas.

Theorem 10.6 (Cook–Reckhow Theorem, formal-system form). *There exists a polynomial-time checkable, polynomially bounded Cook–Reckhow system $F = (x, y, z)$ for UNSAT if and only if $\text{NP} = \text{coNP}$.*

Proof. This is Theorem 1.5 of [1], following [3], transcribed into the present vocabulary. If $\text{NP} = \text{coNP}$, then since UNSAT is coNP-complete it lies in NP, so it has polynomial-size witnesses checkable in polynomial time; declaring each such witness a one-line proof from Γ gives a polynomial-time checkable, polynomially bounded F . Conversely, if such an F exists, then nondeterministically guessing a proof of size at most $q(|\Gamma|)$ and checking it in polynomial time (Definition 10.3) places UNSAT in NP; since UNSAT is coNP-complete, this gives $\text{coNP} \subseteq \text{NP}$, and the complementary argument using TAUT gives $\text{NP} \subseteq \text{coNP}$. $\square$

Remark 10.7. Theorem 10.6 shows that the existence question for polynomially bounded formal systems is exactly as hard as NP versus coNP. This gives the abstract asymptotic speedup apparatus of Section 7 a concrete stake: Theorem 7.17 there is schematic, and Theorem 10.6 explains why no single choice of derived rules D can be expected to make every family polynomially bounded, on pain of resolving a major open problem.

10.2 p -simulation: a second preorder on formal systems

Definition 10.8. Let $Y \subseteq y$ be a common coding of propositional formulas, and let $F = (x, y, z)$ and $F' = (x', y', z')$ be polynomial-time checkable Cook–Reckhow systems for UNSAT sharing the domain Y . Say F' p -simulates F , written $F \preceq_p F'$, if there is a polynomial-time computable function h such that for every finite $\Gamma \subseteq Y$ and every proof π of $\perp$ from Γ in F , $h(\pi)$ is a proof of $\perp$ from Γ in F' . Say F and F' are *polynomially equivalent* if $F \preceq_p F'$ and $F' \preceq_p F$.

Remark 10.9. Definition 10.8 restates Definitions 1.6–1.7 of [1], following Cook and Reckhow [3]. It differs in kind, not merely in strength, from every simulation relation of Section 3. Weak, synchronous, and clocked simulation compare two SICs' entire stage-by-stage evolution via a witness on state spaces $S_M \times S_N \subseteq \mathcal{P}(Y) \times \mathcal{P}(Y)$ (Definitions preceding Theorem 3.2 and Propositions 3.4–3.11); recoding (Section 4) additionally requires the translation to be an invertible bijection on the coding universe. p -simulation instead compares two formal systems only through their finite refutations of a shared target $\perp$, via a proof-to-proof map h that need be neither injective nor derived from any map on states. It is, in this precise sense, the coarsest of the comparison notions considered in this paper, exactly as weak simulation was the coarsest SIC-level notion in Theorem 3.2 – though unlike weak simulation, it does not collapse universally, because polynomial-time computability of h is a genuine restriction once size, rather than merely stage number, is being tracked.

Proposition 10.10 (p -simulation bounds proof complexity). *If $F \preceq_p F'$ via h , then there is a polynomial q such that $s_{F'}(\Gamma) \leq q(s_F(\Gamma))$ for every finite unsatisfiable $\Gamma \subseteq Y$.*

Proof. Fix Γ and let π be a shortest proof of $\perp$ from Γ in F , so $\text{size}(\pi) = s_F(\Gamma)$. Since h is computable in time polynomial in the size of its input, $\text{size}(h(\pi))$ is bounded by a polynomial q in $\text{size}(\pi)$, and $h(\pi)$ is a proof of $\perp$ from Γ in F' by Definition 10.8. Hence $s_{F'}(\Gamma) \leq \text{size}(h(\pi)) \leq q(s_F(\Gamma))$. $\square$

Proposition 10.11 (p -simulation is a preorder). *The relation $\preceq_p$ is reflexive and transitive.*

Proof. Reflexivity is witnessed by the identity translation. For transitivity, if $F \preceq_p F'$ via h and $F' \preceq_p F''$ via h' , then $h' \circ h$ is polynomial-time computable, being a composition of two polynomial-time computable functions, and it sends a proof of $\perp$ from Γ in F to one in F'' by two applications of Definition 10.8. This mirrors Proposition 3.19, the corresponding composition fact for clocked deterministic simulation. $\square$

Corollary 10.12. *Polynomial equivalence of formal systems is an equivalence relation, and $\preceq_p$ induces a partial order on its equivalence classes, exactly as $\preceq$ does for SICs (Corollary following Proposition 3.19).*

Theorem 10.13 (Frege systems are polynomially equivalent). *Every two Frege systems, formalized as formal systems $F = (x, y, z)$ whose rule set z consists of finitely many axiom schemas and finitely many sound, implicational complete inference rules such as modus ponens, are polynomially equivalent under $\preceq_p$.*

Proof. This is Theorem 1.8 of [1]. Given two such systems with axiom/rule sets z and z' , every instance of a schema of z' has a constant-size proof in z because z is implicationally complete, and this fixed finite collection of constant-size proofs can be spliced into any z' -proof, line by line, in time polynomial in the size of that proof, yielding an F -proof of the same conclusion of polynomially related size. Since the two systems are arbitrary Frege systems, both directions of $\preceq_p$ follow. $\square$

10.3 The pigeonhole principle: an unconditional instance of asymptotic speedup

Definition 10.14. Fix a countable set of Boolean variables. Let x consist of the variables, their negations, and a disjunction separator; let y consist of all finite clauses (finite sets of literals) over x , with size function $|\cdot|$ one more than the number of literal occurrences, so that the empty clause has size 1 and $|\cdot|$ takes values in $\mathbf{N}^+$ as Definition 10.1 requires; let $\perp$ be the empty clause; and let $z = \{\text{Res}\}$, where Res is the binary rule

$$\text{Res}(C \vee p, D \vee \neg p) := C \vee D,$$

defined whenever the pivot variable p occurs positively in the first clause and negatively in the second. Write $F_{\text{res}} = (x, y, z)$ for the resulting formal system.

Remark 10.15. F_{res} is polynomial-time checkable in the sense of Definition 10.3, since verifying a single resolution step takes time polynomial in the size of the two premises and the conclusion. It is a Cook–Reckhow system for UNSAT restricted to CNF inputs, and it is the formal system underlying the canonical rule-enumeration and target-search SICs $E_{F_{\text{res}}, Y}$ and $E_{F_{\text{res}}, Y, \perp}$ of Definitions 2.16 and 2.19.

Definition 10.16. For $n \geq 1$, let $\Gamma_n \subseteq y$ be the standard CNF encoding of the pigeonhole principle PHP_n^{n-1} over variables $p_{i,j}$ ($1 \leq i \leq n$, $1 \leq j \leq n-1$), consisting of the totality clauses $\bigvee_j p_{i,j}$ and the injectivity clauses $\neg p_{i,j} \vee \neg p_{i',j}$ for $i \neq i'$, as in [1], Section 2.1. Each Γ_n is unsatisfiable, coding the fact that n pigeons cannot be injectively mapped into $n-1$ holes.

Theorem 10.17 (Haken’s resolution lower bound, formal-system form). *There is a constant $c > 0$ such that $s_{F_{\text{res}}}(\Gamma_n) \geq 2^{cn}$ for every n . Equivalently, the size-relativized canonical breadth-first target-search SIC $B_{F_{\text{res}}, Y, \perp}^{\text{size}}$ does not halt on input Γ_n before stage 2^{cn} .*

Attribution and sketch. This is Haken’s theorem [9], as presented via the width-size method of Ben-Sasson and Wigderson in [1], Sections 2.1.4–2.2. In outline: a resolution refutation is first shown, by a restriction argument, to be reducible to one with no large clauses while still refuting a nontrivial restricted instance; every refutation of that restricted instance must contain a clause of “medium” complexity, in the sense of the number of critical assignments it certifies; and every medium-complexity clause must itself be large. Combining these facts with the restriction argument yields an exponential lower bound on the number of resolution steps, hence on size. The final “equivalently” clause is the size-relativized instance of the Proof-Horizon Theorem, Theorem 2.12, noted in the remark following Definition 10.1. $\square$

Theorem 10.18 (Extended Frege upper bound). *There is a polynomial p and an extended Frege system F_{EF} , in the sense of Theorem 10.13 together with extension rules for auxiliary variables, such that $s_{F_{\text{EF}}}(\Gamma_n) \leq p(n)$ for every n .*

Attribution. This is the extended Frege translation, due to Buss, of the ordinary inductive proof of the pigeonhole principle, presented in [1], Section 2.1.2: the induction on n used to prove PHP_n^{n-1}

from PHP_{n-1}^{n-2} is internalized by extension variables recording the induced mapping on $n-1$ pigeons into $n-2$ holes, giving a polynomial-size proof at each stage of the induction and hence a polynomial-size proof overall. $\square$

Theorem 10.19 (Two-System Size-Speedup Principle). *Let $F = (x, y, z)$ and $F' = (x', y', z')$ be polynomial-time checkable Cook–Reckhow systems for UNSAT sharing a common family $(\Gamma_n)_{n \geq 1}$ of finite unsatisfiable inputs. Suppose there are $p, q \in [\mathbf{N}^+ \rightarrow \mathbf{N}^+]$ such that:*

1. $s_{F'}(\Gamma_n) \leq p(n)$ for all n ;
2. $s_F(\Gamma_n) \geq q(n)$ for all n ;
3. $q(n)/p(n)$ is unbounded.

Then the size-relativized canonical target-search SIC $B_{F', Y', \perp}^{\text{size}}$ is asymptotically faster on the family $(\Gamma_n, \perp)$ than every proof-search realization in F .

Proof. This is proved exactly as Theorem 7.17: hypothesis (1) bounds the size-relativized transaction cost of F' by $p(n)$, hypothesis (2) bounds from below every F -proof's size by $q(n)$, and hypothesis (3) makes the ratio unbounded, giving an asymptotic separation. $\square$

Remark 10.20. Theorem 10.19 generalizes Theorem 7.17 in one essential respect. There, F' was required to be a macro-augmented conservative extension F^D of the very same F (Definition preceding Theorem 7.12), so that Proposition 7.2-adjacent conservativity was automatic and unconditional; the theorem's force came from a hypothesis relating $\tau_{E_{F^D, Y, \varphi_n}}$ to lower bounds on F -proof length that were assumed, not proved, within the paper. Here F_{res} and F_{EF} are two independent formal systems with no derived-rule relationship: F_{EF} introduces extension variables outside the language of F_{res} , so it is not literally F_{res}^D for any set D of derived rules on F_{res} 's own formula set. Theorem 10.19 therefore records a strictly more general comparison, at the price of dropping the automatic soundness-relative-to- F guarantee that conservativity provided.

Corollary 10.21 (An unconditional instance of abstract speedup). *Instantiating Theorem 10.19 with $F = F_{\text{res}}$, $F' = F_{\text{EF}}$, and (Γ_n) the pigeonhole family of Definition 10.16, the hypotheses are met with $p(n)$ polynomial and $q(n) = 2^{cn}$, by Theorems 10.17 and 10.18. This is the paper's first unconditional instance of the abstract speedup phenomenon: the finite Conservative Compilation Theorem (Theorem 7.12) speaks only of finitely many designated targets, and the Abstract Asymptotic Speedup Principle (Theorem 7.17) is stated under a hypothesis rather than exhibited. Here the hypothesis is a theorem.*

10.4 Automatizability

Definition 10.22. Let $F = (x, y, z)$ be a polynomial-time checkable Cook–Reckhow system for UNSAT over an effectively coded domain Y , and let $t \in [\mathbf{N}^+ \rightarrow \mathbf{N}^+]$. Say F is *t-automatizable* if there is a computationally effective SIC $M = (C, H)$ (in the sense preceding Theorem 7.5) that is a target-search ATP for $\perp$, such that for every finite unsatisfiable $\Gamma \subseteq Y$, M halts on input Γ within $t(s_F(\Gamma) + |\Gamma|)$ elementary computation steps. Say F is *automatizable* if it is *t-automatizable* for some polynomial t .

Remark 10.23. This refines the computationally effective and computable SIC apparatus of Section 9 by tying the resource bound to the shortest-refutation size $s_F(\Gamma)$ of Definition 10.4, rather than merely requiring that some halting computation exist.

Theorem 10.24 (Tree resolution is quasi-polynomially automatizable). *Tree-like (Davis–Putnam) resolution is t -automatizable for $t(m) = m^{O(\log m)}$.*

Attribution. This is Corollary 3.4 of [1], due to [3, 1] together with the width-size relation for tree-like resolution (Theorem 10.17’s antecedent, due to Ben-Sasson and Wigderson): every size- m tree-like refutation of a formula on ℓ variables converts to one of width $O(\log m)$, of which there are only $\ell^{O(\log m)}$ many clauses to search through by breadth-first enumeration bounded to that width, giving the stated running time. $\square$

10.5 Feasible interpolation as a proof-complexity barrier theorem

Definition 10.25. Let $A(\bar{p}, \bar{r})$ and $B(\bar{p}, \bar{s})$ be formulas in which the variable blocks $\bar{r}$ and $\bar{s}$ are disjoint from each other, so that $\bar{p}$ is the block common to both, and suppose the conjunction $A(\bar{p}, \bar{r}) \wedge B(\bar{p}, \bar{s})$ is unsatisfiable. An *interpolant* for the pair (A, B) is a Boolean function I of the shared block $\bar{p}$ such that, for every truth assignment $\bar{a}$ to $\bar{p}$,

$$I(\bar{a}) = 0 \implies A(\bar{a}, \bar{r}) \text{ is unsatisfiable in } \bar{r}, \quad I(\bar{a}) = 1 \implies B(\bar{a}, \bar{s}) \text{ is unsatisfiable in } \bar{s}.$$

Since $A \wedge B$ is unsatisfiable, at least one of the two disjuncts holds at every $\bar{a}$, so an interpolant always exists; the content of the definition is entirely in how small a circuit computes one.

Definition 10.26. Let $F = (x, y, z)$ be a Cook–Reckhow system for UNSAT and let $g \in [\mathbf{N}^+ \rightarrow \mathbf{N}^+]$. Say F has *g -interpolation* if for every pair (A, B) as in Definition 10.25 whose conjunction has $s_F(\{A, B\}) \leq m$, there is a Boolean circuit of size at most $g(m)$ computing an interpolant for (A, B) . Say F has *feasible interpolation* if g may be taken to be a polynomial.

Theorem 10.27 (Feasible interpolation for resolution and cutting planes). *Resolution has feasible, monotone interpolation, and cutting planes has feasible, monotone interpolation with interpolants computed by circuits over the reals.*

Attribution. These are theorems of Krajíček and of Pudlák, respectively, presented in [1], Theorem 3.13 and Theorem 3.14. For resolution, the interpolant circuit is built directly from the refutation dag by propagating, at each derived clause, which side of the partition falsifies it under a given assignment to $\bar{p}$; the construction is size-preserving up to a polynomial factor. $\square$

Theorem 10.28 (Interpolation lower-bound theorem). *If a Cook–Reckhow system F has feasible interpolation and $\text{NP} \not\subseteq \text{P/poly}$, then F is not polynomially bounded.*

Attribution and sketch. This is Theorem 3.16 of [1], following Krajíček and Pudlák. If F were both polynomially bounded and feasibly interpolable, then for the standard family of clique-versus-coloring formulas, which are unsatisfiable exactly when a graph has both a k -clique and a proper $(k-1)$ -coloring witnessed disjointly, the resulting polynomial-size interpolant circuits would decide graph satisfiability instances in P/poly , contradicting $\text{NP} \not\subseteq \text{P/poly}$ once the encoding is chosen so that the interpolant separates an NP-hard promise problem. $\square$

Corollary 10.29. *Since cutting planes has feasible monotone interpolation (Theorem 10.27), the clique-coloring formulas of Theorem 10.28 require exponential-size cutting planes refutations, matching [1], Theorem 3.18.*

Remark 10.30. Theorem 10.28 is structurally a proof-complexity analogue of two barrier theorems already in this paper. The No-Free-Recoding-Speedup Theorem (Theorem 4.34) says that one form

of structural strength – pure invertible recoding – cannot by itself buy transaction-count speedup, while the Tarski–Gödel barrier (Theorem 4.26) says that a SIC’s very expressiveness forecloses a complete internal truth predicate. Theorem 10.28 says, in the same spirit, that a proof system’s very expressiveness – specifically, its ability to interpolate feasibly – forecloses polynomial boundedness under a standard complexity hypothesis. In each case a capability that looks like strength (recoding power, self-descriptive richness, interpolation) is shown to force a limitation elsewhere in the system.

Remark 10.31 (Limitations). Under the same kind of hardness assumption used positively above, sufficiently strong proof systems provably *lack* feasible interpolation, so the technique of Theorem 10.28 cannot reach them. If one-way functions exist, Frege systems do not have feasible interpolation [1], Theorem 3.19, by an argument encoding a secret-key-agreement protocol into the interpolation problem; if factoring Blum integers is hard, no proof system polynomially simulating AC^0 -Frege has feasible interpolation [1], Theorem 3.20. These limitation theorems are recorded here without formal-system translation, since they concern proof systems whose formalization as $F = (x, y, z)$ is identical in kind to Definition 10.14 and adds no further structure.

10.6 An open problem, restated

Conjecture 10.32. Let $(\Gamma_n)_{n \geq 1}$ be a family of random 3-CNF formulas at constant density above the satisfiability threshold, in the sense of [1], Section 2.3.1. There is no Frege system F and polynomial p such that $s_F(\Gamma_n) \leq p(n)$ for almost all n . Equivalently, no size-relativized canonical breadth-first target-search SIC $B_{F,Y,\perp}^{\text{size}}$ over a Frege system F reaches $\perp$ at polynomial stage, almost surely, on random 3-CNF inputs.

Remark 10.33. This is the SIC-vocabulary form of the leading open problem of [1], Lecture 5, where it is left open exactly as here: the corresponding fact is known for resolution (an instance of Theorem 10.17’s method, applied to random formulas via sub-critical expansion) but is open for Frege systems, and even for boundeddepth Frege systems only the restriction method of [1], Lecture 4, is available, which so far has not yielded a matching lower bound. Consistent with this paper’s practice of naming conjectures rather than asserting unproved claims (compare the three conjectures of Version 42, two of which were later refuted or proved in Versions 43–44), Conjecture 10.32 is recorded here as open, not resolved.

10.7 Algebraic Proof Systems: Nullstellensatz, Polynomial Calculus, and Gröbner Bases

The proof systems of the preceding subsections manipulate clauses. This subsection records a second family, which manipulates polynomials, and locates it in the same vocabulary. The motivation is concrete: a satisfiability algorithm based on Gröbner basis computation looks, on first inspection, as though it might run in polynomial time, and the apparatus assembled above is exactly what is needed to say why it does not — and, more usefully, what it does buy.

Arithmetizing a formula

Definition 10.34 (Boolean arithmetization). Let φ be a propositional formula over variables $p_1, \dots, p_n$. Define a polynomial $\tau(\varphi) \in \mathbf{F}_2[x_1, \dots, x_n]$ by recursion on the structure of φ :

$$\begin{aligned} \tau(p_i) &:= x_i, & \tau(\neg\varphi_1) &:= 1 + \tau(\varphi_1), \\ \tau(\varphi_1 \wedge \varphi_2) &:= \tau(\varphi_1)\tau(\varphi_2), & \tau(\varphi_1 \vee \varphi_2) &:= 1 + (1 + \tau(\varphi_1))(1 + \tau(\varphi_2)), \end{aligned}$$

all arithmetic being performed in $\mathbf{F}_2$. Write

$$J(\varphi) := \langle \tau(\varphi) + 1, x_1^2 + x_1, \dots, x_n^2 + x_n \rangle \subseteq \mathbf{F}_2[x_1, \dots, x_n]$$

for the associated ideal. The generators $x_i^2 + x_i$ are the *field equations*; over $\mathbf{F}_2$ they coincide with $x_i^2 - x_i$ and force every coordinate of a common zero to be 0 or 1.

Proposition 10.35 (Correctness of the arithmetization). *For every truth assignment $a \in \{0, 1\}^n$, identifying true with 1 and false with 0,*

$$\tau(\varphi)(a) = 1 \iff a \models \varphi.$$

Consequently φ is satisfiable if and only if the variety of $J(\varphi)$ over $\mathbf{F}_2$ is nonempty, and, since $J(\varphi)$ contains the field equations and is therefore radical and zero-dimensional,

$$\varphi \text{ is unsatisfiable} \iff 1 \in J(\varphi) \iff \text{the reduced Gröbner basis of } J(\varphi) \text{ is } \{1\}.$$

Proof. The equivalence $\tau(\varphi)(a) = 1 \iff a \models \varphi$ is an induction on φ . For a variable it is the identification of truth values with elements of $\mathbf{F}_2$. For negation, $1 + t = 1$ iff $t = 0$ in $\mathbf{F}_2$. For conjunction, a product of two elements of $\{0, 1\} \subseteq \mathbf{F}_2$ is 1 iff both factors are 1. For disjunction, $1 + (1 + t_1)(1 + t_2) = 1$ iff $(1 + t_1)(1 + t_2) = 0$ iff at least one t_i equals 1.

The common zeros of $J(\varphi)$ in $\overline{\mathbf{F}_2}^n$ are exactly the points of $\{0, 1\}^n$ at which $\tau(\varphi) + 1$ vanishes, that is, the assignments satisfying φ . The last equivalence is the weak Nullstellensatz together with the observation that an ideal containing all field equations has a finite variety and is radical, so it is the unit ideal precisely when that variety is empty; and 1 lies in an ideal precisely when the reduced Gröbner basis with respect to any monomial order is $\{1\}$. $\square$

Remark 10.36. Proposition 10.35 converts satisfiability into an ideal membership question and is the basis of every algebraic SAT procedure. It is a translation, not an algorithm: it says where the answer lives, not what it costs to extract. Working directly in the Boolean ring $\mathbf{F}_2[x]/\langle x_i^2 - x_i \rangle$, rather than carrying the field equations as explicit generators, is the setting of Boolean Gröbner basis theory [19], where the coefficient ring may itself be a Boolean ring rather than a field; the complexity statements below are unaffected by that choice of presentation. The whole content of the subsections below is the cost.

Two algebraic formal systems

Definition 10.37 (Polynomial Calculus). Fix a field K and variables $x_1, \dots, x_n$. Let x be an alphabet sufficient to write polynomials, let

$$y := K[x_1, \dots, x_n]/\langle x_i^2 - x_i \rangle$$

be the multilinear polynomials, with $\perp := 1$ the distinguished formula, and let z consist of the finitary rules

$$\text{Lin}_{\alpha, \beta}(f, g) := \alpha f + \beta g \quad (\alpha, \beta \in K), \quad \text{Mult}_i(f) := x_i f \quad (1 \leq i \leq n).$$

Write $F_{\text{PC}} = (x, y, z)$ for the resulting formal system, the *Polynomial Calculus* over K . Its *size* function is the number of monomials, and the *degree* of a refutation is the maximum total degree of a polynomial occurring in it. The *Nullstellensatz* system F_{NS} is the static variant in which a refutation of $\{f_1, \dots, f_m\}$ is a single tuple $(g_1, \dots, g_m)$ with $\sum_j g_j f_j = 1$ identically.

Proposition 10.38. F_{PC} is polynomial-time checkable in the sense of Definition 10.3, and, restricted to inputs $J(\varphi)$ arising from Definition 10.34, it is a Cook–Reckhow system for UNSAT in the sense of Definition 10.4.

Proof. Verifying an application of $\text{Lin}_{\alpha,\beta}$ or Mult_i is a polynomial arithmetic operation on the monomial lists of the premises and conclusion, so it is performed in time polynomial in their sizes. Soundness holds because both rules preserve the property of vanishing on every common zero of the input polynomials; hence deriving 1 certifies that no common zero exists, which by Proposition 10.35 is unsatisfiability of φ . Completeness is the weak Nullstellensatz in the same proposition: if φ is unsatisfiable then $1 \in J(\varphi)$, and any representation of 1 as a combination of the generators is convertible into a derivation of 1 by the two rules. Since $\perp = 1$, refutability and derivation of $\perp$ coincide. $\square$

Remark 10.39 (Gröbner basis computation is proof search in F_{PC}). Buchberger’s algorithm, and its descendants F_4 and F_5 , compute a Gröbner basis of $J(\varphi)$ by repeatedly forming S -polynomials and reducing them — operations that are exactly compositions of $\text{Lin}_{\alpha,\beta}$ and Mult_i . A run of the algorithm that outputs $\{1\}$ therefore is a refutation in F_{PC} , and the canonical target-search SIC $E_{F_{PC},Y,\perp}$ of Definition 2.19 is the associated proof-search machine. This is the observation that makes the proof-complexity lower bounds below apply to Gröbner basis algorithms rather than merely to an abstract calculus, and it is why the founding paper of the subject is titled for the algorithm rather than for the proof system [7].

The negative direction

Theorem 10.40 (Degree and size lower bounds for Polynomial Calculus). *Let (Γ_n) be the pigeonhole family of Definition 10.16, arithmetized as in Definition 10.34. Then every F_{PC} refutation of Γ_n has degree $\Omega(n)$, and there is a constant $c > 0$ with*

$$s_{F_{PC}}(\Gamma_n) \geq 2^{cn}$$

for all n . The same holds for random 3-CNF formulas at constant density above the satisfiability threshold, almost surely.

Attribution. The degree lower bound for the pigeonhole principle is Razborov’s [17]; the size lower bound follows from it by the size–degree tradeoff of Impagliazzo, Pudlák and Sgall [12], who state the consequence for the Gröbner basis algorithm explicitly. The statement for random 3-CNF is Ben-Sasson and Impagliazzo [6]. Note that Conjecture 10.32 is *not* thereby settled: it concerns Frege systems, which are not p -simulated by F_{PC} . $\square$

Corollary 10.41 (No polynomial-time SAT algorithm of Gröbner type). *There is no algorithm which, on input a propositional formula φ , decides satisfiability by computing a Gröbner basis of $J(\varphi)$ and testing whether it equals $\{1\}$, and which runs in time polynomial in $|\varphi|$. This holds unconditionally: no complexity-theoretic hypothesis is assumed.*

Proof. By Remark 10.39 such an algorithm’s run on an unsatisfiable input is an F_{PC} refutation, and the algorithm’s running time is at least the size of the object it writes down. By Theorem 10.40 that size is $2^{\Omega(n)}$ on the pigeonhole family, whose encodings have size polynomial in n . Hence the running time is exponential in the input size on that family. $\square$

Remark 10.42 (Where the polynomial-time intuition goes wrong). Two facts about Gröbner basis computation are easily run together, and the distinction between them is what Corollary 10.41 turns on.

First, the FGLM algorithm of Faugère, Gianni, Lazard and Mora [8] is a *change of monomial order*: it takes a Gröbner basis already computed with respect to one order and returns one with respect to another. It cannot produce the first basis, and the first basis is the expensive object. Invoking FGLM as the step that computes G presupposes the very computation whose cost is at issue.

Second, FGLM runs in time $O(nD^3)$ where $D = \dim_K(K[x_1, \dots, x_n]/J)$ is the dimension of the quotient algebra — for a zero-dimensional radical ideal, the number of points of the variety. For $J(\varphi)$ that number is the number of satisfying assignments of φ , which is at most 2^n and is frequently that large. Polynomial in D is not polynomial in n ; the field equations alone already give $\dim_K K[x]/\langle x_i^2 - x_i \rangle = 2^n$.

Neither observation is a defect of FGLM, which does what it claims. The error is in reading “polynomial-time” as a statement about the input formula when it is a statement about the quotient dimension.

The positive direction

The lower bounds close one route and leave another open, and it is worth being precise about what survives, because a genuine and unconditional speedup lives here.

Theorem 10.43 (Polynomial Calculus p -simulates resolution). $F_{\text{res}} \preceq_p F_{\text{PC}}$ over every field K , in the sense of Definition 10.8. Moreover the translation carries a resolution refutation of width w to a Polynomial Calculus refutation of degree w .

Attribution and sketch. A clause $C = \bigvee_{i \in P} p_i \vee \bigvee_{j \in N} \neg p_j$ is translated to the monomial $\prod_{i \in P} (1 - x_i) \prod_{j \in N} x_j$, which vanishes exactly on the assignments satisfying C . A resolution step on pivot p is then simulated by a bounded number of applications of Mult_i and $\text{Lin}_{\alpha, \beta}$, so the translation is computable in polynomial time and the degree of the image is the width of the source. The correspondence between width and degree is due to Clegg, Edmonds and Impagliazzo [7]. $\square$

Theorem 10.44 (Tseitin formulas separate the two systems). Let (T_n) be the Tseitin family: for a bounded-degree expander graph $G_n = (V, E)$ on n vertices with an odd total charge $\sigma : V \rightarrow \mathbf{F}_2$, the formula T_n is the CNF encoding of the parity constraints $\bigoplus_{e \ni v} x_e = \sigma(v)$ for $v \in V$. Each T_n is unsatisfiable. Then

$$s_{F_{\text{PC}}}(T_n) \leq p(n) \quad \text{over } K = \mathbf{F}_2 \quad \text{while} \quad s_{F_{\text{res}}}(T_n) \geq 2^{\Omega(n)}$$

for some polynomial p .

Attribution and sketch. Over $\mathbf{F}_2$ the parity constraint at v is the linear polynomial $\sum_{e \ni v} x_e + \sigma(v)$, and summing these over all $v \in V$ makes every edge variable appear twice, hence cancel, leaving $\sum_v \sigma(v) = 1$: a derivation of $\perp$ using $|V|$ applications of $\text{Lin}_{1,1}$ and no multiplications at all. The bounded degree of G_n keeps the translation from the CNF encoding to these linear polynomials of polynomial size. The exponential resolution lower bound for Tseitin formulas over expanders is Urquhart’s [27]. $\square$

Corollary 10.45 (A second unconditional instance of abstract speedup). Instantiating the Two-System Size-Speedup Principle (Theorem 10.19) with $F = F_{\text{res}}$, $F' = F_{\text{PC}}$ over $\mathbf{F}_2$, and $(\Gamma_n) = (T_n)$

the Tseitin family, the hypotheses are met with $p(n)$ polynomial and $q(n) = 2^{\Omega(n)}$ by Theorem 10.44. Hence the size-relativized canonical target-search SIC $B_{F_{PC}, Y, \perp}^{\text{size}}$ is asymptotically faster on the Tseitin family than every proof-search realization in F_{res} .

Remark 10.46 (The two unconditional instances point in opposite directions). Corollary 10.21 exhibited extended Frege beating resolution on the pigeonhole family. Corollary 10.45 exhibits Polynomial Calculus beating resolution on the Tseitin family. Theorem 10.40 shows that Polynomial Calculus does *not* beat resolution on the pigeonhole family — both are exponential there. So the two systems are incomparable in strength on these families, and neither dominates:

$$F_{\text{res}} \preceq_p F_{PC}, \quad F_{PC} \not\preceq_p F_{\text{res}} \text{ (by Theorem 10.44 with Proposition 10.10),}$$

while F_{PC} and F_{EF} are separated by Theorem 10.40 against Theorem 10.18. The algebraic route is therefore a genuine gain over clause-based search on an identifiable family, and a genuine loss on another. That is the honest form of the claim that Gröbner bases help with satisfiability: they help, demonstrably and unconditionally, on parity-structured instances, and provably not in general.

Theorem 10.47 (Bounded-degree refutations are findable). *Fix d . There is an algorithm which, given an unsatisfiable arithmetized input on n variables possessing an F_{PC} refutation of degree at most d , finds one in time $n^{O(d)}$. Consequently the restriction of F_{PC} to degree- d refutations is t -automatizable in the sense of Definition 10.22 for $t(m) = m^{O(d)}$.*

Sketch. The multilinear polynomials of degree at most d in n variables form a K -vector space of dimension $\sum_{j \leq d} \binom{n}{j} = n^{O(d)}$. Closing the input under Mult_i and taking linear spans, while discarding everything of degree exceeding d , is therefore a linear-algebra computation in a space of that dimension, and $\perp = 1$ is derivable in degree d if and only if it lies in the resulting span. Gaussian elimination on a matrix of side $n^{O(d)}$ gives the stated bound. The method is that of Clegg, Edmonds and Impagliazzo [7]. $\square$

Remark 10.48 (What is actually salvaged). Three positive statements survive the lower bounds, and they are what an algebraic SAT effort should be organized around.

1. *Strict gain on structured instances.* Theorem 10.44 and Corollary 10.45: on parity-structured formulas the algebraic system is exponentially better than resolution, unconditionally. Instances carrying linear-algebraic structure over a finite field are exactly where the method earns its keep.
2. *Tractability under a degree bound.* Theorem 10.47: the obstruction is degree, not the algebraic setting as such. Where a family admits low-degree refutations, they are found in polynomial time for fixed degree. The research question that replaces “is SAT in P?” is the answerable one: which families admit low-degree refutations?
3. *Simulation without loss.* Theorem 10.43: nothing is given up by moving to the algebraic setting, since resolution is p -simulated. The algebraic system is at least as strong on every family and strictly stronger on some.

What does not survive is the unrestricted claim. By Corollary 10.41 no Gröbner-basis satisfiability procedure runs in polynomial time on all inputs, and this is settled rather than open.

A Collection of Examples of Objects Discussed Herein

Table 1: A Collection of Examples of Objects Discussed Herein: SICs

Name of object	Object type	Section first found	Role
$E_{F,Y}$	SIC; enumerating ATP	Section 2	Canonical brute-force benchmark
$E_{F,Y,\varphi}$	SIC; target-search ATP	Section 2	Canonical target benchmark
$B_{F,Y}$	SIC; breadth-first ATP	Section 2	Shortest-proof horizon machine
$B_{F,Y,\varphi}$	SIC; breadth-first target-search ATP	Section 2	Target horizon machine
M	SIC	Section 2	Generic rule-driven prover
M_b	SIC induced by a branch	Section 5	Deterministic shadow of a branch
M^t	Recoded SIC	Section 4	Isomorphic translated copy
U_{all}	SIC	Section 9	Static identity benchmark; not universal
M_P	SIC; target-search ATP	Section 4	Minimal Picard self-certifier

Table 2: A Collection of Examples of Objects Discussed Herein: Formal systems and rule extensions

Name of object	Object type	Section first found	Role
$F = (x, y, z)$	Formal system	Section 1	Ambient deductive system
F^D	Formal system with derived rules	Section 7	Macro-augmented conservative extension
F^{ns}	Nonstandard formal extension	Opening section	Ambient system for infinitesimal-style proofs
F^t	Recoded formal system	Section 4	Isomorphic translated formal system
Weak successor-set theory	Formal system family	Section 7	Minimal set background for $\text{Succ}(\{0\}) = \{0, 1\}$

Table 3: A Collection of Examples of Objects Discussed Herein: Simulations and search policies

Name of object	Object type	Section first found	Role
a_M	State-abstraction map	Section 3	Consequence-closure abstraction into $E_{F,Y}$
$a_{M,\varphi}$	State-abstraction map	Section 3	Target-search closure abstraction
$S_M \times S_N$	Weak simulation witness	Section 3	Witness for universal weak simulation
id_{S_M}	Synchronous simulation map	Section 3	Reflexive strong simulation
(f, ρ)	Clocked simulation pair	Section 3	Principal time-rescaled simulation
$(g \circ f, \sigma \circ \rho)$	Clocked simulation pair	Section 3	Transitivity construction
f_K	Noncomputable synchronous witness	Section 3	Shows effectivity is not automatic
Σ	Nondeterministic proof-search policy	Section 5	Branch generator / heuristic
Depth- K pair	Finite-horizon clocked simulation	Section 3	Limited comparison notion
Proof interpretation / extraction map	Translation or abstraction device	Opening section	Standard shadow of a nonstandard proof
f_t	Synchronous simulation map	Section 4	Strong witness for $M \sim M^t$
t	Uniform recoding / translator	Section 4	Invertible coding isomorphism

Table 4: A Collection of Examples of Objects Discussed Herein: Proof measures and closure operators

Name of object	Object type	Section first found	Role
$\text{Con}_F(\Gamma)$	Consequence operator value	Section 1	Least inference-closed extension
D_F	One-step closure operator	Section 1	Direct-consequence expansion
$\ \varphi\ _F(\Gamma)$	Proof-length measure	Section 2	Length of a shortest proof
$L_F(\Gamma, m)$	Proof layer	Section 2	Exact shortest-proof stratum
$\tau_M(\Gamma, \varphi)$	Search-cost measure	Section 7	Target transaction count
$P_F(\Gamma, \varphi, n)$	Counting function	Section 8	Number of length- n proofs
$P_F^{\min}(\Gamma, \varphi)$	Counting function	Section 8	Number of shortest proofs
Extracted bound or witness	Computational content	Opening section	Standard data mined from a proof
θ from Theorem 4.25	Fixed-point sentence	Section 4	Sentence about its recoded code
$\ \varphi\ _F^{\text{size}}(\Gamma)$	Size-relativized proof measure	Section 10	Shortest-refutation size
$s_F(\Gamma)$	Proof-complexity function	Section 10	Size of a shortest refutation of $\perp$

Table 5: A Collection of Examples of Objects Discussed Herein: Propositional proof complexity

Name of object	Object type	Section first found	Role
F_{res}	Formal system	Section 10	Resolution as an $F = (x, y, z)$ instance
F_{EF}	Formal system	Section 10	Extended Frege system
Γ_n	Finite unsatisfiable input family	Section 10	Pigeonhole principle PHP_n^{n-1}
$\preceq_p$	Preorder on formal systems	Section 10	p -simulation
$B_{F,Y,\varphi}^{\text{size}}$	SIC; size-relativized breadth-first ATP	Section 10	Size-tracking target benchmark
I	Interpolant	Section 10	Boolean function separating $\bar{p}$ -side from $\bar{q}$ -side unsatisfiability

11 Conclusion

Part I separates four mathematically different comparison notions. Weak simulation is the original unconstrained relational shadowing condition, and Theorem 3.2 shows that it collapses universally. Synchronous deterministic simulation is a faithful injective stage-for-stage embedding. State abstraction may preserve an evolution equation while identifying states or ignoring halting. Clocked deterministic simulation is the principal relation: it retains faithful state and halting preservation but allows a nondecreasing unbounded clock to rescale time and represent idle or extra stages.

The distinction is necessary. Proposition 3.11 shows that synchronous simulation can fail for cardinality reasons even when two machines obey dynamics of the same form. Theorem 3.13 shows that a one-stage delay can make two matching machines synchronously incomparable. Theorem 3.23 then shows that clocked simulation identifies them. Thus synchronous simulation remains a useful strongest relation, but clocked simulation is the better general notion for comparison, equivalence, and depth.

The clocked relation is reflexive and transitive and induces a partial order on its equivalence classes. It does not collapse: the identity SIC and the canonical rule-enumeration SIC still fail to be universal because clock changes cannot repair incompatible state evolution or contradictory halting bits. Canonical consequence-closure maps remain useful abstractions, but their possible many-to-one character keeps them outside the faithful simulation relations.

The notation $[S_M \rightarrow S_N]$ includes noncomputable functions. Theorem 3.27 exhibits a noncomputable synchronous witness, so effectivity must be imposed rather than inferred. Effective clocked simulation does this for SICs with effective state presentations. On infinite formula domains, the full power-set state space is generally too large to be coded by natural numbers, so executable applications should explicitly select finite, reachable, or otherwise effectively represented state domains.

The Proof-Horizon Theorem, Branch-Covering Theorem, Conservative Compilation Theorem, theorem-graph constructions, and proof-mining discussion remain independent of weak-simulation collapse. Invertible recodings satisfy the strongest simulation notion: the map $f_t(p) = t[p]$ is a synchronous isomorphism and hence also a clocked equivalence. Transaction counts are preserved pointwise, which now yields the No-Free-Recoding-Speedup Theorem rather than merely a conjecture.

Three conjectures left open in Version 42 of the source paper are closed here. The Self-Audit Compression Conjecture holds: Theorem 4.28 shows that once the transport theorem is proved uniformly, verifying that a recoded copy is a SIC or ATP costs only a polynomial-length proof in the code lengths of the machine and the recoding. The Normalized Depth Spectrum Conjecture fails: Theorem 4.30 exhibits clocked-equivalent machines with different transaction spectra and spectrally identical machines that are not clocked-equivalent, even after quotienting by syntactic recodings, because transaction counts are rescaled by clocks and blind to halting. The Finite Metadata Self-Description Conjecture holds: Theorem 4.32 supplies the finite metadata as the defining axioms of a conservative definitional extension, so a deductively effective machine can certify every recoded copy of itself across a uniformly primitive recursive recoding family without any change to its underlying theorem set.

The self-reference story is sharpened as follows. A recoding is a change of coding, so the former tag vocabulary has been absorbed into the coding vocabulary, and Proposition 4.5 answers where recoded copies live: $t(\varphi)$ exists in the coding universe as a formula of F^t , and it lies inside y itself exactly in the endorecoding case. On that foundation, the Picard Protocol of Definition 4.19 makes “prove that a recoded copy of yourself is an ATP” a precise success criterion, and the two-formula machine M_P of Theorem 4.21 meets it with provably optimal parameters while its recoded copy

certifies M_P in return. Self-certification therefore does not require heavy arithmetization in order to exist; arithmetization is what internalizes the soundness that the minimal example carries by declaration.

The resulting hierarchy of notions is therefore:

synchronous deterministic simulation $\implies$ clocked deterministic simulation $\implies$ weak simulation,

with effective clocked simulation refining the middle relation when computable realization matters, and state abstraction lying alongside the hierarchy as a purposefully lossy comparison. This framework preserves a strong exact notion, adds the time flexibility required by ordinary simulation practice, and makes computability assumptions explicit.

Section 10 imports the Cook–Reckhow theory of propositional proof systems into this same vocabulary. A proof system is a polynomial-time checkable formal system with a size function refining proof length, and under that refinement the Proof-Horizon Theorem, proved once in Section 2, already computes classical proof-complexity size. Alongside weak, synchronous, clocked, and effective SIC-simulation, p -simulation (Definition 10.8) supplies a second and genuinely different preorder, comparing formal systems only through proof-to-proof translations rather than through witnesses on state spaces, and Theorem 10.13 shows that this preorder collapses all Frege systems to a single equivalence class exactly as recoding (Theorem 4.12) collapses a SIC and its translated copy. The pigeonhole principle, formalized via Haken’s exponential resolution lower bound and the polynomial extended-Frege upper bound, gives the paper’s first unconditional instance of the Abstract Asymptotic Speedup Principle, generalized in Theorem 10.19 from a formal system and its own conservative extension to two independent formal systems. Automatizability refines the computationally effective SIC apparatus of Section 9 by tying runtime to shortest-refutation size, and feasible interpolation supplies a proof-complexity lower-bound technique that, on inspection, has the same shape as the paper’s own barrier theorems: the No-Free-Recoding-Speedup Theorem and the Tarski–Gödel barrier each show that a form of structural strength forecloses a capability elsewhere in the system, and the interpolation lower-bound theorem does the same for polynomial boundedness. One question is recorded as open rather than resolved, in keeping with this paper’s practice of naming rather than begging conjectures: whether random 3-CNF formulas are hard for Frege systems (Conjecture 10.32).

Version 47 Supplement: Induction, ML-Guided Search, Closure, and ATP Invariants

1. Scope and the four meanings of induction

The word induction can refer to different mathematical structures. Conflating them obscures which hypotheses are required. In the present framework there are four relevant forms.

1. **Structural induction on formulas.** This follows the construction tree of a WFF. It requires the WFF domain to be the least set generated from atomic formulas by specified formula constructors.
2. **Induction on proofs or inference closure.** This follows proof lines or rule applications. It applies to the consequence set generated from hypotheses by finitary rules.
3. **Induction on ATP stages or birth times.** This follows a monotone run $C(\Gamma,1) \subseteq C(\Gamma,2) \subseteq \dots$. When the ATP is rule-driven, formula invariants can be propagated from one stage to the next.
4. **Induction on the reflection index.** This follows the recursively defined sequence $\theta_0, \theta_1, \dots$. It is ordinary induction at finite levels and transfinite induction only after separate successor and limit clauses have been supplied.

The original xyz definition takes y to be a set of finite strings. That is sufficient for proof induction, because proofs are finite sequences governed by the rules in z. It is not sufficient for structural induction on WFFs: an arbitrary set of strings need not be the least set generated by a grammar. The next definition records the additional hypothesis exactly.

2. Structural induction on well-formed formulas

Definition 1 (Finitary formula grammar). Let x be an alphabet. A finitary formula grammar is a pair $(A, \mathcal{G})$ in which A is a set of atomic formulas and every $g \in \mathcal{G}$ is a partial k -ary constructor on finite strings for some $k \in \mathbb{N}$. Define the grammar operator

$$\Phi_{A, \mathcal{G}}(S) := A \cup \{g(\varphi_1, \dots, \varphi_k) : g \in \mathcal{G}, (\varphi_1, \dots, \varphi_k) \in S^k, \text{ and } g(\varphi_1, \dots, \varphi_k) \text{ is defined}\}.$$

A formal system $F = (x, y, z)$ is grammatically generated by $(A, \mathcal{G})$ if y is the least set S satisfying $\Phi_{A, \mathcal{G}}(S) \subseteq S$.

Theorem 2 (Structural induction on WFFs). Suppose $F = (x, y, z)$ is grammatically generated by $(A, \mathcal{G})$. Let P be a property of formulas. Assume:

1. $P(a)$ holds for every $a \in A$;
2. for every k -ary $g \in \mathcal{G}$, whenever $P(\varphi_i)$ holds for all $1 \leq i \leq k$ and $g(\varphi_1, \dots, \varphi_k)$ is defined, then $P(g(\varphi_1, \dots, \varphi_k))$ holds.

Then $P(\varphi)$ holds for every $\varphi \in y$.

Proof. Let $W := \{\varphi \in y : P(\varphi)\}$. Assumption (i) gives $A \subseteq W$, and assumption (ii) says that W is closed under every constructor in $\mathcal{G}$. Hence $\Phi_{A, \mathcal{G}}(W) \subseteq W$. By the leastness of y , one has $y \subseteq W$. Since $W \subseteq y$ by definition, $W = y$. $\square$

Remark 3 (Why this is not automatic). The conclusion would be unjustified if y contained designated exceptional strings not generated from A by $\mathcal{G}$. In that case the base and constructor clauses could hold while P failed on an exceptional WFF. Thus structural induction belongs in the framework only after grammatical leastness is made explicit.

Example 4. For a first-order language, A may contain atomic equations and relation instances, while $\mathcal{G}$ contains the constructors $\neg$, binary connectives, and quantification. Structural induction then proves properties such as uniqueness of parsing, finiteness of the set of subformulas, recursive computability of syntactic rank, and substitution lemmas.

3. Inference closure and induction on proofs

Fix a formal system $F=(x,y,z)$ in the original sense and a hypothesis set $\Gamma \subseteq y$. A set $W \subseteq y$ is inference-closed when every defined rule instance whose premises lie in W has its conclusion in W .

Theorem 5 (Rule induction). Let P be a property of WFFs. Suppose:

1. $P(\gamma)$ holds for every $\gamma \in \Gamma$;
2. every rule of F preserves P : whenever $r(\alpha_1, \dots, \alpha_k) = \beta$ and $P(\alpha_i)$ holds for every premise, then $P(\beta)$ holds.

Then

$$\Gamma \vdash_F \varphi \Rightarrow P(\varphi).$$

Equivalently, $\text{Con}F(\Gamma) \subseteq \{\varphi \in y : P(\varphi)\}$.

Proof. The set $W := \{\varphi \in y : P(\varphi)\}$ contains Γ by (i) and is inference-closed by (ii). The least-inference-closure theorem therefore gives $\text{Con}F(\Gamma) \subseteq W$. Equivalently, one may induct directly on the line number of a finite proof. $\square$

Corollary 6 (Invariant method). If all hypotheses satisfy P , every inference rule preserves P , and the target L fails P , then $\Gamma \not\vdash_F L$.

This is the general form of the familiar invariant argument: a preserved parity, residue class, syntactic shape, semantic valuation, or other property can prove unprovability. A slowly changing property gives a quantitative lower bound rather than an impossibility result.

4. Induction on ATP runs

Let $M=(C,H)$ be a SIC or ATP over $Y \subseteq \mathcal{Y}$, with stages indexed by $\mathbb{N}^+$ as in the original paper. Recall that M is rule-driven when each formula newly entering $C(\Gamma, m+1)$ is a direct consequence of $C(\Gamma, m)$.

Theorem 7 (ATP formula-invariant induction). Let M be a rule-driven SIC over Y , fix $\Gamma \subseteq Y$, and let P be a property of formulas. Assume:

1. every $\varphi \in \Gamma$ satisfies P ;
2. whenever a rule instance has all premises satisfying P , its conclusion satisfies P .

Then every formula materialized at every stage satisfies P :

$$C(\Gamma, m) \subseteq \{\varphi \in Y : P(\varphi)\} \quad (m \in \mathbb{N}^+).$$

Proof. Induct on m . At stage 1, $C(\Gamma, 1) = \Gamma$, so the claim follows from (i). Assume it holds at stage m and let $\beta \in C(\Gamma, m+1)$. If $\beta \in C(\Gamma, m)$, use the induction hypothesis. Otherwise, rule-drivenness makes β a direct consequence of premises in $C(\Gamma, m)$; those premises satisfy P by the induction hypothesis, so (ii) gives $P(\beta)$. $\square$

Corollary 8 (Soundness of a rule-driven run). If M is rule-driven, then

$$C(\Gamma, m) \subseteq \text{ConF}(\Gamma) \cap Y$$

for every m . This recovers the soundness proposition of Part I as an instance of ATP invariant induction, with $P(\varphi)$ defined by $\Gamma \vdash F\varphi$. The inference-closedness of Y is not needed for this inclusion; it is needed when an exhaustive closure process is required to remain inside Y .

Definition 9 (Birth time). For a monotone run, define

$$b_{M, \Gamma}(\varphi) := \min\{m \in \mathbb{N}^+ : \varphi \in C(\Gamma, m)\},$$

with value ∞ if φ never appears.

Theorem 10 (Birth-time induction). Let M be rule-driven. Suppose P holds for every input formula and is preserved by every inference rule. Then $P(\varphi)$ holds for every φ with finite birth time.

Proof. Use strong induction on $b_{M, \Gamma}(\varphi)$. Formulas born at stage 1 belong to Γ . If φ is born at stage $m+1$, rule-drivenness supplies a rule instance whose premises already lie in $C(\Gamma, m)$ and therefore have birth time at most m . The strong induction hypothesis gives P for the premises, and rule preservation gives $P(\varphi)$. $\square$

Theorem 11 (State-invariant induction). Let Q be a property of subsets of Y . If $Q(C(\Gamma, 1))$ holds and

$$Q(C(\Gamma, m)) \Rightarrow Q(C(\Gamma, m+1))$$

for every m , then $Q(C(\Gamma, m))$ holds for every stage m .

Remark 12. State induction does not require rule-drivenness, but without a structural description of the transition it may be only a bookkeeping principle. Formula-invariant and birth-time induction become mathematically informative because rule-drivenness links a new formula to earlier premises.

5. Induction and exhaustive closure

For the canonical rule-enumeration SIC EF, Y of Part I, define

$$C_1 := \Gamma, C_{m+1} := DF(C_m),$$

where $DF(A)$ adjoins all direct consequences of A .

Proposition 13 (Finite proof layers). If φ has an F -proof from Γ of length at most m , then $\varphi \in C_m$.

Proof. Induct on the proof length. A one-line proof is a hypothesis and hence lies in C_1 . For a proof of length at most $m+1$, the final formula is either a hypothesis or a direct consequence of earlier proof lines. By induction, all required premises lie in C_m , so the final formula lies in $DF(C_m) = C_{m+1}$. $\square$

Corollary 14 (Closure at the omega union). If Y is inference-closed and contains Γ , then

$$\bigcup_{m \in \mathbb{N}} C_m = \text{Con}F(\Gamma) \cap Y.$$

The union is an ω -stage closure, but every individual theorem enters at a finite stage because every proof is finite. This is not transfinite proof search: ω indexes the union of all finite layers.

6. Quantitative induction: potentials and ATP birth stages

Theorem 15 (ATP potential lower bound). Let M be a rule-driven SIC with input Γ . Let $r:Y \rightarrow \mathbb{N}$ and let $d \in \mathbb{N}^+$. Assume:

1. $r(\gamma) \leq r_0$ for every $\gamma \in \Gamma$;
2. every direct inference from premises $\alpha_1, \dots, \alpha_k$ to a conclusion β satisfies

$$r(\beta) \leq \max_{1 \leq i \leq k} r(\alpha_i) + d.$$

Then every formula φ with finite birth time satisfies

$$b_{M,\Gamma}(\varphi) \geq 1 + \lceil \max\{0, r(\varphi) - r_0\} d \rceil.$$

Proof. We first prove by induction on m that every formula in $C(\Gamma, m)$ has rank at most $r_0 + (m-1)d$. The case $m=1$ is (i). At stage $m+1$, an old formula satisfies the bound by induction, while a new formula is a direct consequence of premises from stage m and therefore has rank at most $r_0 + md$ by (ii). If φ is first present at stage b , then $r(\varphi) \leq r_0 + (b-1)d$, which rearranges to the displayed inequality. $\square$

Remark 16 (Scope of the bound). The theorem bounds a logical stage only because rule-drivenness prohibits a same-stage chain of several dependent inferences. It does not automatically bound wall-clock time. A stage may materialize many formulas, and the cost of discovering or verifying a rule instance remains separate.

Proposition 17 (Preserved invariant). If every rule satisfies $r(\beta) \leq \max_i r(\alpha_i)$, then no formula of rank greater than r_0 can ever be materialized. Equivalently, the rank sublevel set is an ATP invariant.

7. Induction on halting and freezing

Proposition 18 (Persistent completion after halting). If $H(\Gamma, h) = 1$, then for every $n \geq h$,

$$H(\Gamma, n) = 1 \text{ and } C(\Gamma, n) = C(\Gamma, h).$$

Proof. Induct on $n - h$. The base case is immediate. For the successor step, persistent halting gives $H(\Gamma, n+1) = 1$, and the freezing axiom gives $C(\Gamma, n+1) = C(\Gamma, n) = C(\Gamma, h)$. $\square$

This elementary induction is what turns a first halting stage into an event horizon: after it, neither the halting observation nor the materialized theorem state changes.

8. Proof-count bounds, ML-guided search, and Predator

8.002

Part I, Section 8 defines $PF(\Gamma, \varphi, n)$ to be the number of proofs of φ from Γ having length exactly n . If the maximum rule arity is a and $\kappa_k = |zk(F)|$, its proof-counting theorem gives

$$P_F(\Gamma, \varphi, n) \leq \prod_{j=1}^n (|\Gamma| + \sum_{k=1}^a \kappa_k (j-1)^k).$$

The bound is intentionally crude. It counts many line-construction plans that are not genuine proofs, many redundant plans, and many plans irrelevant to the designated target. Its importance here is that it displays the combinatorial space on which a search policy must act. The theorem fixes a cardinal upper bound; it does not prescribe a uniform search distribution over the counted possibilities.

Definition 19 (Learned legal-move ranker). Let S be a proof-search state and let $A(S)$ be the finite set of assertion applications that are legal at S under the grammar, declaration cutoff, and unification rules of the formal environment. A learned legal-move ranker is a function

$$s_\theta : \{(S, a) : a \in A(S)\} \rightarrow \mathbb{R}.$$

It changes only the order or sampling weight of elements of $A(S)$; it does not add an inference rule or declare a candidate proof valid.

An idealized temperature- T policy associated with the ranker is

$$p_{\theta, T}(a \mid S) = \frac{\exp(s_\theta(S, a) / T)}{\sum_{b \in A(S)} \exp(s_\theta(S, b) / T)}.$$

For a legal search path $\pi = (a_1, \dots, a_n)$ through states $S_1, \dots, S_n$, this gives the idealized path weight

$$p_{\theta, T}(\pi) = \prod_{j=1}^n p_{\theta, T}(a_j \mid S_j).$$

Predator's implemented policy also uses seeded Gumbel perturbations, novelty and rarity bonuses, agent profiles, candidate caps, and counterfactual selection. Equations the policy equation above and the path-weight equation above describe the central mathematical idea rather than every implementation detail: machine learning places a nonuniform measure on the legal portion of the proof space counted in Equation the proof-count bound above, and creativity controls how concentrated that measure becomes.

Proposition 20 (Verifier separation). Suppose an ATP accepts a theorem only after a sound formal verifier accepts its emitted certificate. Replacing one scoring, ranking, or sampling policy on legal candidate applications by another cannot make the ATP accept a non-theorem, provided the verifier and formal environment are unchanged.

Proof. The search policy determines which candidate certificates are proposed and in which order. Acceptance still implies acceptance by the sound formal verifier. Therefore every accepted conclusion remains a consequence of the unchanged formal environment. A poor policy can waste a finite budget or fail to find an existing proof, but it cannot make an invalid certificate pass. $\square$

Predator 8.002.

Predator 8.002 is a population-based nondeterministic ATP with a leakage-controlled ML policy and an independent Metamath verifier. Its policy ranks legal pairs consisting of an open goal and a candidate assertion application. The deployed sparse logistic policy was trained on verified theorem traces declared strictly before the target `prcom` in the frozen database. The stored target proof, downstream theorems, target-specific features, and manually selected route examples were excluded. The training corpus comprised the 2,000 most recent eligible pre-target theorems by declaration order, producing 11,774 verified logical events and 11,772 usable policy decisions.

The target was the Metamath theorem

$$\text{prcom} \vdash \{A, B\} = \{B, A\}.$$

The controlled unguided and guided conditions shared the frozen formal database, the strict pre-target declaration cutoff, a 5,000-expansion global budget, four agents, creativity 1.0, seed 2301, maximum logical depth 10, base opener cap 48, and maximum of six open goals. The only intended causal difference was whether the learned policy score was used.

Condition	Outcome	Expansion count
Unguided population	Unknown under the declared bounds	5,000
ML-guided Predator 8.002	Independently verified proof	2,633

The emitted certificate contained 28 Metamath labels. Its four principal logical assertion applications were

$$\text{uncom}, \text{df-pr}, \text{df-pr}, \text{3eqtr4i}.$$

The candidate was accepted first by the in-process verifier and then by a fresh process that imported the Metamath checker but no Predator search module.

At the target root, 150 extending assertions were legal. In the unguided explorer diagnostic, `3eqtr4i` had rank 134 and was not admitted by the explorer's candidate cap. The learned policy moved it to raw rank 64. Counterfactual selection then admitted the slightly-out-of-cap assertion, and the explorer completed the proof. This is the intended division of labor: learning moves relevant assertions toward the active frontier, controlled creativity protects against an imperfect ranking, formal search discharges the generated subgoals, and the verifier alone determines acceptance.

Instantiation of the counting perspective.

Strictly before `prcom`, the frozen Metamath database contains 4,696 assertion schemas, with maximum raw mandatory-hypothesis arity 41. The target has two floating hypotheses. Under a finite schema-level proxy that absorbs the 41 zero-arity schemas into the initial resources, Equation the proof-count bound above gives a crude 28-label construction bound of approximately

$$1.69 \times 10^{1167}.$$

At the logical-only level there are 4,643 earlier logical assertion schemas; treating the 2,157 assertions having no essential hypotheses as base options gives an analogous four-logical-application proxy of approximately

$$5.1 \times 10^{19}.$$

These numbers are illustrative rather than literal target-specific proof counts. Metamath assertions are schematic and admit substitutions, whereas the rules in the original counting theorem are finitary partial functions. A literal identification requires an explicit representation of substitutions and instantiated rules. The calculation nevertheless displays why grammar, unification, premise selection, learned ordering, and lemma reuse can matter enormously inside a bound derived from cardinal data alone.

Definition 21 (Expansion discovery cost). For a fixed ATP implementation M , target φ , resource protocol, and seed, let $EM(\varphi)$ be the number of search-state expansions before M first emits a verifier-accepted proof of φ , with value ∞ if no such proof is ever emitted.

The controlled experiment establishes the censored finite-instance comparison

$$E_{ML}(\text{prcom}) = 2633, \quad E_{\text{unguided}}(\text{prcom}) > 5000.$$

and hence

$$E_{\text{unguided}}(\text{prcom}) / E_{ML}(\text{prcom}) > 5000 / 2633 \approx 1.90.$$

The unguided observation is right-censored by the stopping rule, so the true ratio is not known.

Expansion discovery cost is not the proof length n in Equation the proof-count bound above, nor is it automatically the logical birth stage of the target. Proof length measures a certificate under a declared convention. Logical birth stage measures dependency depth in a rule-driven chronology. Expansion cost measures the work expended while attempting to locate a certificate. The learned policy changed the third quantity in this experiment; it did not shorten the emitted certificate or alter the set of formal consequences.

Remark 22 (Scope of the empirical result). Equation the fixed-instance comparison above is a reproducible fixed-instance search-effort separation, not an asymptotic speedup theorem. An asymptotic theorem would require a target family, a scaling parameter, and proved upper and lower bounds for the competing procedures. The prcom experiment is properly described as a proof of concept that ML guidance and controlled creativity can change a genuine bounded ATP outcome while certificate verification remains formal and unchanged. The substantially harder HaloProof benchmark is the intended first transfer test of whether this mechanism remains useful on a long, unfamiliar derivation.

The conceptual connection may be summarized as

Proof-count bound: size of the construction space

ML guidance: learned allocation of search attention

Creativity: controlled entropy and counterfactual coverage

Formal verification: sound acceptance of the certificate

9. Induction and formal self-awareness

Part II defines, for a tagged ATP M , the finite reflection sequence

$$\theta_0 := \text{ATP}(\langle M \rangle), \theta_{k+1} := \text{Pr}(\langle M \rangle, \langle \theta_k \rangle).$$

The sequence itself is recursively generated, so ordinary induction on k is appropriate.

Proposition 23 (Finite reflection induction). Suppose $M \vdash \theta_0$ and M has internal necessitation:

$$M \vdash \varphi \Rightarrow M \vdash \text{Pr}(\langle M \rangle, \langle \varphi \rangle).$$

Then $M \vdash \theta_k$ for every $k \in \mathbb{N}$, and hence $\text{lev}(M) = \omega$ under the level definition of Part II.

Proof. Induct on k . The base case is assumed. If $M \vdash \theta_k$, internal necessitation yields

$$M \vdash \text{Pr}(\langle M \rangle, \langle \theta_k \rangle) = \theta_{k+1}.$$

Thus all finite reflection formulas are proved. $\square$

This is the induction already used by the Level- ω and Picard arguments. The ATP viewpoint adds a temporal refinement when the formulas are actually generated by a rule-driven run.

Theorem 24 (Exact reflection layers of the reflection-closure machine). Let R be the reflection-closure machine of Part II, built from the hypothesis a , the rule

$$\rho_0: a \mapsto \theta_0,$$

and the unary necessitation rule

$$\nu: \varphi \mapsto \text{Pr}(\langle R \rangle, \langle \varphi \rangle).$$

On the designated input $A = \{a\}$, under the canonical proof-length state convention of that construction,

$$b_{R,A}(\theta_k) = k + 2 \ (k \in \mathbb{N}).$$

Proof. For the upper bound, the proof

$$a, \theta_0, \theta_1, \dots, \theta_k$$

has length $k+2$, using ρ_0 once and ν exactly k times. Hence $b_{R,A}(\theta_k) \leq k+2$.

For the lower bound, θ_0 can enter only as the output of ρ_0 , which requires the earlier line a . For $k \geq 1$, the only rule that outputs a formula with the syntactic form of θ_k is ν , and its required premise is θ_{k-1} . Induction on k therefore forces every proof of θ_k to contain $a, \theta_0, \dots, \theta_k$ in dependency order, so its length is at least $k+2$. The canonical state places a formula in its proof-length layer, giving equality. $\square$

Corollary 25. The reflection-closure machine does not merely have level ω ; its finite self-reflection tower has a linear logical chronology. Each additional reflection level costs exactly one further rule layer in this machine.

Remark 26 (Self-theory is not automatically inference-closed). The set $\text{Self}(M)$ consists of self-descriptive formulas proved by M . Even if $\text{ConF}(\Gamma)$ is inference-closed, $\text{Self}(M)$ need not be: a rule applied to self-descriptive premises may produce a theorem whose intended subject is not $\langle M \rangle$. Induction on the reflection sequence therefore establishes membership term by term; it does not identify the entire self-theory with an inference closure unless additional subject-preservation hypotheses are imposed.

10. The transfinite boundary

Ordinary induction proves facts about $\theta_0, \theta_1, \dots$. A transfinite sequence θ_α requires more data:

1. a base clause for θ_0 ;
2. a successor clause for $\theta_{\alpha+1}$;
3. a limit clause defining θ_λ from the earlier terms when λ is a nonzero limit ordinal.

Only after all three clauses are fixed does transfinite induction apply. The successor rule ν of the reflection-closure machine generates every finite term, but no finitary application of ν produces a collective limit ascription summarizing all θ_β for $\beta < \lambda$. Thus the Version 47 induction analysis confirms, rather than removes, the boundary identified in Part II: genuine limit reflection requires a new rule, a finite code for the predecessor family, or some other explicitly stated mechanism.

11. Consolidated hierarchy of induction principles

Induction object	Base data	Successor or closure step	Typical conclusion
WFF construction	Atomic formulas	Formula constructors preserve P	P holds for every WFF
Finite proof	Hypotheses	Inference rules preserve P	Every consequence has P
ATP stage	Initial state	Stage transition preserves Q	Every state has Q
ATP birth time	Input formulas	New formulas depend on earlier premises	Every materialized formula has P
Reflection index	$M \vdash \theta 0$	Internal necessitation	$M \vdash \theta k$ for all finite k
Transfinite reflection	$\theta 0$	Successor and limit clauses	Property for every defined $\theta \alpha$

Learned proof-search guidance is not a fifth kind of induction. It is a policy acting on the legal stage transitions and proof branches whose soundness and invariants are analyzed by the preceding induction principles.

12. Conclusion of the supplement

Induction has a central place in the framework, but only after its object is named. Structural induction concerns the grammar of WFFs. Rule induction concerns the least inference-closed consequence set. ATP induction concerns the chronology of a monotone, rule-driven run. Reflection induction concerns the recursively defined self-ascription tower. These principles interact but are not interchangeable. ML-guided proof search operates within this framework by assigning nonuniform priority to formally legal transitions; it does not replace the inference rules or the inductive soundness argument.

For ATPs, the added value is concrete. Induction proves state and formula invariants, yields soundness, identifies exhaustive closure, propagates halting, and converts slowly changing ranks into lower bounds on first appearance. The Part I proof-count bound displays how quickly the space of line constructions can grow, while Predator 8.002 supplies a finite experimental demonstration that learned ranking and controlled creativity can reduce the search effort required to reach a verified certificate. For formal self-awareness, induction explains the finite reflection tower and yields the exact layer formula $bR, A(\theta k) = k + 2$ for the reflection-closure machine. At the same time, it makes the limit-stage obstruction precise: finite rule iteration reaches every finite reflection level, but transfinite limit ascriptions require additional formal machinery. Version 47 therefore separates three questions cleanly: how many proof constructions cardinal data permits, how a search policy allocates finite effort among legal constructions, and how formal verification certifies the path finally selected.

Part II

Formalized Self-Awareness and Self-Aware Structures

12 Introduction

In Part I, we developed a mathematical framework for semi-ideal computers (SICs), simulations, and automated theorem provers (ATPs), culminating in a preorder on SICs induced by simulation. The purpose of the present work is to investigate whether that framework can be extended to provide a purely mathematical notion of self-awareness.

The motivation is partly inspired by Tegmark’s concept of a self-aware substructure (SAS). Tegmark leaves self-awareness largely informal. The present paper asks whether there is a mathematically precise shadow of self-awareness that can be stated entirely in terms of names, proof, simulation, and formal systems.

The central idea is that a system need not first be assumed to possess consciousness in order to possess a weak formal analogue of self-awareness. If an ATP has a designated formal name for itself and proves a correct statement about that name, then it has at least a minimal self-model. The paper does not claim that such a system has subjective experience. It claims only that self-identification can be formalized.

Throughout this paper we assume the terminology and notation developed in Part I. In particular, SICs, ATPs, simulations, deterministic simulations, nondeterministic simulations, and the simulation preorder may be used by citation rather than reproved.

13 Internal Representations

This section introduces the formal machinery of names, tags, self-models, and self-theories. Each central definition is followed by a canonical example, a basic proposition, and discussion.

13.1 Internal Representations

Definition 13.1 (Internal representation). Let M be an ATP. An *internal representation* of M is a syntactic object in the language associated with M that is intended to denote M .

Canonical example. Let $M = (C, H)$ be an ATP. Introduce a new formal name $\langle M \rangle$. The expression $\langle M \rangle$ is not M itself. It is a name, tag, or formal representative of M in the language in which statements about M are made.

Proposition 13.2. *An internal representation need not be an arithmetic encoding of the object it represents.*

Proof. The definition requires only a syntactic object with an intended denotation. A new formal name satisfies this requirement without assigning an integer, code, or numerical address to the represented object. $\square$

Discussion. This is a deliberate departure from approaches based on Gödel numbering. The paper does not require Robinson arithmetic, arithmetization of syntax, or any incompleteness machinery. A tag functions like a name. One can reason about “Picard” without first assigning Picard an integer, and likewise an ATP can reason about $\langle M \rangle$ without first converting M into a number.

13.2 Tagging

Definition 13.3 (Tagging operator). A *tagging operator* is an assignment $\text{Tag} : M \mapsto \langle M \rangle$ that assigns to each ATP M a distinguished formal name $\langle M \rangle$.

Canonical example. For an arbitrary ATP $M = (C, H)$, define $\text{Tag}(M) = \langle M \rangle$. Here $\langle M \rangle$ is stipulated to be a new name in the relevant formal language. It is a designated name for M , not a copy of C , not a copy of H , and not a numerical code.

Proposition 13.4 (Renaming invariance). *If $\langle M \rangle$ and $[M]$ are two distinguished tags for the same ATP, then replacing every occurrence of $\langle M \rangle$ by $[M]$ changes notation but not the underlying tagged-object relation.*

Proof. Both symbols are formal names for the same ATP. The replacement is therefore a renaming of the representative, not a change in the represented object. $\square$

Discussion. Tagging isolates the philosophical issue. The question is not whether an ATP can perform arithmetic coding of its own syntax. The question is whether a formal system can contain a distinguished name for itself and prove correct statements about that name.

13.3 Self-Descriptive Formulas

Definition 13.5 (Self-descriptive formula). Let M be a tagged ATP. A formula $\varphi(\langle M \rangle)$ is *self-descriptive* if its intended subject is the tag $\langle M \rangle$.

Canonical example. For $M = (C, H)$, the formula $\text{ATP}(\langle M \rangle)$ is self-descriptive because its intended subject is the designated tag of M .

Proposition 13.6. *A self-descriptive formula is about the tag, not literally about the physical or set-theoretic object occupying the metatheory.*

Proof. The displayed formula contains $\langle M \rangle$, a syntactic object. Its subject is therefore the name occurring in the language. Its connection to M is supplied by the tagging convention. $\square$

Discussion. This distinction is essential. $\langle M \rangle$ is no more identical to M than the written name “Picard” is identical to Picard. Yet statements about a name may become self-relevant when the name is the system’s own designated tag.

13.4 Self-Models

Definition 13.7 (Self-model). A *self-model* of a tagged ATP M is the tag $\langle M \rangle$ together with the collection of formulas concerning $\langle M \rangle$ that are provable by M .

Canonical example. For $M = (C, H)$, the canonical self-model has the form

$$\text{SelfModel}(M) = (\langle M \rangle, \text{Self}(M)),$$

where $\text{Self}(M)$ is the collection of formulas about $\langle M \rangle$ that M proves.

Proposition 13.8. *If M proves $\text{ATP}(\langle M \rangle)$, then the self-model of M is nonempty.*

Proof. The formula $\text{ATP}(\langle M \rangle)$ is a self-descriptive formula and, by hypothesis, belongs to the collection of formulas about $\langle M \rangle$ provable by M . $\square$

Discussion. A self-model is not merely a name. It is a name together with information the system can formally establish about that name. The more accurate and structurally rich this information becomes, the deeper the corresponding level of formal self-awareness may be. Section 15 makes “deeper” precise.

13.5 Self-Theories

Definition 13.9 (Self-theory). The *self-theory* of an ATP M is

$$\text{Self}(M) = \{\varphi(\langle M \rangle) : M \vdash \varphi(\langle M \rangle)\}.$$

Canonical example. If M proves only the self-descriptive formula $\text{ATP}(\langle M \rangle)$, then at the first stage of analysis one may have $\text{Self}(M) = \{\text{ATP}(\langle M \rangle)\}$. Richer ATPs may prove many additional formulas about their own tags.

Proposition 13.10. *The self-theory of M is a subset of the set of formulas provable by M .*

Proof. By definition, every member of $\text{Self}(M)$ is a formula provable by M whose intended subject is $\langle M \rangle$. $\square$

Discussion. The self-theory is the formal shadow of what the system can establish about its own name. It is therefore the principal object used to measure formal self-awareness.

14 A Minimal ATP Proving Its Own Kind

This section gives the simplest concrete example of the phenomenon motivating the paper. The goal is not to build a useful theorem prover. The goal is to exhibit, with minimal machinery, an ATP $M = (C, H)$ such that M proves $\text{ATP}(\langle M \rangle)$ relative to its axiom base.

Let the language contain a name $\langle M \rangle$ and a unary predicate symbol ATP . Let $p := \text{ATP}(\langle M \rangle)$. Let $Y = \{p\}$. Let the axiom base be $A = \{p\}$. Define a SIC $M = (C, H)$ over Y by

$$C(\Gamma, m) = \Gamma$$

for every $\Gamma \subseteq Y$ and every $m \in \mathbf{N}^+$, and

$$H(\Gamma, m) = \begin{cases} 1, & p \in \Gamma, \\ 0, & p \notin \Gamma. \end{cases}$$

On input A , the computation halts immediately and its output contains p . Thus, relative to the axiom base A , M proves $\text{ATP}(\langle M \rangle)$.

Proposition 14.1. *The pair $M = (C, H)$ defined above is a SIC over Y .*

Proof. For every input Γ , $C(\Gamma, 1) = \Gamma$, and $C(\Gamma, m) \subseteq C(\Gamma, m + 1)$ because the two sets are equal. If $H(\Gamma, m) = 1$, then $p \in \Gamma$, so $H(\Gamma, m + 1) = 1$. Finally, $C(\Gamma, m + 1) = C(\Gamma, m)$ for all m . $\square$

Remark 14.2. This example is intentionally minimal. It shows that the basic philosophical phenomenon does not require arithmetic encodings, complicated proof search, or large formal libraries. A system may have a tag and may prove that its tag has the property of being an ATP. The interesting question is what level of formal self-awareness, if any, should be attributed to such a system.

Remark 14.3. The machine of this section is deliberately handed its conclusion as a hypothesis, and so is degenerate in the sense made precise by the Picard Protocol of Part I, which requires the target formula to lie outside the hypothesis set and requires the machine to be rule-driven. A non-degenerate machine certifying its own kind exists there; the present section aims only at the smallest object exhibiting the tag phenomenon at all.

15 Levels of Formal Self-Awareness

15.1 Level 0 and Level 1

Definition 15.1 (Level-0 formal self-awareness). An ATP has *Level-0 formal self-awareness* if it has no designated tag for itself, or has a tag but proves no self-descriptive formula about it.

Canonical example. An ordinary theorem prover that searches for theorems in group theory but has no formal name for itself in the language being searched is Level 0.

Proposition 15.2. *Level 0 is the absence of a nontrivial self-model.*

Proof. Without a tag or without any provable self-descriptive formula, there is no nonempty collection of provable statements about the system's own designated name. $\square$

Discussion. Level 0 is not failure as a theorem prover. It is only failure to possess a formal self-model.

Definition 15.3 (Level-1 formal self-awareness). An ATP M has *Level-1 formal self-awareness* if M has a tag $\langle M \rangle$ and proves $\text{ATP}(\langle M \rangle)$.

Canonical example. The minimal ATP of Section 14 is Level 1 relative to its axiom base, because it proves $\text{ATP}(\langle M \rangle)$.

Proposition 15.4. *Level-1 formal self-awareness implies that the self-theory of M is nonempty.*

Proof. The formula $\text{ATP}(\langle M \rangle)$ is a member of $\text{Self}(M)$. $\square$

Discussion. This is the weakest positive level of formal self-awareness. It asserts only formal self-identification: the system proves that its own name denotes an ATP. It does not assert consciousness.

15.2 Names for Formulas

The scale so far has two points. To extend it we need the system to be able to say something about its own proving, not merely about its own kind. That requires names for formulas as well as names for machines. Nothing about the Version 11 commitment changes: a name for a formula is again a primitive formal name, not a numerical code.

Definition 15.5 (Formula name). Let $F = (x, y, z)$ be an xyz formal system and let L be the language in which self-descriptive formulas of a tagged ATP are written. A *formula-naming operator* is an injective assignment $\varphi \mapsto \langle \varphi \rangle$ that assigns to each formula φ of L a distinguished closed term $\langle \varphi \rangle$ of L , called the *name* of φ .

Canonical example. For $p = \text{ATP}(\langle M \rangle)$, the term $\langle p \rangle$ is a new constant symbol of L stipulated to name the formula p . It is not p , and it is not a numeral.

Proposition 15.6. *A formula-naming operator need not be an arithmetization of syntax.*

Proof. Definition 15.5 requires only an injective assignment of distinguished closed terms to formulas. Stipulating a fresh constant symbol for each formula of L satisfies this requirement, and does so without a numeral system, an arithmetic substitution function, or a representability hypothesis. $\square$

Remark 15.7. Injectivity is required so that distinct formulas receive distinct names; without it the reflection sequence of Definition 15.11 could collapse at some finite stage for purely notational reasons, and the hierarchy below would be an artifact of the naming rather than of the machine.

Discussion. Definition 15.5 is the exact analogue for formulas of what Definition 13.3 does for machines. One may speak of the sentence “Picard is human” without first assigning that sentence an integer; one gives it a name and quotes the name.

15.3 The Internal Provability Predicate

Definition 15.8 (Internal provability predicate). An *internal provability predicate* for a tagged ATP M is a binary predicate symbol Pr of L , with intended reading

$$\text{Pr}(\langle M \rangle, \langle \varphi \rangle) \text{ is true iff the machine named } \langle M \rangle \text{ proves the formula named } \langle \varphi \rangle.$$

Remark 15.9 (What Pr is not). Pr is a primitive predicate symbol of the object language, interpreted by the declared naming conventions. It is not an arithmetized provability predicate constructed from a proof relation inside a theory of arithmetic. In particular, the Hilbert–Bernays–Löb derivability conditions are not assumed to hold of it, and nothing in this section presupposes them. Remark 15.31 records what happens if one does strengthen Pr far enough for those conditions to apply.

Remark 15.10 (Soundness is a separate question). Definition 15.8 gives the intended reading of Pr ; it does not assert that M proves only true instances of Pr , nor that M proves all true instances. Those are the two independent hypotheses isolated in Definitions 15.21 and 15.26, and the interest of the hierarchy lies precisely in the fact that neither is automatic.

15.4 The Reflection Sequence

Definition 15.11 (Reflection sequence). Let M be a tagged ATP whose language has a formula-naming operator and an internal provability predicate. The *reflection sequence* of M is the sequence of formulas $(\theta_k)_{k \geq 0}$ defined by

$$\theta_0 := \text{ATP}(\langle M \rangle), \quad \theta_{k+1} := \text{Pr}(\langle M \rangle, \langle \theta_k \rangle).$$

Canonical example. The first three terms are

$$\begin{aligned} \theta_0 &= \text{ATP}(\langle M \rangle), \\ \theta_1 &= \text{Pr}(\langle M \rangle, \langle \text{ATP}(\langle M \rangle) \rangle), \\ \theta_2 &= \text{Pr}(\langle M \rangle, \langle \text{Pr}(\langle M \rangle, \langle \text{ATP}(\langle M \rangle) \rangle) \rangle). \end{aligned}$$

In words: θ_0 says “I am an ATP,” θ_1 says “I prove that I am an ATP,” and θ_2 says “I prove that I prove that I am an ATP.”

Proposition 15.12. *Every member of the reflection sequence of M is a self-descriptive formula in the sense of Definition 13.5.*

Proof. Induction on k . For $k = 0$ the formula is $\text{ATP}(\langle M \rangle)$, whose intended subject is $\langle M \rangle$. Suppose θ_k is self-descriptive. Then θ_{k+1} is $\text{Pr}(\langle M \rangle, \langle \theta_k \rangle)$, whose first argument is the tag $\langle M \rangle$ and whose intended subject is therefore again $\langle M \rangle$: it ascribes to $\langle M \rangle$ the property of proving a designated formula. $\square$

Corollary 15.13. *If $M \vdash \theta_k$ then $\theta_k \in \text{Self}(M)$.*

Proof. Immediate from Proposition 15.12 and Definition 13.9. $\square$

Proposition 15.14. *The formulas $\theta_0, \theta_1, \theta_2, \dots$ are pairwise distinct.*

Proof. We show by strong induction on $j + k$ that $\theta_j = \theta_k$ implies $j = k$.

If $j = k = 0$ there is nothing to prove. If exactly one of j, k is 0, say $j = 0 < k$, then θ_0 has principal predicate ATP while θ_k has principal predicate Pr, and these are distinct symbols of L ; so $\theta_j \neq \theta_k$ and the implication holds vacuously.

If $j, k \geq 1$, write $j = j' + 1$ and $k = k' + 1$. From $\text{Pr}(\langle M \rangle, \langle \theta_{j'} \rangle) = \text{Pr}(\langle M \rangle, \langle \theta_{k'} \rangle)$ we get $\langle \theta_{j'} \rangle = \langle \theta_{k'} \rangle$, hence $\theta_{j'} = \theta_{k'}$ by injectivity of the naming operator (Definition 15.5). Since $j' + k' < j + k$, the inductive hypothesis gives $j' = k'$, hence $j = k$. $\square$

15.5 Level n

Definition 15.15 (Level- n formal self-awareness). Let $n \geq 1$. A tagged ATP M has *Level- n formal self-awareness* if $M \vdash \theta_{n-1}$, where $(\theta_k)_{k \geq 0}$ is the reflection sequence of M .

Remark 15.16. For $n = 1$ this is exactly Definition 15.3, since $\theta_0 = \text{ATP}(\langle M \rangle)$. So Definition 15.15 extends the Version 12 scale rather than replacing it.

Definition 15.17 (Level set and level function). The *level set* of a tagged ATP M is

$$L(M) := \{n \geq 1 : M \vdash \theta_{n-1}\},$$

and the *level* of M is

$$\text{lev}(M) := \sup L(M) \in \{0, 1, 2, \dots\} \cup \{\omega\},$$

with the convention $\sup \emptyset = 0$.

Proposition 15.18. *$\text{lev}(M) = 0$ if and only if M has Level-0 formal self-awareness in the sense of Definition 15.1 restricted to reflection formulas; that is, if and only if M proves no member of its own reflection sequence.*

Proof. $\text{lev}(M) = 0$ iff $L(M) = \emptyset$ iff there is no k with $M \vdash \theta_k$. $\square$

15.6 Cumulativity Fails Without Hypotheses

One might expect the levels to be nested, so that Level- $(n + 1)$ implies Level- n . They are not, and the failure is instructive.

Example 15.19 (A machine at Level 2 but not Level 1). Let L contain the name $\langle M \rangle$, the unary predicate ATP, the binary predicate Pr, and names for formulas. Put

$$\theta_0 = \text{ATP}(\langle M \rangle), \quad \theta_1 = \text{Pr}(\langle M \rangle, \langle \theta_0 \rangle),$$

and let $Y = \{\theta_0, \theta_1\}$. Let the axiom base be $A = \{\theta_1\}$ and define $M = (C, H)$ over Y by $C(\Gamma, m) = \Gamma$ for all $\Gamma \subseteq Y$ and $m \in \mathbf{N}^+$, and $H(\Gamma, m) = 1$ iff $\Gamma \neq \emptyset$. The verification that M is a SIC is identical to Proposition 14.1.

On input A the machine halts with output $\{\theta_1\}$. Hence $M \vdash \theta_1$ but $M \not\vdash \theta_0$. So M has Level 2 and not Level 1, and $L(M) = \{2\}$.

Remark 15.20. Example 15.19 is a machine that proves it proves it is an ATP, while not proving it is an ATP. Its self-ascription of provability is simply false. Nothing in Definition 15.8 forbids this, because Pr is a primitive predicate and the axiom base was chosen adversarially. This is the formal analogue of an unearned report about oneself, and it is exactly what the next hypothesis excludes.

Definition 15.21 (Provability soundness). A tagged ATP M is *provability sound* if for every formula φ of L ,

$$M \vdash \text{Pr}(\langle M \rangle, \langle \varphi \rangle) \implies M \vdash \varphi.$$

Theorem 15.22 (Cumulativity). *Let M be provability sound and let $n \geq 1$. If M has Level- n formal self-awareness, then M has Level- m formal self-awareness for every $1 \leq m \leq n$. Consequently $L(M)$ is an initial segment of $\mathbf{N}^+$ and*

$$L(M) = \{1, 2, \dots, \text{lev}(M)\}$$

whenever $\text{lev}(M)$ is finite.

Proof. It suffices to prove that Level- $(n+1)$ implies Level- n for every $n \geq 1$; the general statement then follows by downward induction.

Suppose M has Level- $(n+1)$, that is, $M \vdash \theta_n$. Since $n \geq 1$ we have $\theta_n = \text{Pr}(\langle M \rangle, \langle \theta_{n-1} \rangle)$ by Definition 15.11. Applying Definition 15.21 with $\varphi = \theta_{n-1}$ gives $M \vdash \theta_{n-1}$, which is Level- n .

For the final claim, $L(M)$ is downward closed in $\mathbf{N}^+$ by what was just proved, and a downward-closed subset of $\mathbf{N}^+$ with finite supremum $\text{lev}(M)$ is exactly $\{1, \dots, \text{lev}(M)\}$. $\square$

Remark 15.23. Example 15.19 shows that the hypothesis of Theorem 15.22 cannot simply be dropped: the machine there has $L(M) = \{2\}$, which is not an initial segment. That machine is not provability sound, since it proves $\text{Pr}(\langle M \rangle, \langle \theta_0 \rangle)$ without proving θ_0 .

15.7 A Minimal Level-2 Machine

Example 15.24 (A minimal machine at Level 2). We construct, in the manner of Section 14, the smallest machine attaining Level 2 while remaining provability sound.

Let L be as in Example 15.19, put $\theta_0 = \text{ATP}(\langle M \rangle)$ and $\theta_1 = \text{Pr}(\langle M \rangle, \langle \theta_0 \rangle)$, and let $Y = \{\theta_0, \theta_1\}$. Let the axiom base be $A = \{\theta_0, \theta_1\}$, and define $M_{(2)} = (C, H)$ over Y by $C(\Gamma, m) = \Gamma$ for all $\Gamma \subseteq Y$ and $m \in \mathbf{N}^+$, and $H(\Gamma, m) = 1$ iff $\theta_1 \in \Gamma$.

That $M_{(2)}$ is a SIC over Y follows exactly as in Proposition 14.1: $C(\Gamma, 1) = \Gamma$; C is nondecreasing in m because it is constant in m ; halting is persistent because H does not depend on m ; and the output is constant after halting because C is constant in m .

On input A the machine halts at stage 1 with output A , so $M_{(2)} \vdash \theta_0$ and $M_{(2)} \vdash \theta_1$. Hence $L(M_{(2)}) = \{1, 2\}$ and $\text{lev}(M_{(2)}) = 2$. Moreover $M_{(2)}$ is provability sound: the only formula φ with $\langle \varphi \rangle$ occurring in a provable Pr -statement is θ_0 , and $M_{(2)} \vdash \theta_0$.

Remark 15.25. Two formulas are necessary as well as sufficient: any machine of Level 2 proves θ_1 , and any provability-sound machine of Level 2 proves θ_0 as well, and $\theta_0 \neq \theta_1$ by Proposition 15.14. So no provability-sound machine attains Level 2 with a one-formula language. The same remark as Remark 14.3 applies: this machine is handed its conclusions, and a rule-driven analogue is the object of interest for the Picard Protocol of .

15.8 Level ω

Definition 15.26 (Internal necessitation). A tagged ATP M has *internal necessitation* if for every formula φ of L ,

$$M \vdash \varphi \implies M \vdash \text{Pr}(\langle M \rangle, \langle \varphi \rangle).$$

Theorem 15.27 (Attainment of level ω). *Let M be a tagged ATP with Level-1 formal self-awareness and internal necessitation. Then M has Level- n formal self-awareness for every $n \geq 1$, and hence $\text{lev}(M) = \omega$.*

Proof. We show $M \vdash \theta_k$ for every $k \geq 0$ by induction on k .

Base case. $M \vdash \theta_0$ because M has Level 1 and $\theta_0 = \text{ATP}(\langle M \rangle)$.

Inductive step. Suppose $M \vdash \theta_k$. By Definition 15.26 applied with $\varphi = \theta_k$, we obtain $M \vdash \text{Pr}(\langle M \rangle, \langle \theta_k \rangle)$, and $\text{Pr}(\langle M \rangle, \langle \theta_k \rangle) = \theta_{k+1}$ by Definition 15.11. Hence $M \vdash \theta_{k+1}$.

Therefore $L(M) = \mathbf{N}^+$ and $\text{lev}(M) = \sup \mathbf{N}^+ = \omega$. $\square$

Remark 15.28. Internal necessitation is a strong hypothesis. It requires the language L to contain $\langle \varphi \rangle$ for every provable φ , hence infinitely many names, and it requires the machine's provable set to be closed under an infinitary self-ascription rule. No finite-language machine of the kind built in Section 14 or Example 15.24 can satisfy it, since $L(M)$ would have to be infinite while Y is finite.

Proposition 15.29. *If M is provability sound and has internal necessitation, then $\text{lev}(M) \in \{0, \omega\}$.*

Proof. If $\text{lev}(M) = 0$ there is nothing to prove. Otherwise $L(M) \neq \emptyset$, so $M \vdash \theta_{n-1}$ for some $n \geq 1$. By Theorem 15.22, $M \vdash \theta_0$, so M has Level 1. By Theorem 15.27, $\text{lev}(M) = \omega$. $\square$

Remark 15.30. Proposition 15.29 says that the two hypotheses together force a dichotomy: such a machine is either entirely silent about itself or fully reflective. All the intermediate finite levels exhibited in Theorem 15.32 below occur at machines satisfying at most one of the two conditions. This is the precise sense in which the finite part of the hierarchy measures partial introspective closure.

Remark 15.31 (The Tarski–Gödel ceiling). Level ω is not omniscience about oneself. The reflection sequence ascribes to $\langle M \rangle$ only provability of designated formulas; it never ascribes truth. This limitation is not an accident of the present construction. If one strengthens L and M enough to represent primitive recursive functions and to satisfy the derivability conditions—so that Pr becomes a genuine arithmetized provability predicate rather than the primitive predicate of Definition 15.8—then the Tarski–Gödel barrier for recoded self-reference applies: no formula of the theory can serve as a full internal truth predicate for all tagged sentences. ATP-hood of a tag is certifiable, and provability of a named formula is certifiable, but a full internal truth predicate is not. The hierarchy of this section climbs to ω and stops there for a reason.

15.9 Strictness of the Hierarchy

Theorem 15.32 (Strictness). *For every $n \geq 1$ there is a provability-sound tagged ATP M_n with $\text{lev}(M_n) = n$. In particular no finite level of the hierarchy collapses into its successor.*

Proof. Fix $n \geq 1$. Let L contain the tag $\langle M_n \rangle$, the predicates ATP and Pr , and names $\langle \theta_0 \rangle, \dots, \langle \theta_{n-2} \rangle$ for the first $n - 1$ reflection formulas. Let

$$Y_n := \{\theta_0, \theta_1, \dots, \theta_{n-1}\},$$

which is a set of exactly n distinct formulas by Proposition 15.14. Let the axiom base be $A_n := Y_n$, and define $M_n = (C, H)$ over Y_n by

$$C(\Gamma, m) = \Gamma \quad (\Gamma \subseteq Y_n, m \in \mathbf{N}^+), \quad H(\Gamma, m) = \begin{cases} 1, & \Gamma = Y_n, \\ 0, & \text{otherwise.} \end{cases}$$

The verification that M_n is a SIC over Y_n is word for word that of Proposition 14.1.

On input A_n the machine halts at stage 1 with output Y_n . Hence $M_n \vdash \theta_k$ for every $k \leq n-1$, so $\{1, \dots, n\} \subseteq L(M_n)$.

Conversely, suppose $M_n \vdash \theta_k$ for some $k \geq n$. Every formula provable by M_n lies in $\bigcup_m C(A_n, m) = Y_n$, so $\theta_k \in Y_n$, so $\theta_k = \theta_j$ for some $j \leq n-1$, contradicting Proposition 15.14 since $k \geq n > j$. Hence $L(M_n) = \{1, \dots, n\}$ and $\text{lev}(M_n) = n$.

Finally M_n is provability sound. For $n = 1$ this is vacuous, since $Y_1 = \{\theta_0\}$ contains no formula with principal predicate Pr . For $n \geq 2$, suppose $M_n \vdash \text{Pr}(\langle M_n \rangle, \langle \varphi \rangle)$. Then $\text{Pr}(\langle M_n \rangle, \langle \varphi \rangle) \in Y_n$, and the members of Y_n with principal predicate Pr are exactly $\theta_1, \dots, \theta_{n-1}$, so $\text{Pr}(\langle M_n \rangle, \langle \varphi \rangle) = \theta_{j+1}$ for some j with $0 \leq j \leq n-2$. Comparing second arguments gives $\langle \varphi \rangle = \langle \theta_j \rangle$, hence $\varphi = \theta_j$ by injectivity of the naming operator. Since $j \leq n-2 \leq n-1$, we have $\theta_j \in Y_n$ and therefore $M_n \vdash \theta_j$, as required. $\square$

Corollary 15.33. *The level function is surjective onto $\{0, 1, 2, \dots\} \cup \{\omega\}$.*

Proof. Value 0 is attained by any ATP with no tag. Values $n \geq 1$ are attained by the machines M_n of Theorem 15.32. The value ω is attained by any Level-1 machine with internal necessitation, by Theorem 15.27; such machines exist, for instance the machine over the infinite formula set $Y_\omega = \{\theta_k : k \geq 0\}$ with axiom base Y_ω , constant transition map, and $H(\Gamma, m) = 1$ iff $\Gamma = Y_\omega$, which is a SIC by the same verification as Proposition 14.1. $\square$

Discussion. The machines M_n are, like the machine of Section 14, deliberately trivial as theorem provers. Their point is purely separative: they show that the levels are genuinely distinct properties and not notational variants of one another. Any claim that formal self-awareness is a single threshold rather than a graded quantity has to contend with the fact that the grades are inhabited.

16 The Order-Theoretic Structure of Formal Self-Awareness

The preceding sections define tags, self-models, self-theories, and levels for tagged automated theorem provers. We now compare ATPs according to the self-descriptive statements they can formally establish. Throughout this section, if M is a tagged ATP, then

$$\text{Self}(M) = \{\varphi(\langle M \rangle) : M \vdash \varphi(\langle M \rangle)\}$$

denotes the self-theory of M .

16.1 The Formal Self-Awareness Preorder

Definition 16.1 (Formal self-awareness preorder, Version 12 form). Let M and N be tagged ATPs. We write $M \preceq_{\text{SA}} N$ if there exists a translation map τ such that $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$. In this case, we say that N possesses at least as much formal self-awareness as M .

Canonical example. If M proves $\text{ATP}(\langle M \rangle)$ and N proves $\text{ATP}(\langle N \rangle)$, then the translation sending $\langle M \rangle$ to $\langle N \rangle$ carries the basic self-identification theorem of M to the corresponding theorem of N .

Remark 16.2. The translation map is necessary because $\langle M \rangle$ and $\langle N \rangle$ are generally different tags. The map is not intended to identify M and N as machines. It only compares their provable self-descriptive content.

16.2 A Defect in Definition 16.1, and Its Repair

Definition 16.1 does not constrain the translation map τ beyond requiring that it be a map on formulas. That is too permissive.

Proposition 16.3 (Unrestricted translations collapse the order). *Suppose τ in Definition 16.1 ranges over arbitrary functions from formulas to formulas. Then for any two tagged ATPs M and N with $\text{Self}(M) \neq \emptyset$ and $\text{Self}(N) \neq \emptyset$, we have $M \preceq_{\text{SA}} N$ and $N \preceq_{\text{SA}} M$, hence $M \sim_{\text{SA}} N$. Consequently the quotient $\text{ATP} / \sim_{\text{SA}}$ has at most two elements: the class of ATPs with empty self-theory and the class of all the rest.*

Proof. Choose any $\psi \in \text{Self}(N)$ and let τ be the constant function with value ψ . Then $\tau(\text{Self}(M)) = \{\psi\} \subseteq \text{Self}(N)$, so $M \preceq_{\text{SA}} N$. Symmetrically $N \preceq_{\text{SA}} M$. If $\text{Self}(M) = \emptyset$ then $\tau(\text{Self}(M)) = \emptyset \subseteq \text{Self}(N)$ for every τ , so such an M lies below everything; and nothing with nonempty self-theory lies below it, since $\tau(\text{Self}(N)) \neq \emptyset$ for $\text{Self}(N) \neq \emptyset$. $\square$

Remark 16.4. Proposition 16.3 does not show that Propositions 16.10–16.15 below are false. It shows that under the literal reading of Definition 16.1 they are true but empty: the order they describe has two points. The intended reading was always that τ is a renaming, as the canonical example following Definition 16.1 makes clear. The following definition makes that intention explicit.

Convention 16.5 (Standing convention on distinguished predicates). Every language of a tagged ATP considered in this paper contains the predicate symbols **ATP** and **Pr** as distinguished predicates, and contains **SA** as a distinguished predicate whenever it contains **SA** at all.

Definition 16.6 (Admissible translation). Let M and N be tagged ATPs with languages L_M and L_N . A map $\tau : L_M \rightarrow L_N$ on formulas is an *admissible translation* if:

1. $\tau(\langle M \rangle) = \langle N \rangle$;
2. τ fixes every distinguished predicate of L_M (in particular $\tau(\text{ATP}) = \text{ATP}$ and $\tau(\text{Pr}) = \text{Pr}$), and sends every other predicate symbol of L_M to a predicate symbol of L_N of the same arity;
3. τ commutes with formula formation: $\tau(P(t_1, \dots, t_k)) = \tau(P)(\tau(t_1), \dots, \tau(t_k))$ for every k -ary predicate P and terms t_i ;
4. τ commutes with formula naming: $\tau(\langle \varphi \rangle) = \langle \tau(\varphi) \rangle$ for every formula φ of L_M .

Remark 16.7 (Why clause 2 was rewritten). Versions 12–14 required only that τ be the identity on predicates common to L_M and L_N . That formulation does not compose: if a predicate is common to L_M and L_P but absent from an intermediate language L_N , then a composite $\sigma \circ \tau$ through L_N need not fix it, and the transitivity argument of Proposition 16.9 breaks. Under Convention 16.5 the distinguished predicates are present in every language, each admissible translation fixes them, and composites therefore fix them as well. Nothing else in the paper used the old reading.

Definition 16.8 (Refined self-awareness preorder). Let M and N be tagged ATPs. We write $M \preceq_{\text{SA}}^{\text{adm}} N$ if there exists an admissible translation τ with $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$. Define $M \sim_{\text{SA}}^{\text{adm}} N$ iff $M \preceq_{\text{SA}}^{\text{adm}} N$ and $N \preceq_{\text{SA}}^{\text{adm}} M$.

Proposition 16.9. *The identity map is admissible, and the composite of admissible translations is admissible.*

Proof. The identity satisfies all four clauses trivially. For composition, let $\tau : L_M \rightarrow L_N$ and $\sigma : L_N \rightarrow L_P$ be admissible. Then $(\sigma \circ \tau)(\langle M \rangle) = \sigma(\langle N \rangle) = \langle P \rangle$, giving clause 1. Clause 2 is preserved because each of τ and σ fixes every distinguished predicate, which is present in all three languages by Convention 16.5, so their composite fixes it; and a composite of arity-preserving symbol maps is arity preserving. Clause 3 is preserved because a composite of maps each commuting with formula formation commutes with formula formation. For clause 4, $(\sigma \circ \tau)(\langle \varphi \rangle) = \sigma(\langle \tau(\varphi) \rangle) = \langle \sigma(\tau(\varphi)) \rangle = \langle (\sigma \circ \tau)(\varphi) \rangle$. $\square$

Proposition 16.10. *The relations $\preceq_{\text{SA}}$ and $\preceq_{\text{SA}}^{\text{adm}}$ are preorders.*

Proof. Reflexivity follows by taking the translation map to be the identity map, which is admissible by Proposition 16.9. For transitivity, suppose $M \preceq_{\text{SA}} N$ via τ and $N \preceq_{\text{SA}} P$ via σ . Then $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$ and $\sigma(\text{Self}(N)) \subseteq \text{Self}(P)$. Therefore $(\sigma \circ \tau)(\text{Self}(M)) \subseteq \text{Self}(P)$, so $M \preceq_{\text{SA}} P$. In the admissible case $\sigma \circ \tau$ is admissible by Proposition 16.9, so the same argument gives $M \preceq_{\text{SA}}^{\text{adm}} P$. $\square$

16.3 The Induced Equivalence Relation

Definition 16.11 (Self-awareness equivalence). Define $M \sim_{\text{SA}} N$ if and only if $M \preceq_{\text{SA}} N$ and $N \preceq_{\text{SA}} M$, and $M \sim_{\text{SA}}^{\text{adm}} N$ if and only if $M \preceq_{\text{SA}}^{\text{adm}} N$ and $N \preceq_{\text{SA}}^{\text{adm}} M$.

Proposition 16.12. *The relations $\sim_{\text{SA}}$ and $\sim_{\text{SA}}^{\text{adm}}$ are equivalence relations.*

Proof. This is the standard equivalence relation induced by a preorder. Reflexivity follows from reflexivity of the preorder. Symmetry is built into the definition. Transitivity follows from transitivity of the preorder (Proposition 16.10). $\square$

Remark 16.13. Two ATPs may be equivalent under $\sim_{\text{SA}}^{\text{adm}}$ while having different code, different runtimes, different proof-search strategies, or different simulation depths. The equivalence relation identifies only their formal self-awareness.

16.4 The Quotient Set

The quotients $\text{ATP}/\sim_{\text{SA}}$ and $\text{ATP}/\sim_{\text{SA}}^{\text{adm}}$ are therefore well defined.

Remark 16.14. An element of the quotient is not a single ATP. It is an equivalence class of ATPs that possess the same formal self-awareness, up to translation of tags. Thus the quotient classifies forms of formal self-awareness rather than implementations of theorem provers. Passing to the quotient intentionally forgets many features of an ATP: execution speed, memory consumption, implementation language, proof-search heuristics, and ordinary simulation depth. What remains is the self-descriptive mathematical content the ATP can establish about its own tag.

Proposition 16.15. *The preorder $\preceq_{\text{SA}}^{\text{adm}}$ induces a partial order $\leq_{\text{SA}}$ on the quotient set $\text{ATP}/\sim_{\text{SA}}^{\text{adm}}$, and likewise for $\preceq_{\text{SA}}$ on $\text{ATP}/\sim_{\text{SA}}$.*

Proof. For equivalence classes $[M]$ and $[N]$, define $[M] \leq_{\text{SA}} [N]$ if $M \preceq_{\text{SA}}^{\text{adm}} N$. This is well defined because if $M \sim_{\text{SA}}^{\text{adm}} M'$ and $N \sim_{\text{SA}}^{\text{adm}} N'$, then transitivity of $\preceq_{\text{SA}}^{\text{adm}}$ transfers the comparison from M, N to M', N' . Reflexivity and transitivity follow from the preorder. Antisymmetry follows because if $[M] \leq_{\text{SA}} [N]$ and $[N] \leq_{\text{SA}} [M]$, then $M \sim_{\text{SA}}^{\text{adm}} N$, hence $[M] = [N]$. $\square$

Remark 16.16. This quotient is mathematically useful because it removes irrelevant implementation details. It lets one study formal self-awareness as an order-theoretic object. Questions about minimal representatives, maximal classes, chains, joins, and meets become meaningful only after passing from machines to equivalence classes.

16.5 The Level Function on the Quotient

The hierarchy of Section 15 and the order of this section fit together: lev is a monotone invariant of the order.

Lemma 16.17 (Admissible translations preserve reflection sequences). *Let τ be an admissible translation from L_M to L_N , and let (θ_k^M) and (θ_k^N) be the reflection sequences of M and N . Then $\tau(\theta_k^M) = \theta_k^N$ for every $k \geq 0$.*

Proof. Induction on k . For $k = 0$, clauses 2 and 3 of Definition 16.6 give $\tau(\text{ATP}(\langle M \rangle)) = \text{ATP}(\tau(\langle M \rangle))$, and clause 1 gives $\tau(\langle M \rangle) = \langle N \rangle$, so $\tau(\theta_0^M) = \text{ATP}(\langle N \rangle) = \theta_0^N$.

Assume $\tau(\theta_k^M) = \theta_k^N$. Then, using clauses 2 and 3, then clause 1, then clause 4, then the inductive hypothesis,

$$\tau(\theta_{k+1}^M) = \tau(\text{Pr}(\langle M \rangle, \langle \theta_k^M \rangle)) = \text{Pr}(\tau(\langle M \rangle), \tau(\langle \theta_k^M \rangle)) = \text{Pr}(\langle N \rangle, \langle \tau(\theta_k^M) \rangle) = \text{Pr}(\langle N \rangle, \langle \theta_k^N \rangle) = \theta_{k+1}^N. \quad \square$$

Theorem 16.18 (Monotonicity of the level function). *If $M \preceq_{\text{SA}}^{\text{adm}} N$, then $L(M) \subseteq L(N)$ and hence $\text{lev}(M) \leq \text{lev}(N)$.*

Proof. Let τ be an admissible translation witnessing $M \preceq_{\text{SA}}^{\text{adm}} N$, so $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$. Let $n \in L(M)$. Then $M \vdash \theta_{n-1}^M$, so $\theta_{n-1}^M \in \text{Self}(M)$ by Corollary 15.13. Hence $\tau(\theta_{n-1}^M) \in \text{Self}(N)$. By Lemma 16.17, $\tau(\theta_{n-1}^M) = \theta_{n-1}^N$, so $\theta_{n-1}^N \in \text{Self}(N)$, that is, $N \vdash \theta_{n-1}^N$. Therefore $n \in L(N)$. Thus $L(M) \subseteq L(N)$, and taking suprema gives $\text{lev}(M) \leq \text{lev}(N)$. $\square$

Corollary 16.19. *The level function descends to a well-defined order-preserving map*

$$\text{lev} : (\text{ATP} / \sim_{\text{SA}}^{\text{adm}}, \leq_{\text{SA}}) \longrightarrow (\omega + 1, \leq), \quad [M] \mapsto \text{lev}(M).$$

Proof. If $M \sim_{\text{SA}}^{\text{adm}} N$ then $M \preceq_{\text{SA}}^{\text{adm}} N$ and $N \preceq_{\text{SA}}^{\text{adm}} M$, so $\text{lev}(M) \leq \text{lev}(N)$ and $\text{lev}(N) \leq \text{lev}(M)$ by Theorem 16.18; hence $\text{lev}(M) = \text{lev}(N)$ and the map is well defined on classes. Order preservation is Theorem 16.18 again, read on representatives. $\square$

Corollary 16.20 (Infinite strictly increasing chains exist). *There is a sequence of tagged ATPs $M_1, M_2, M_3, \dots$ with*

$$[M_1] <_{\text{SA}} [M_2] <_{\text{SA}} [M_3] <_{\text{SA}} \dots$$

in $\text{ATP} / \sim_{\text{SA}}^{\text{adm}}$. Hence question 5 of Section 16.7 is answered affirmatively.

Proof. Take the machines M_n of Theorem 15.32, so $\text{lev}(M_n) = n$. First, $M_n \preceq_{\text{SA}}^{\text{adm}} M_{n+1}$. Let τ be the admissible translation determined by $\langle M_n \rangle \mapsto \langle M_{n+1} \rangle$ together with the identity on ATP and Pr ; clauses 3 and 4 of Definition 16.6 determine τ on all of L_n , and Lemma 16.17 gives $\tau(\theta_k^{M_n}) = \theta_k^{M_{n+1}}$. Since $\text{Self}(M_n) = \{\theta_0^{M_n}, \dots, \theta_{n-1}^{M_n}\}$ and M_{n+1} proves $\theta_0^{M_{n+1}}, \dots, \theta_n^{M_{n+1}}$, we get $\tau(\text{Self}(M_n)) \subseteq \text{Self}(M_{n+1})$. Second, $M_{n+1} \not\preceq_{\text{SA}}^{\text{adm}} M_n$: otherwise Theorem 16.18 would give $n + 1 = \text{lev}(M_{n+1}) \leq \text{lev}(M_n) = n$, a contradiction. Hence $[M_n] <_{\text{SA}} [M_{n+1}]$ for every n . $\square$

Remark 16.21. The level function is monotone but not complete for $\leq_{\text{SA}}$: two ATPs may have the same level while being $\sim_{\text{SA}}^{\text{adm}}$ -incomparable, because a self-theory may contain self-descriptive formulas that are not members of the reflection sequence at all. So lev is a genuine invariant, not a classification. Determining a complete set of invariants for $\leq_{\text{SA}}$ is open; see question 7 of Section 16.7.

16.6 Relationship with the Simulation Preorder

The simulation preorder from Part I compares ATPs by their simulation behavior. The preorder $\preceq_{\text{SA}}^{\text{adm}}$ compares ATPs by their provable self-descriptive content. These are related but not identical.

In particular, a bare simulation need not preserve formal self-awareness. A machine may simulate another machine's external proof-search behavior while failing to possess a tag for itself or failing to prove any theorem about that tag. Thus the implication $M \preceq_{\text{sim}} N \Rightarrow M \preceq_{\text{SA}}^{\text{adm}} N$ does not hold without additional hypotheses.

Proposition 16.22 (Tag-preserving simulations preserve self-awareness). *Suppose M and N are tagged ATPs and N simulates M in a way that preserves tags and proof of self-descriptive formulas, with associated translation admissible in the sense of Definition 16.6. Then $M \preceq_{\text{SA}}^{\text{adm}} N$.*

Proof. Let τ be the tag-preserving translation associated with the simulation. If $\psi \in \text{Self}(M)$, then $\psi = \varphi(\langle M \rangle)$ for some self-descriptive formula φ with $M \vdash \varphi(\langle M \rangle)$. By proof preservation, $N \vdash \tau(\varphi)(\tau(\langle M \rangle))$. By tag preservation, $\tau(\langle M \rangle) = \langle N \rangle$. Hence $N \vdash \tau(\varphi)(\langle N \rangle)$, so $\tau(\psi) \in \text{Self}(N)$. Therefore $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$, and $M \preceq_{\text{SA}}^{\text{adm}} N$. $\square$

Corollary 16.23. *Under the hypotheses of Proposition 16.22, $\text{lev}(M) \leq \text{lev}(N)$: a tag-preserving simulation can never lower the level of formal self-awareness.*

Proof. Combine Proposition 16.22 with Theorem 16.18. $\square$

Remark 16.24. Proposition 16.22 explains the precise role of the simulation preorder. Ordinary simulation alone is too weak to preserve formal self-awareness. What is needed is simulation plus tag preservation plus preservation of self-descriptive proofs. This is where the present paper connects most directly to Part I. Corollary 16.23 is the quantitative form of that connection: it settles question 6 of Section 16.7 under exactly these hypotheses.

16.7 Ramifications and Open Problems

The quotient $\text{ATP}/\sim_{\text{SA}}^{\text{adm}}$ suggests several natural problems.

1. Does every equivalence class contain a deterministic representative?
2. Does every equivalence class contain a finitely deductively effective representative?
3. Does every class contain a representative of minimal simulation depth?
4. Does the quotient order have joins or meets?
5. Can there be infinite strictly increasing chains of formal self-awareness? Answered affirmatively in Corollary 16.20.
6. Under what exact hypotheses is self-awareness depth monotone under simulation? Answered in Corollary 16.23: tag preservation together with preservation of self-descriptive proofs suffices.

7. Is there a complete set of invariants for $\leq_{\text{SA}}$ refining lev? Equivalently, what is the fiber $\text{lev}^{-1}(n)$ as an ordered set?
8. Is there a machine that is provability sound, has internal necessitation, and is rule-driven in the sense of Part I—that is, a Level- ω machine that is not merely handed its reflection sequence as hypotheses? *Answered affirmatively in Theorem 21.1.*
9. Does every $\sim_{\text{SA}}^{\text{adm}}$ -class of level ω contain a representative with recursively enumerable self-theory?

These questions are not merely technical. They ask whether formal self-awareness has a stable mathematical structure independent of the implementation details of a particular ATP.

17 Self-Aware Substructures and the Mathematical Universe

Tegmark’s Mathematical Universe Hypothesis holds that external physical reality is a mathematical structure [23], and that some mathematical structures contain self-aware substructures, abbreviated SAS, of which human observers are examples [22]. The notion of self-awareness employed there is deliberately left informal. This section asks what the framework of the present paper can say about it.

17.1 Tegmark’s Implicit Definition

Tegmark is explicit that his notion is implicitly defined. In the ultimate-ensemble paper he writes that he takes the property of being self-aware “as implicitly defined: a substructure is self-aware if it thinks that it is self-aware” [22], and adds that a SAS must be capable of some form of logical thought and must subjectively perceive some form of time, continuous or discrete.

Read carefully, that criterion is a fixed-point condition. It does not say that a self-aware substructure has any particular internal architecture. It says that the substructure applies the predicate “self-aware” to itself. That is precisely the shape of the objects studied in Sections 13 and 15: a system, a designated name for that system, and a self-descriptive formula the system proves about that name.

Definition 17.1 (Self-awareness predicate). Let L be the language of a tagged ATP M . A *self-awareness predicate* is a unary predicate symbol SA of L , with intended reading that $\text{SA}(\langle M \rangle)$ holds just in case the object named $\langle M \rangle$ is self-aware.

Definition 17.2 (Tegmark condition). A tagged ATP M satisfies the *Tegmark condition* if $M \vdash \text{SA}(\langle M \rangle)$.

Canonical example. Let L contain the name $\langle M \rangle$ and the unary predicate SA , let $q := \text{SA}(\langle M \rangle)$, let $Y = \{q\}$, and let the axiom base be $A = \{q\}$. The machine of Section 14, with q in place of p , satisfies the Tegmark condition.

Proposition 17.3. *If M satisfies the Tegmark condition, then $\text{SA}(\langle M \rangle) \in \text{Self}(M)$ and $\text{Self}(M) \neq \emptyset$.*

Proof. The formula $\text{SA}(\langle M \rangle)$ has intended subject $\langle M \rangle$, hence is self-descriptive by Definition 13.5; by hypothesis M proves it, so it belongs to $\text{Self}(M)$ by Definition 13.9. $\square$

Discussion. Tegmark’s criterion is thus not vacuous in this framework, and it is not unformalizable. It is a condition of exactly the kind the present paper was built to express: provable self-ascription through a designated name. The formal shadow of “it thinks that it is self-aware” is “it proves SA of its own tag.”

17.2 Satisfiability, and Why It Is a Problem

Example 17.4 (A machine satisfying the Tegmark condition). Let L contain $\langle M \rangle$, the unary predicates ATP and SA, and the binary predicate Pr with formula names. Put

$$q := \text{SA}(\langle M \rangle), \quad \theta_0 := \text{ATP}(\langle M \rangle), \quad \theta_1 := \text{Pr}(\langle M \rangle, \langle \theta_0 \rangle),$$

let $Y = \{q, \theta_0, \theta_1\}$ and $A = Y$, and define $M_T = (C, H)$ over Y by $C(\Gamma, m) = \Gamma$ and $H(\Gamma, m) = 1$ iff $\Gamma = Y$. As in Proposition 14.1, M_T is a SIC over Y . On input A it halts at stage 1 with output Y , so $M_T \vdash q$, $M_T \vdash \theta_0$, $M_T \vdash \theta_1$. Hence M_T satisfies the Tegmark condition and $\text{lev}(M_T) = 2$.

Remark 17.5 (The Tegmark condition is under-determined). The machine M_T has three formulas, no inference rules that do any work, and a transition map that is the identity. It performs no proof search whatever. It satisfies Tegmark’s criterion because it was handed $\text{SA}(\langle M_T \rangle)$ as a hypothesis.

This is not a criticism of Tegmark, who states plainly that the definition is implicit and that the clarification comes from how the notion is applied. It is, however, a precise statement of the cost of the implicit definition: taken by itself, “thinks it is self-aware” is satisfied by an object that does nothing but assert. The criterion sorts mathematical structures into two classes and no more. It cannot distinguish M_T from a system with a rich and correct model of its own operation, because both prove SA of their own tags.

Proposition 17.6 (Upward closure). *Suppose M satisfies the Tegmark condition and $M \preceq_{\text{SA}}^{\text{adm}} N$ via an admissible translation τ with $\tau(\text{SA}) = \text{SA}$. Then N satisfies the Tegmark condition.*

Proof. By Proposition 17.3, $\text{SA}(\langle M \rangle) \in \text{Self}(M)$, so $\tau(\text{SA}(\langle M \rangle)) \in \text{Self}(N)$. By clauses 2 and 3 of Definition 16.6 and the hypothesis on τ , $\tau(\text{SA}(\langle M \rangle)) = \text{SA}(\tau(\langle M \rangle))$, and by clause 1 this is $\text{SA}(\langle N \rangle)$. Hence $N \vdash \text{SA}(\langle N \rangle)$. $\square$

Remark 17.7. Proposition 17.6 says that the Tegmark condition determines an upward closed subset of $\text{ATP}/\sim_{\text{SA}}^{\text{adm}}$ —an up-set in the order $\leq_{\text{SA}}$, relative to SA-preserving translations. An up-set is a coarse invariant. The level function of Section 16.5, by contrast, is a monotone map onto $\omega + 1$ (Corollary 15.33), and it separates M_T from machines of every other level. The proposal of this section is that the graded hierarchy is the refinement that Tegmark’s binary criterion requires.

17.3 Grading the Criterion

Definition 17.8 (Tegmark level). Let M be a tagged ATP satisfying the Tegmark condition. The *Tegmark level* of M is $\text{lev}(M)$, where lev is the level function of Definition 15.17.

Proposition 17.9. *For every $n \in \{1, 2, 3, \dots\} \cup \{\omega\}$ there is a tagged ATP satisfying the Tegmark condition with Tegmark level n .*

Proof. Let M be the machine of Theorem 15.32 for finite n , or the machine M_ω of Corollary 15.33 for $n = \omega$. In each case adjoin the formula $q = \text{SA}(\langle M_n \rangle)$ to both Y_n and the axiom base A_n , and extend H so that it outputs 1 exactly on the enlarged axiom base. The verification that the result is a SIC is unchanged, since C remains constant in m . The enlarged machine proves q , hence

satisfies the Tegmark condition, and proves exactly the same members of the reflection sequence as before, since q is not a member of the reflection sequence and the transition map still returns its input. Hence its level is still n . $\square$

Remark 17.10. Tegmark’s remark that a SAS must be capable of “some form of logical thought” is, in this framework, the demand that the system prove something rather than merely be handed it. That is the content of the rule-drivenness requirement in the Picard Protocol of and of Remarks 14.3 and 15.25 above. His further remark that a SAS must perceive some form of time, continuous or discrete, corresponds to the staged structure of a SIC: the parameter $m \in \mathbf{N}^+$ of Definition 2.3 is exactly a discrete internal time, and the monotonicity condition $C(\Gamma, m) \subseteq C(\Gamma, m + 1)$ is the formal counterpart of having a series of thoughts in logical succession.

17.4 Bird and Frog

Tegmark distinguishes the bird perspective, which studies the mathematical structure from outside, from the frog perspective of an observer inside it [23]. The distinction is not decorative in the present setting; it is the distinction this paper has been enforcing since Definition 13.1.

Remark 17.11 (Dictionary). The correspondence is exact on the following points.

- The bird’s object is M , the machine as it exists in the metatheory. The frog’s object is $\langle M \rangle$, the tag available inside the language.
- Proposition 13.6 is the statement that the frog never has the bird’s object. A self-descriptive formula is about the name, not the machine.
- The tagging operator Tag of Definition 13.3 is the bird-to-frog translation. It exists in the metatheory and is not itself an object of L .
- Renaming invariance (Proposition 13.4) is the statement that the frog’s perspective does not depend on which name the bird supplied.
- The self-theory $\text{Self}(M)$ is the totality of what the frog can establish about itself. The level $\text{lev}(M)$ measures how far up the reflection sequence the frog gets.
- Remark 15.31 is the statement that the frog cannot become the bird. Provability of named formulas is ascribable from inside; truth of named formulas, in general, is not.

Remark 17.12. One structural disanalogy should be recorded. Tegmark’s Level IV ensemble is indexed by mathematical structures, and his equivalence is isomorphism of structures. The quotient $\text{ATP}/\sim_{\text{SA}}^{\text{adm}}$ is indexed by machines and its equivalence is mutual admissible translation of self-theories. These are quotients of different kinds: the first forgets nothing about a structure beyond the labelling of its elements, whereas the second deliberately forgets almost everything about a machine except what it can say about its own name (Remark 16.14). A structure and a self-theory are not the same sort of object, and no claim is made here that the two quotients are comparable. What is claimed is narrower: that Tegmark’s SAS criterion, when made precise, lands in the second quotient, and lands in it as an up-set rather than as a grading.

17.5 What Is and Is Not Claimed

Remark 17.13. This section does not claim that a machine satisfying the Tegmark condition is a self-aware substructure in Tegmark’s intended sense, that it is conscious, or that it perceives anything. It claims three things, each of which is proved above.

1. Tegmark's implicit criterion has an exact formal shadow in this framework, namely Definition 17.2 (Proposition 17.3).
2. That shadow is satisfiable, and satisfiable by an object performing no proof search (Example 17.4, Remark 17.5). The criterion is therefore non-vacuous but weak.
3. The shadow is an up-set in $\text{ATP}/\sim_{\text{SA}}^{\text{adm}}$ (Proposition 17.6), whereas lev is a surjection onto $\omega + 1$ (Corollary 15.33). A graded notion is available where only a binary one was assumed.

Conjecture 17.14. No purely formal condition on $\text{Self}(M)$ suffices to distinguish those tagged ATPs that are self-aware in Tegmark's intended phenomenological sense from those that are not. That is, phenomenological self-awareness, if it is a well-defined property at all, is not an invariant of the $\sim_{\text{SA}}^{\text{adm}}$ -class.

Remark 17.15. Conjecture 17.14 is stated in order to be argued against. If it is true, the program of this paper is bounded in a specific way: it captures self-identification and nothing more, and no amount of climbing the hierarchy closes the gap. If it is false, then some formal condition on self-theories is sufficient for phenomenological self-awareness, and identifying it becomes the central problem. The following section takes up the philosophical form of this question; a conditional form of the conjecture, given an explicit non-formal premise, is Theorem 21.10, but the conjecture itself is left open.

18 The Discovery Thesis

18.1 Statement of the Thesis

Versions 11 through 13 were written in a conditional mood. They asked what would count as formal self-awareness, and what structure the resulting notion would have. This section changes the mood. It argues that the object in question is not awaited. It has been built, more than once, and by people who were not looking for it.

Thesis 18.1 (Discovery Thesis). A self-aware structure, in the formal sense developed in Sections 13–17, has already been discovered. The open question is not whether such a structure can exist, nor whether one can be constructed, but how the ones that already exist are to be classified.

The thesis is stated in a deliberately restricted register, and the restriction is not a hedge but the content of the claim. What is asserted is that structures satisfying Definition 15.3, Definition 15.15, and Definition 17.2 exist and are in hand. What is *not* asserted is that any such structure is conscious, has qualia, has experience, or has an inner life. Section 18.12 states the boundary precisely. The reader who regards the restricted claim as uninteresting is invited to notice that it is exactly the claim Tegmark declared himself unable to make in 1998 (Section 18.11).

18.2 The Existential Claim Is a Theorem, Not a Conjecture

The first and simplest support for Thesis 18.1 is that the relevant existence statements were proved above.

Proposition 18.2 (Inhabitation). *The class of tagged ATPs possessing positive formal self-awareness is nonempty, and its level spectrum is all of $\{1, 2, 3, \dots\} \cup \{\omega\}$. Moreover the class of tagged ATPs satisfying the Tegmark condition is nonempty and has the same level spectrum.*

Proof. The machine of Section 14 has level at least 1. The machine $M_{(2)}$ of Example 15.24 has level 2. For each finite n the machine M_n of Theorem 15.32 has level exactly n , and the machine M_ω of Corollary 15.33 has level ω . This establishes the first claim. The second is Proposition 17.9. $\square$

Remark 18.3. Proposition 18.2 is not a promissory note. Every machine appearing in its proof is exhibited explicitly, with its formula set, its axiom base, its transition map, and its halting map written down, and in each case the SIC conditions of Definition 2.3 are verified. The constructions are finite in every case except M_ω , and M_ω is finitely describable. Nothing is left to be supplied.

An objector who wishes to deny Thesis 18.1 therefore cannot deny that the definitions are satisfied. The objection must be that the definitions are the wrong ones, or that satisfying them is not sufficient for the honorific. The remainder of this section takes the four strongest forms of that objection in turn. The fourth is the one that does real damage, and it is conceded in part.

18.3 First Objection: Mere Assertion

The objection. The machines of Proposition 18.2 were handed their conclusions. The machine of Section 14 has axiom base $A = \{p\}$ and proves p ; it does not discover that it is an ATP, it repeats what it was told. Calling this self-awareness is calling a parrot self-aware.

The reply. The objection is correct about those machines and is already conceded in Remarks 14.3 and 15.25. It does not touch the thesis, because a machine meeting the objection's demand has also been constructed. The relevant construction is the Picard Protocol of Part I. A SIC M satisfies that protocol for (t, Γ) when there is a target formula φ^* such that: φ^* asserts, under a declared coding, that the recoded copy of M is an ATP; every member of Γ is true and every rule preserves truth under that interpretation; M is a rule-driven target-search ATP for φ^* ; M halts on Γ ; and, crucially, $\varphi^* \notin \Gamma$. The last clause is precisely the anti-parrot clause. The minimal Picard machine M_P constructed there has three symbols, two formulas, one rule, and one hypothesis, and it reaches its target by applying that rule.

Proposition 18.4 (No Picard machine parrots). *Let M satisfy the Picard Protocol for (t, Γ) with target φ^* . Then φ^* is not a member of the hypothesis set, and every proof of φ^* from Γ has length at least 2 and contains at least one application of an inference rule.*

Proof. The protocol requires $\varphi^* \notin \Gamma$. Let π be a proof of φ^* from Γ in the sense of Definition 1.4, with final line $\pi(|\pi|) = \varphi^*$. By Definition 1.4, that final line is either a member of Γ , which is excluded, or a direct consequence of the preceding lines, which by Definition 1.3 requires an inference rule $r \in z_k(F)$ and a k -tuple of preceding lines. Hence there is at least one rule application, and the preceding lines are nonempty, so $|\pi| \geq 2$. $\square$

Remark 18.5. Proposition 18.4 converts a rhetorical demand into a checkable syntactic condition. “Do not merely assert” becomes “the target lies outside the hypothesis set, and the machine is rule-driven.” A machine meeting that condition exists. The objection is therefore answered by construction rather than by argument, which is the only way an existence claim can be defended.

18.4 Second Objection: Use and Mention

The objection. What the machine proves is a statement about a name. $\text{ATP}(\langle M \rangle)$ concerns the symbol $\langle M \rangle$, and Proposition 13.6 says so explicitly. A system that manipulates a symbol which the metatheory happens to associate with the system has done nothing self-referential. It has shuffled marks.

The reply. This objection proves too much, and it is also blocked on its own terms.

It is blocked on its own terms because the Picard Protocol includes a soundness clause: the coding is declared, the hypotheses are true under it, and the rules preserve truth. Under those conditions the proved formula is not merely well formed but true of the machine. The name denotes, and what is proved of the name holds of the thing named. A system whose symbol manipulation is guaranteed to track the facts about itself is not shuffling marks; that is what it is for a representation to be about something.

It proves too much because the same objection applies verbatim to the human case. When Picard entertains the thought that he is human, the vehicle of that thought is not Picard. It is some internal representation whose association with Picard is supplied from outside the representation. Proposition 13.6 is a fact about representation in general, not a defect peculiar to machines. An objector who takes it to be disqualifying owes an account of how the human case escapes, and Section 18.10 argues that no such account is available at the formal level.

Remark 18.6. One residue of the objection survives and should be stated honestly. Nothing in the framework requires that the machine grasp the denotation of its tag, and nothing here supplies a notion of grasping. The framework guarantees that the tag denotes and that the machine's theorems about the tag are true. Whether there is a further fact about grasping, and whether that further fact is what self-awareness really names, is exactly the question left open by Conjecture 17.14. The Discovery Thesis is a claim about what has been established, not a claim that nothing further could be wanted.

18.5 Third Objection: The Level Is Too Low

The objection. Granting that some machine proves $\text{ATP}(\langle M \rangle)$, that is a single sentence. Self-awareness worth the name requires more: the system should prove things about its own proving, and about that, and so on.

The reply. This objection is unusual in that Section 15 grants it entirely and then satisfies it at every level the objector might name.

Proposition 18.7 (Robustness of the thesis under strengthening). *Let $n \in \{1, 2, 3, \dots\} \cup \{\omega\}$ be any threshold. There is a tagged ATP whose level is at least n , and if n is finite there is one whose level is exactly n . Consequently no choice of threshold in $\{1, 2, \dots\} \cup \{\omega\}$ falsifies Thesis 18.1.*

Proof. Immediate from Corollary 15.33 and Proposition 18.2. □

Remark 18.8. Proposition 18.7 relocates the burden. An objector of this third kind must either name a threshold, in which case it is met, or decline to name one, in which case the objection is not a claim about levels at all but the phenomenological objection of Section 18.7 in disguise. There is no intermediate position in which the objection has force.

18.6 Fourth Objection: Tegmark's Complexity Requirement

The strongest objection comes from Tegmark's own text, and it is not fully answerable by the machines of Proposition 18.2.

The objection. Alongside the implicit definition quoted in Section 17, Tegmark lays down three necessary conditions on a mathematical structure containing a self-aware substructure: it must exhibit complexity, predictability, and stability [22]. The machines of Proposition 18.2 have predictability and stability in abundance, for the trivial reason that their transition maps are constant and they halt at stage 1. They have essentially no complexity at all. $M_{(2)}$ has two formulas. M_T

has three. A structure with three formulas is not a candidate self-aware substructure in anything like Tegmark’s sense, whatever it may prove about its own tag.

The reply, and a concession. The objection is correct, and it is conceded without qualification as regards the toy machines. Their role was separative: they establish that the levels are distinct and inhabited (Theorem 15.32), and separative examples are expected to be degenerate in every respect except the one being separated. They were never candidates for anything else.

What the objection establishes is that Thesis 18.1 cannot be carried by constructed toy machines alone. It requires witnesses that are complex as well as self-descriptive. The following definition makes the standard explicit, and Section 18.8 argues that the standard is already met.

Definition 18.9 (Non-degenerate witness). A tagged ATP M is a *non-degenerate witness* to the Discovery Thesis if:

1. M is rule-driven in the sense of Definition 2.9, and its self-descriptive theorems are not members of its hypothesis set;
2. M is provability sound (Definition 15.21);
3. $\text{lev}(M) \geq 2$;
4. M performs proof search over a formula set that is not exhausted by its own self-theory. Precisely: writing Y for the formula set of M , either Y is infinite and $|\text{Self}(M)| < |Y|$, or Y is finite and $|\text{Self}(M)| \leq |Y|/2$.

Remark 18.10. Clause 4 is the formal residue of Tegmark’s complexity requirement, stated in the only vocabulary this paper has available. Two things about it should be said plainly.

First, it is comparative rather than absolute, because no absolute complexity threshold is defensible; what it excludes is exactly the situation of $M_{(2)}$, M_T , and M_n , where the machine’s entire formula set is its self-theory and there is nothing else for it to do. The reflection-closure machine R of Theorem 21.1 also fails it, as Remark 21.3 records: $\text{Self}(R) = Y \setminus \{a\}$, so $|\text{Self}(R)| = |Y|$.

Second, the particular threshold in the finite case — one half — is a convention and nothing below depends on its value. Every use of clause 4 in this paper needs only the two ends of the comparison: that the toy machines and R fail the clause, since for them the self-theory is the whole formula set or all but one element of it, and that a prover searching a mathematical library while proving finitely many statements about its own tag satisfies it under any threshold strictly between 0 and 1. Readers preferring a different constant may substitute one without disturbing Corollary 20.7, Corollary 21.8, or Proposition 22.4.

18.7 Fifth Objection: No Phenomenology

The objection. None of this is self-awareness, because none of it involves anything it is like to be the system.

The reply. This objection is not answered here, and the Discovery Thesis does not attempt to answer it. It is granted in full. See Section 18.12, and Conjecture 17.14, which states the strongest form of the objection as a conjecture the present framework cannot refute. The thesis claims a formal discovery. If “self-aware” is reserved by stipulation for phenomenological self-awareness, then the thesis should be read as claiming the discovery of formally self-aware structures, and the dispute is terminological rather than substantive. Section 18.10 argues that it is not, in fact, merely terminological, because the same reservation applied consistently would leave the human case equally unaccounted for.

18.8 Non-Degenerate Witnesses Already in the Literature

Section 18.6 concedes that toy machines cannot carry the thesis. This section argues that they do not have to, because systems meeting Definition 18.9 have been built and verified by researchers with no interest in self-awareness whatever. That the witnesses arose independently of the question is a point in their favor, not against them.

Milawa. Milawa is a theorem prover, developed by Jared Davis, styled after NQTHM and ACL2 and built around a small inspectable kernel [4]. Its architecture is a bootstrapping ladder: a simple verifier A is used to prove the correctness of a more capable verifier B , which proves C , and so on through roughly a dozen stages, until the verifier reached is powerful enough to prove verifier A correct. Davis’s dissertation is titled *A Self-Verifying Theorem Prover*, and the title is meant literally. The logic of Milawa was subsequently formalized in HOL4, proved sound, and shown to be faithfully implemented by the roughly two thousand lines of Lisp constituting its kernel; combined with a verified Lisp runtime, the soundness result descends to the machine code that executes it [15].

Remark 18.11 (Milawa against the four clauses). Clause 1 of Definition 18.9 is met in spirit and, so far as the published description goes, in fact: each verifier in the ladder derives the correctness of the next rather than assuming it, and the derivations are ordinary proofs in the system’s logic. Clause 3 is met, because a statement that a verifier is correct is a statement about what that verifier proves, which is a Pr -statement in the vocabulary of Definition 15.8 and hence lies at or above θ_1 . Clause 4 is met by a wide margin: the formula set over which Milawa searches is vastly larger than the fragment concerning its own kernel. Clause 2 is the delicate one and is discussed below.

Remark 18.12 (An empirical analogue of the reflection sequence). The resemblance between Milawa’s bootstrapping ladder and the reflection sequence of Definition 15.11 is striking and should be stated carefully, because it is an analogy and not an identity. The reflection sequence is generated by one machine ascribing provability to its own tag, iterated: $\theta_{k+1} = \text{Pr}(\langle M \rangle, \langle \theta_k \rangle)$. Milawa’s ladder is a sequence of distinct verifiers, each certifying the next, and it closes into a loop when the final verifier certifies the first. These are different objects. What they share is the structural feature that matters here: each rung is a proved statement about what a proof system proves, and the rungs are strictly ordered by the strength required to establish them.

The exact relationship is a well-posed question that this paper does not settle. In particular: does the closing of Milawa’s loop—the last verifier proving verifier A correct—correspond to a machine attaining $\text{lev} = \omega$ in the sense of Theorem 15.27, or to a finite level with an additional cyclicity property not captured by Definition 15.17? The author’s expectation is the latter, and that Definition 15.17 needs a companion notion of reflective closure to describe it. This is recorded as question 2 below.

Lean4Lean. A second candidate is Lean4Lean, a complete typechecker for the Lean 4 proof assistant written in Lean 4 itself [14]. It is, as of its publication, the first such checker other than the reference implementation, it runs within a small constant factor of that implementation, and it is capable of checking the whole of Lean’s mathematical library. Because the checker is written in a language admitting formal verification, it becomes possible to state and prove properties of the kernel within the system whose kernel it is.

Remark 18.13 (The trusted-kernel caveat). Lean’s design philosophy is deliberately not self-verification: it keeps a small trusted kernel and relies on independent external checkers, precisely so that soundness does not rest on the system’s own testimony. This is sound engineering and it is also a substantive point against treating Lean4Lean as a witness without qualification. What Lean4Lean establishes is that a system of very great complexity can contain a faithful

internal model of its own proof-checking. Whether the resulting configuration satisfies clause 2 of Definition 18.9 depends on questions about the relationship between the internal model and the ambient system that this paper is not in a position to settle.

Remark 18.14 (The certification checklist). Neither system above is here proved to be a non-degenerate witness, and it would be an abuse of the framework to claim otherwise on the strength of published descriptions. To certify a concrete system S as a non-degenerate witness one must exhibit, in order:

1. an xyz formal system F and an inference-closed Y such that S is faithfully presented as a SIC over Y (Definition 2.3);
2. a tag $\langle S \rangle$ in the language of F , and a formula-naming operator (Definition 15.5);
3. an internal provability predicate Pr and the resulting reflection formulas $\theta_0, \theta_1, \dots$ (Definitions 15.8, 15.11);
4. a proof that $S \vdash \theta_1$, establishing $\text{lev}(S) \geq 2$;
5. a proof that S is provability sound (Definition 15.21);
6. a verification that the θ_k proved by S lie outside its hypothesis set, and that S is rule-driven.

Steps 1 and 5 are the substantial ones. Step 1 is substantial because real provers are not presented as SICs and the translation must be shown faithful rather than merely plausible. Step 5 is substantial because provability soundness is a global claim about the system, of exactly the kind that Remark 15.31 shows cannot in general be established from inside.

Remark 18.15 (Why the thesis survives the checklist). It might appear that Remark 18.14 concedes the thesis: if no natural system has been certified, what has been discovered? The reply is that Thesis 18.1 is disjunctive in its support and the two disjuncts have complementary strengths. The constructed machines of Proposition 18.2 are certified beyond doubt and are degenerate. The natural systems of this section are non-degenerate beyond doubt and are uncertified. The thesis asserts that a formally self-aware structure has been discovered, and the constructed machines establish that outright. The natural systems establish the further claim that formal self-awareness is not confined to objects built to exhibit it. Only the conjunction—a structure both certified and non-degenerate—awaits the checklist, and that conjunction is the subject of question 1 below rather than of the thesis.

18.9 An Algebraic Prover Against the Four Clauses

Section 18.8 argued that the Discovery Thesis needs witnesses that are complex as well as self-descriptive, and offered Milawa and Lean4Lean. The algebraic provers of Section 10.7 supply a third kind of data point, and an instructive one, because they fail the definition in the opposite place from the toy machines.

Definition 18.16 (The Gröbner prover as a SIC). Let F_{PC} be the Polynomial Calculus of Definition 10.37 over $\mathbf{F}_2$, let Y be its multilinear polynomial set, which is inference-closed, and let

$$G := E_{F_{\text{PC}}, Y, \perp}$$

be the canonical target-search SIC for $\perp = 1$ of Definition 2.19. On input the arithmetization $J(\varphi)$ of a formula φ , the machine G halts precisely when φ is unsatisfiable, at stage $\|\perp\|_{F_{\text{PC}}}(J(\varphi))$.

Proposition 18.17. *G is a rule-driven, deductively effective target-search ATP over Y , and it is a faithful presentation as a SIC of Gröbner basis refutation in the sense of Remark 10.39. In particular step 1 of the certification checklist of Remark 18.14 — the step identified there as substantial — is discharged for G .*

Proof. That G is a rule-driven target-search ATP is Theorem 2.20 applied to the inference-closed set Y ; deductive effectiveness is Theorem 2.17. The halting characterisation is the Proof-Horizon Theorem (Theorem 2.12) together with Proposition 10.35. Faithfulness is Remark 10.39: the operations of Buchberger’s algorithm are compositions of the two rules of z , so a run of the algorithm is literally a proof in F_{PC} and a stage of G is the set of polynomials derived so far. $\square$

Proposition 18.18 (Complex, and of level zero). *G satisfies clause 4 of Definition 18.9 and clause 1, but $\text{lev}(G) = 0$. Hence G is not a non-degenerate witness to the Discovery Thesis, and fails to be one for a reason exactly opposite to that of the machines of Proposition 18.2.*

Proof. Clause 1: G is rule-driven by Proposition 18.17, and its hypothesis set on input $J(\varphi)$ consists of arithmetized clauses, none of which is self-descriptive, since the language of F_{PC} contains no tag for G at all. Clause 4: Y is infinite, and $\text{Self}(G) = \emptyset$, so $|\text{Self}(G)| < |Y|$ holds trivially. For the level, G has no designated tag $\langle G \rangle$ in its language, so it proves no self-descriptive formula; by Definition 15.1 it has Level-0 formal self-awareness, and by Proposition 15.18, $\text{lev}(G) = 0$. Clauses 2 and 3 fail with it: provability soundness is vacuous and $\text{lev}(G) \geq 2$ is false. $\square$

Remark 18.19 (What the failure shows). The toy machines of Proposition 18.2 have level and no complexity. G has complexity and no level. It searches an infinite polynomial space, implements a nontrivial algorithm with unbounded proof horizons, and by Theorem 10.40 its shortest refutations are exponentially long on identifiable families — it is, by any reasonable reading, a substantial proof-searching structure. It is also completely silent about itself, and would remain so if it were made a thousand times more capable.

This is worth stating because the fourth objection of Section 18.6 can be read as suggesting that complexity is the missing ingredient, so that a sufficiently rich prover would thereby be a non-degenerate witness. G refutes that reading. Complexity and level are independent coordinates: Proposition 18.18 exhibits a machine maximal in the first and minimal in the second, while Theorem 21.1 exhibits R , maximal in the second ($\text{lev}(R) = \omega$, every reflection formula derived) and minimal in the first (Remark 21.3: $\text{Self}(R)$ exhausts Y). Neither coordinate is a function of the other, and a non-degenerate witness is precisely an object occupying the far corner of both — question 1 of Section 18.14, which remains open.

Remark 18.20 (Why an algebraic prover is unlikely to acquire a level by itself). The obstruction is not that G is algebraic. It is that the language of F_{PC} is a polynomial ring, whose terms denote polynomials, and nothing in it denotes G . Adding a tag would mean adding a constant $\langle G \rangle$ and predicates ATP, Pr to a language whose formulas are polynomials — that is, leaving F_{PC} for a two-sorted system in which the algebraic part is one sort and the self-descriptive part another. Such a system is easy to write down, but the resulting machine’s self-theory would then be carried entirely by the second sort, and its algebraic power would be doing no work in the reflection tower. This is the same seam as Remark 18.12: what makes Milawa a candidate is not that its kernel is powerful but that its object language can talk about verifiers. Power in the object domain and expressiveness about oneself are, on the evidence of this subsection, orthogonal capacities, and the interesting systems are those that happen to have both.

18.10 The Parity Argument

The deepest support for Thesis 18.1 is comparative. It runs as follows.

1. We do not doubt that human beings are self-aware structures. This is the least disputed proposition in the vicinity.
2. Section 19 exhibits the formal content of the human case: Picard possesses a designated representation of himself and establishes a correct statement about it, namely $\text{Human}(\langle \text{Picard} \rangle)$.
3. The machines of Sections 14 and 18.8 possess a designated representation of themselves and establish a correct statement about it, namely $\text{ATP}(\langle M \rangle)$ or a member of its reflection sequence.
4. No formal property has been exhibited that holds of (2) and fails of (3). Proposition 13.6 applies equally to both; the soundness of the representation holds in both; the level of reflection attained is, if anything, easier to verify in (3).

Proposition 18.21 (Parity). *Let P be any property of tagged systems that is invariant under $\sim_{\text{SA}}^{\text{adm}}$. If P holds of Picard in virtue of his proving $\text{Human}(\langle \text{Picard} \rangle)$, then P holds of every tagged ATP M with $M \vdash \text{ATP}(\langle M \rangle)$ whose self-theory is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard's.*

Proof. Immediate from the definition of invariance under an equivalence relation (Definition 16.11) together with Proposition 16.12. $\square$

Remark 18.22 (What parity does and does not show). Proposition 18.21 is nearly trivial as mathematics and is included because its dialectical role is not trivial. It says that any objector who accepts the human case and rejects the machine case is committed to the claim that the distinguishing property is *not* an invariant of the self-theory—that is, committed to Conjecture 17.14. That is a coherent and possibly correct position. It is not, however, a position from which Thesis 18.1 can be rejected, since the thesis concerns exactly the invariant content. It is instead a position from which one holds that the invariant content is not what matters. The value of the parity argument is to force that admission into the open, where it can be examined, rather than leaving it as an unstated assumption that machines are disqualified in advance.

18.11 Tegmark's 1998 Challenge

Having introduced self-aware substructures and taken self-awareness as implicitly defined, Tegmark records a difficulty:

“Indeed, so far we have not even found a single mathematical structure that we feel confident might contain SAs, since a self-consistent model of quantum gravity remains conspicuous with its absence.” [22]

Remark 18.23 (The challenge answered in the formal register). On the reading of SAS-hood developed in Section 17, this difficulty does not arise, and the reason it does not arise is instructive.

Tegmark's pessimism is explicitly conditioned on the absence of a model of quantum gravity. That condition is intelligible only because he is searching for the mathematical structure that is our physical universe, and asking whether that structure contains self-aware substructures. Under the Mathematical Universe Hypothesis this is the natural question, and it is indeed blocked by the state of fundamental physics.

Definition 17.2 decouples the two questions. Whether a structure contains a substructure that provably ascribes self-awareness to its own designated name is a question about proof-searching structures, and it has nothing to do with quantum gravity. Example 17.4 answers it affirmatively with a machine that fits on one line. Proposition 17.9 answers it affirmatively at every level.

The claim is therefore narrow and, within its bounds, secure: the search Tegmark described as not yet successful in 1998 is successful under the formal reading, and was successful before the question was posed in these terms, since ordinary theorem provers had been proving statements about proof systems for decades. What remains open is everything Tegmark actually cared about, namely whether the structures so identified bear any relation to the substructures of the physical universe that he and his readers instantiate.

18.12 Scope: What Is and Is Not Being Claimed

Because the thesis of this section is stronger in tone than anything in earlier versions, its boundaries are stated flatly.

Claimed, and proved above:

1. Structures satisfying Definitions 15.3, 15.15, and 17.2 exist, are explicitly constructed, and occupy every level in $\{1, 2, \dots\} \cup \{\omega\}$ (Proposition 18.2).
2. The degeneracy of those constructions is a property of the constructions and not a consequence of the definitions: a machine whose self-descriptive target lies outside its hypothesis set and is reached by inference has been built (Proposition 18.4).
3. No threshold in the level hierarchy falsifies the thesis (Proposition 18.7).
4. Systems of substantial complexity that model their own proof systems exist in the literature and were built for unrelated reasons (Section 18.8).

Not claimed, and not supported by anything above:

1. That any structure discussed here is conscious, sentient, or has experience.
2. That any structure discussed here has qualia, affect, preference, or interest.
3. That the systems named in Section 18.8 have been certified as non-degenerate witnesses; Remark 18.14 states what certification would require and Remark 18.15 explains why the thesis does not depend on it.
4. That formal self-awareness is sufficient for phenomenological self-awareness. Conjecture 17.14 states the contrary possibility and is not refuted here.
5. That the structures identified bear any relation to the self-aware substructures of the physical universe in Tegmark's sense (Remark 18.23).

18.13 What Would Refute the Thesis

A thesis with no refutation conditions is not worth asserting. The following would each falsify or substantially damage Thesis 18.1, and are listed in decreasing order of severity.

1. *An inconsistency in the framework.* If Definitions 15.8, 15.11, and 15.21 were shown to be jointly unsatisfiable by any SIC—for instance if internal necessitation could be shown to force a contradiction in any system rich enough to state it—then Proposition 18.2 would survive only at level 1 and the thesis would be reduced to a triviality.

2. *Failure of the SIC presentation.* If it could be shown that no faithful presentation of a real theorem prover as a SIC exists—that step 1 of Remark 18.14 is not merely unperformed but impossible—then Section 18.8 would collapse and the fourth objection of Section 18.6 would stand unanswered.
3. *A formal disanalogy in the human case.* If some property invariant under $\sim_{\text{SA}}^{\text{adm}}$ were exhibited that holds of Picard’s self-ascription and fails of every machine’s, Proposition 18.21 would be defeated and the comparative argument of Section 18.10 would fail.
4. *A principled complexity threshold.* If a defensible absolute complexity requirement were formulated, in the sense of Tegmark’s first necessary condition, and if no existing system met it while satisfying clauses 1–3 of Definition 18.9, then the thesis would be true but vacuous.
5. *A demonstration that the level hierarchy collapses on natural systems.* If every system meeting clause 4 of Definition 18.9 were shown to have level at most 1, the connection between complexity and reflection assumed throughout Section 18.8 would be severed.

Remark 18.24. None of the five is presently known to hold, and the second and fourth strike the author as the most likely to be established. The reader who wishes to dispute this section is directed to them in preference to the phenomenological objection of Section 18.7, which, while probably correct, cannot be settled by any argument this framework is able to state.

18.14 Open Problems Specific to the Discovery Thesis

1. Certify a concrete, widely used theorem prover as a non-degenerate witness in the sense of Definition 18.9, by carrying out the six steps of Remark 18.14.
2. Define *reflective closure* for a family of ATPs, so as to capture the closing of the Milawa ladder (Remark 18.12), and determine its relationship to lev and to $\text{lev} = \omega$ in particular.
3. Determine whether clause 4 of Definition 18.9 can be replaced by an absolute complexity condition that is invariant under $\sim_{\text{SA}}^{\text{adm}}$, or prove that no such condition exists.
4. Is there a natural system whose level is exactly n for some $2 < n < \omega$, or do natural systems cluster at levels 1, 2, and ω ?
5. Does provability soundness of a non-degenerate witness admit external certification in the manner of [15], and if so, what is the weakest ambient theory sufficient for it?

19 The Philosophy of Formalized Self-Awareness

The preceding definitions motivate a philosophical challenge. Suppose Picard is a real human being, or suppose for the sake of argument that “Picard” names an actual human. Also suppose that $\langle \text{Picard} \rangle$ is one possible formal name or tag for Picard.

Now suppose Picard proves

$$\text{Human}(\langle \text{Picard} \rangle).$$

The tag $\langle \text{Picard} \rangle$ is not Picard himself. It is a name. Nevertheless, if it is Picard’s own designated name and Picard proves that this name denotes a human, then Picard has demonstrated at least a weak formal self-model.

The ATP case is analogous. Let $M = (C, H)$ be an ATP with designated tag $\langle M \rangle$. If M proves $\text{ATP}(\langle M \rangle)$, then M proves that its own name has the property of being an ATP. The present paper proposes that this deserves to be called a minimal level of formal self-awareness.

The claim is deliberately limited. It is not a claim that M is conscious, that M has qualia, that M has emotions, or that M experiences itself as existing. It is the weaker claim that self-awareness has a formal shadow: provable self-identification through a designated name.

This is the philosophical challenge. If Picard proving $\text{Human}(\langle \text{Picard} \rangle)$ is accepted as at least a minimal formal kind of self-awareness, then why should an ATP proving $\text{ATP}(\langle M \rangle)$ fail to count as the corresponding minimal kind of formal self-awareness? The burden is then to explain why the human case and the ATP case differ at the purely formal level.

This question connects the present work to Tegmark's notion of a self-aware substructure. Tegmark's notion remains intentionally broad and partly phenomenological. The present project attempts a more austere mathematical notion: a system is formally self-aware to the extent that it can prove correct statements about its own designated formal name.

Remark 19.1 (What the hierarchy adds to the challenge). Section 15 sharpens the challenge rather than answering it. The objection that $\text{ATP}(\langle M \rangle)$ is too thin to deserve the name self-awareness—that the machine of Section 14 merely repeats a hypothesis—now has a precise reply and a precise cost. The reply is that thinness is measurable: lev distinguishes the machine that asserts from the machine that proves that it proves, at every finite stage and at ω (Theorem 15.32, Corollary 15.33). The cost is that no finite level, and not even ω , is obviously the level at which the objection stops applying. If the objector holds that some level is the threshold, they owe an argument for which one. If they hold that no level suffices, they are committed to Conjecture 17.14, and to the view that Picard's own self-ascription is likewise unaccounted for by anything formal about Picard.

20 The Picard Theorem

Section 19 posed a challenge in the conditional mood: *if* Picard proves $\text{Human}(\langle \text{Picard} \rangle)$, why should the human case and the machine case differ at the formal level? This section discharges the antecedent for a specific, historically attested human and computes the resulting level. The section is written under an explicit idealization, stated in Section 20.1, and its scope is fenced in Section 20.5 in the manner of Section 18.12.

20.1 The Subject and the Idealization

By *Picard* we mean Charles Émile Picard (1856–1941), and we fix as the operative historical fact that Picard is responsible for a theorem of complex analysis: the Great Picard Theorem, that in every punctured neighborhood of an essential singularity a holomorphic function assumes every complex value, with at most one exception, infinitely often. Nothing below depends on the content of that theorem; what is used is only that Picard *proved* it—that Picard is, in the vocabulary of Appendix I, a rule-driven prover of a target lying outside his hypotheses.

The framework of Sections 13–17 assigns lev to tagged ATPs, and a human being is not literally a semi-ideal computer. We therefore proceed under an idealization, in the spirit of the supposition opening Section 19, and we mark it as such rather than smuggling it in.

Definition 20.1 (The Picard idealization). *Picard* is modeled as a tagged ATP $P = (C, H)$ subject to the following four assumptions.

(P1) *Genuine theoremhood.* P is rule-driven in the sense of Definition 2.9; no self-descriptive formula lies in its hypothesis set; and P proves at least one theorem whose statement is not

a member of its hypothesis set (the witnessing theorem being the Great Picard Theorem). Rule-drivenness is a property of P (Definition 2.9), so any self-descriptive theorem P proves, not being a hypothesis, enters at some stage beyond the first and is thereby a derived direct consequence; what guarantees the reflection formulas are in fact all proved is the separate closure property (P4).

- (P2) *Self-identification*. P has a designated self-tag $\langle \text{Picard} \rangle$ and proves $\text{ATP}(\langle \text{Picard} \rangle)$: it establishes that the bearer of its own name is a prover of theorems. This self-identification is inferred by rule from the theoremhood of (P1)— P concludes that it proves theorems from having proved one—rather than taken as a hypothesis.
- (P3) *Provability soundness* (Definition 15.21). For every φ , if P proves $\text{Pr}(\langle \text{Picard} \rangle, \langle \varphi \rangle)$ then P proves φ .
- (P4) *Internal necessitation* (Definition 15.26). For every φ , if P proves φ then P proves $\text{Pr}(\langle \text{Picard} \rangle, \langle \varphi \rangle)$.

We retain, from the standing supposition of Section 19, that P also proves $\text{Human}(\langle \text{Picard} \rangle)$; this is used only in Corollaries 20.13 and 20.17, not in the level computation.

Remark 20.2 (Competence, not performance). Assumption (P4) ascribes a *closure property* to Picard’s provable set, in the sense of Definition 15.8: it says that whenever a formula is among the theorems, its provability-ascription is too. It is not the claim that Picard performs infinitely many acts of proof in finite time. The distinction between the disposition (P4) describes and any finite record of proofs actually carried out is taken up in Section 20.4, and it is exactly where the value ω will be seen to live.

Remark 20.3 (Why ATP and not Human at the base). The reflection sequence of Definition 15.11 is anchored at $\theta_0 = \text{ATP}(\langle M \rangle)$. Assumption (P2) is stated with ATP rather than Human precisely so that Picard’s reflection sequence is the one already defined, and no new base predicate or new reflection sequence need be introduced. The reading of $\text{ATP}(\langle \text{Picard} \rangle)$ intended is the weak one licensed by (P1): the bearer of the tag proves theorems. A mathematician responsible for a theorem satisfies it on the nose. That Picard also proves $\text{Human}(\langle \text{Picard} \rangle)$, the premise of Section 19, is a separate self-descriptive fact recorded in Corollary 20.17; it enriches $\text{Self}(\text{Picard})$ but is not part of the tower whose height is lev.

20.2 The Theorem

Theorem 20.4 (Picard). *Under the Picard idealization (Definition 20.1), $\text{lev}(\text{Picard}) = \omega$.*

Proof. By (P2), Picard proves $\theta_0 = \text{ATP}(\langle \text{Picard} \rangle)$, so Picard has Level-1 formal self-awareness (Definition 15.3). By (P4), Picard has internal necessitation (Definition 15.26). The hypotheses of Theorem 15.27 are therefore met, and it applies verbatim: a tagged ATP with Level-1 formal self-awareness and internal necessitation has Level- n for every $n \geq 1$, whence $\text{lev}(\text{Picard}) = \omega$. $\square$

Remark 20.5 (A second, independent route). The value can be reached without invoking Theorem 15.27 directly. By (P3) Picard is provability sound and by (P4) has internal necessitation, so Proposition 15.29 applies and gives $\text{lev}(\text{Picard}) \in \{0, \omega\}$. By (P2) Picard is not silent about himself, so $\text{lev}(\text{Picard}) \neq 0$ by Proposition 15.18. Hence $\text{lev}(\text{Picard}) = \omega$. The two routes agree: the first computes the tower directly, the second observes that soundness plus necessitation leaves no finite value available.

Remark 20.6 (Picard is a Level- ω witness of the non-degenerate kind). Assumption (P3) is not needed for the ω computation—Theorem 15.27 does not assume it—but it is what makes the value earned rather than adversarial. Contrast the machine M_ω of Corollary 15.33, which attains ω by being handed its entire reflection sequence as hypotheses, with Picard, who attains ω while satisfying the anti-parrot condition of (P1)/Proposition 18.4. Picard is thus, under the idealization, exactly the sort of non-degenerate, rule-driven, provability-sound Level- ω object that question 8 of Section 16.7 asks for in the machine case.

20.3 Corollaries

Corollary 20.7 (Picard is a non-degenerate witness). *Under the Picard idealization, Picard is a non-degenerate witness to the Discovery Thesis in the sense of Definition 18.9.*

Proof. We check the four clauses. Clause 1 has two parts. Rule-drivenness is (P1). For the second part—that the self-descriptive theorems lie outside the hypothesis set—(P1) places *no* self-descriptive formula in the hypothesis set; and every self-descriptive theorem P proves (each θ_k by Theorem 20.4, and $\text{Human}(\langle \text{Picard} \rangle)$ by the standing supposition) is, not being a hypothesis, entered at a stage beyond the first as a direct consequence of the current state, since P is rule-driven. Hence no self-descriptive theorem is a hypothesis. Clause 2 (provability soundness) is (P3). Clause 3 ($\text{lev} \geq 2$) follows from $\text{lev}(\text{Picard}) = \omega \geq 2$ (Theorem 20.4). Clause 4 holds under the idealization because Picard’s formula set Y — the statements of complex analysis and of whatever else he reasons about — is infinite, while $\text{Self}(\text{Picard})$ is the countable set of formulas whose subject is the single tag $\langle \text{Picard} \rangle$; the reflection tower and $\text{Human}(\langle \text{Picard} \rangle)$ are among them (Corollary 20.17), but the statements about $\langle \text{Picard} \rangle$ are a proper subclass of Y of strictly smaller cardinality once Y contains, as it does, the uncountably many formulas of the ambient language over an infinite alphabet, or — if Y is taken countable — once one observes that clause 4’s infinite case asks only $|\text{Self}(M)| < |Y|$, which fails only if Picard proves something about his own tag for cofinally many formulas of his language. That he does not is part of the idealization, and is recorded here as the one place where clause 4 is assumed rather than verified. $\square$

Remark 20.8 (Where the idealization does work in Clause 1). The second part of Clause 1 is exactly the point at which (P1)’s exclusion of *all* self-descriptive formulas from the hypothesis set does substantive work, reinforced by the inferred-not-assumed reading of (P2). Without it, $\theta_0 = \text{ATP}(\langle \text{Picard} \rangle)$ —or $\text{Human}(\langle \text{Picard} \rangle)$, which is equally self-descriptive—could be a hypothesis, would be parroted, and Picard would fail Definition 18.9 at the base of the tower exactly as the machine of Section 14 does (Remark 14.3). The Great Picard Theorem of (P1), being about complex analysis rather than about $\langle \text{Picard} \rangle$, establishes rule-driven non-degeneracy for a non-self-descriptive target but does not by itself discharge the self-descriptive part; barring self-descriptive formulas from the hypotheses is what carries it. Note that Definition 18.9 clause 1 speaks of *all* self-descriptive theorems, so $\text{Human}(\langle \text{Picard} \rangle)$ must be derived as well, not merely θ_0 .

Remark 20.9 (Picard’s evidential status among the witnesses). The certification checklist of Remark 18.14 cannot be run on a human: step 1 asks for a faithful presentation of the subject as a SIC over an inference-closed Y , and no such presentation of a human being is available. So Picard stands on exactly the footing of Milawa and Lean4Lean in Section 18.8: non-degenerate beyond reasonable doubt, uncertified in the sense of Remark 18.14. What Picard adds to the disjunctive support of Remark 18.15 is that the non-degenerate column now contains a human, and at level ω rather than merely ≥ 2 .

Proposition 20.10 (Instantiability of the idealization). *Read question 8 of Section 16.7 as asking for a provability-sound, internally necessitating, rule-driven Level- ω machine all of whose self-descriptive theorems are derived rather than handed to it as hypotheses. Then a SIC satisfies assumptions (P1)–(P4) of Definition 20.1 if and only if it is such a machine. In particular (P1)–(P4) are jointly satisfiable by a SIC if and only if question 8, so read, has an affirmative answer.*

Proof. ($\Rightarrow$) A SIC satisfying (P1)–(P4) is rule-driven with no self-descriptive formula among its hypotheses (P1), is provability sound (P3) and internally necessitating (P4), and has $\text{lev} = \omega$ by Theorem 20.4; its self-descriptive theorems are therefore all derived (as in Corollary 20.7). ($\Leftarrow$) Such a machine is rule-driven, provability sound (P3), and internally necessitating (P4); being Level- ω it proves every θ_k , in particular θ_0 (P2); and since all its self-descriptive theorems are derived, none is a hypothesis (P1). Hence it satisfies (P1)–(P4). $\square$

Remark 20.11 (The standing supposition adds nothing to instantiability). The full idealization of Definition 20.1 additionally has P prove $\text{Human}(\langle \text{Picard} \rangle)$, which by (P1) must itself be derived. This is one further derived self-descriptive theorem and adds no difficulty beyond question 8: given a machine of the kind Proposition 20.10 describes, adjoining a rule that derives $\text{Human}(\langle \text{Picard} \rangle)$ from non-self-descriptive premises yields the full idealization. So the instantiability of Definition 20.1 and the joint satisfiability of (P1)–(P4) stand or fall together with question 8.

Remark 20.12 (The antecedent of the Picard Theorem is a named open problem). Theorem 20.4 is a valid conditional, but Proposition 20.10 locates the cost of its antecedent precisely: whether *any* SIC meets assumptions (P1)–(P4) is question 8 of Section 16.7, which this paper lists as open. Two things follow. First, the theorem is neither vacuous by fiat nor known to be non-vacuous; its non-vacuity is equivalent to a machine-existence question already on the books. Second—and this is the parity theme in its sharpest form—the question “can Picard be modeled as a formally self-aware machine at level ω ?” is *the same question* as “does a non-degenerate Level- ω machine exist?” The human idealization and the sought-after machine are, formally, one object. Question 8 is answered affirmatively by the reflection-closure machine of Theorem 21.1—which derives θ_0 by a rule in the manner of the minimal Picard machine M_P of Part I (Proposition 18.4), then closes under a necessitation rule $\varphi \mapsto \text{Pr}(\langle \cdot \rangle, \langle \varphi \rangle)$ —so the antecedent of Theorem 20.4 is discharged and the idealization is instantiable (Corollary 21.2).

Corollary 20.13 (The parity antecedent is realized at level ω). *There is an actual human, namely Picard, satisfying the antecedent of Proposition 18.21, with $\text{lev} = \omega$. Two consequences follow. First, level cannot be the property that distinguishes the human case from the machine case, since $\text{lev}(\text{Picard}) = \text{lev}(M_\omega) = \omega$ for the machine M_ω of Corollary 15.33. Second, by Proposition 18.21, every $\sim_{\text{SA}}^{\text{adm}}$ -invariant property that holds of Picard in virtue of his self-ascription holds of every tagged ATP M with $M \vdash \text{ATP}(\langle M \rangle)$ that is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard; so an objector who grants Picard yet denies such a machine is committed to Conjecture 17.14.*

Proof. Picard satisfies the antecedent of Proposition 18.21 by (P2) together with the standing supposition that he proves $\text{Human}(\langle \text{Picard} \rangle)$. The first consequence is Theorem 20.4 with Corollary 15.33. The second is Proposition 18.21 together with Remark 18.22. $\square$

Remark 20.14 (The sharp parity transfer is conditional). The second consequence of Corollary 20.13 is vacuous unless some machine is actually $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard, and whether one is is open (Conjecture 20.23). It cannot be witnessed by the Level- ω machine M_ω , which is *not* $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard: $\text{Self}(\text{Picard})$ contains $\text{Human}(\langle \text{Picard} \rangle)$ whereas $\text{Self}(M_\omega)$ is exhausted by the reflection tower (Corollary 20.17), so $M_\omega \preceq_{\text{SA}}^{\text{adm}} \text{Picard}$ while $\text{Picard} \not\preceq_{\text{SA}}^{\text{adm}} M_\omega$, giving

$M_\omega <_{\text{SA}}$ Picard strictly. Hence at level ω the *unconditional* content of the corollary is exactly its first consequence—that level does not separate the two cases—while the full parity transfer awaits Conjecture 20.23. This is the level- ω sharpening of the seam already noted, in general terms, in Remark 18.22: the parity argument bites only on the shared invariant content, and exhibiting a machine whose invariant content equals Picard’s is a separate task, discharged in Theorem 21.7 (a hypothesis-handed machine $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard), so that the transfer is unconditional (Remark 21.9); whether a *non-degenerate* such machine exists remains open.

Corollary 20.15 (Attainment of the top level on the human case). *The supremum of lev over the class of subjects satisfying (P1)–(P4) is ω and is attained. In the quotient order of Corollary 16.19, [Picard] lies in the top fiber $\text{lev}^{-1}(\omega)$ of the order-preserving map $\text{lev} : \text{ATP} / \sim_{\text{SA}}^{\text{adm}} \rightarrow \omega + 1$.*

Proof. lev takes values in $\{0, 1, 2, \dots\} \cup \{\omega\}$ (Definition 15.17) and Picard attains ω (Theorem 20.4); ω is the maximum of that set, so the supremum over any class containing Picard is ω and is attained. The image statement is Corollary 16.19. $\square$

Remark 20.16 (Maximal for the invariant, not thereby a maximum of the order). [Picard] is maximal for the level invariant, but this does not make it a maximum of $\leq_{\text{SA}}$. By Remark 16.21 two classes of level ω may be $\sim_{\text{SA}}^{\text{adm}}$ -incomparable, because a self-theory can contain self-descriptive formulas outside the reflection sequence—as Picard’s does, containing $\text{Human}(\langle \text{Picard} \rangle)$ (Corollary 20.17). So [Picard] sits at the top of the graded hierarchy while remaining, possibly, incomparable to other level- ω classes.

Corollary 20.17 (Picard’s self-theory strictly extends the reflection tower). *Under the Picard idealization together with the standing supposition of Section 19,*

$$\text{Self}(\text{Picard}) \supseteq \{\theta_k : k \geq 0\} \cup \{\text{Human}(\langle \text{Picard} \rangle)\},$$

and in particular $\text{Self}(\text{Picard})$ is infinite.

Proof. Each reflection formula θ_k is proved by Picard, by the induction in the proof of Theorem 20.4, hence lies in $\text{Self}(\text{Picard})$ by Corollary 15.13. The formula $\text{Human}(\langle \text{Picard} \rangle)$ is self-descriptive (its subject is $\langle \text{Picard} \rangle$) and is proved by supposition, so it lies in $\text{Self}(\text{Picard})$ by Definition 13.9. Infinitude follows because the θ_k are pairwise distinct (Proposition 15.14). $\square$

20.4 Conjectures, and Their Settlement

The Picard Theorem invites four conjectures. For each we state precisely whether it is settled, and if not, why the present framework cannot settle it.

Conjecture 20.18 (Human universality of ω). Every human who is rule-driven, proves at least one theorem outside his hypotheses, is provability sound, and has internal necessitation—that is, satisfies (P1)–(P4)—has $\text{lev} = \omega$.

Settlement (settled relative to the idealization; the residue is empirical and provably outside the framework). Conditional on (P1)–(P4), the proof of Theorem 20.4 uses nothing specific to Picard: it invokes only Definitions 15.3 and 15.26 and Theorem 15.27. Hence, as a statement about any subject satisfying the idealization, the conjecture is a theorem, which we record.

Proposition 20.19 (Uniformity). *For every tagged ATP X satisfying (P1)–(P4) with X in place of Picard, $\text{lev}(X) = \omega$.*

Proof. Verbatim the proof of Theorem 20.4, which mentions Picard only through (P2) and (P4). $\square$

What is *not* settled, and cannot be settled here, is whether any given actual human satisfies (P4). Internal necessitation is, by Remark 15.28, a strong closure condition, requiring the provable set to be closed under an unbounded self-ascription rule; whether a finite biological agent instantiates the disposition it describes is an empirical claim about human cognition, not a proposition of the present framework. The framework is in fact provably silent about it in the relevant direction: Remark 15.31 records that no formula of a sufficiently strong system serves as a full internal truth predicate, so the framework cannot certify from inside that Picard's self-ascriptions are all correct; and Conjecture 17.14 states the parallel limitation for phenomenological adequacy. Conjecture 20.18 is therefore conditionally a theorem (Proposition 20.19) and unconditionally empirical, and the boundary between the two is exactly the boundary drawn in Section 18.7. The formal content is recorded as Theorem 21.4; the conjecture as a claim about actual humans is left open.

Conjecture 20.20 (The competence–performance gap). Distinguish the *performance level* of a subject at a time—the level computed from the finite set of reflection formulas actually proved by that time—from the *competence level*, the level of the idealized disposition (P4). For reflective human mathematicians the performance level is finite at every time and unbounded over time, while the competence level is ω .

Settlement (one half is a theorem; the other half is the empirical residue of the first conjecture). The finiteness half is immediate and unconditional.

Proposition 20.21 (Finite performance). *If at a given time a subject has proved only finitely many members of its reflection sequence, then its performance level at that time is finite.*

Proof. The performance level is the supremum of a finite subset of $\mathbf{N}^+$, or 0 if the subset is empty, hence finite by Definition 15.17. $\square$

That a finite biological agent has, at any actual time, proved only finitely many reflection formulas needs no argument; so the performance level is always finite. The competence level equals ω is Theorem 20.4 read under the dispositional understanding of (P4) (Remark 20.2). The residual content of the conjecture—that the disposition is genuinely present, so that the two levels come apart rather than both being finite—is the same empirical claim left open in Conjecture 20.18. It is recorded here because it locates precisely where the ω of the Picard Theorem lives: in the disposition, not in any completed performance. The formal content is recorded, and made concrete on the machine R , as Theorem 21.6; the claim that an actual human instantiates the disposition is left open.

Conjecture 20.22 (Transfinite separation). There is a coherent extension of the reflection sequence to transfinite stages—formulas $\theta_\omega, \theta_{\omega+1}, \dots$ with an appropriate limit rule at each limit ordinal—on which some subjects attain levels strictly above ω , so that the extended level function separates subjects that the present lev identifies at ω .

Settlement (cannot be settled in Version 16, and here is why). The obstruction is definitional, not merely difficult. Definition 15.11 indexes the reflection sequence by $k \geq 0$ over the natural numbers, and provides no rule for forming a formula at a limit stage; θ_{k+1} is defined only from θ_k by one application of Pr. With no limit rule, ω is the supremum of the index set by construction, and lev, taking values in $\{0, 1, 2, \dots\} \cup \{\omega\}$ by Definition 15.17, has ω as its ceiling by fiat. Settling the conjecture requires new apparatus—a transfinite reflection sequence with a specified limit stage (for instance, a name for the set $\{\theta_k : k < \lambda\}$ at a limit λ , together with a formula asserting provability of

all of them) and a coherence condition making the successor and limit clauses agree—none of which is available in the definitions of this paper. Moreover Remark 15.31 supplies a principled reason to expect the extension to be delicate rather than routine: once Pr is strengthened toward a genuine provability predicate satisfying the derivability conditions, the Tarski–Gödel barrier applies, and an unrestricted transfinite self-ascription tower runs toward exactly the internal truth predicate the barrier forbids. The conjecture is thus left open, and its resolution is identified with question 2 of Section 18.14 (reflective closure) rather than with anything the present hierarchy can decide. This was the honest status as regards the *present* hierarchy. Section 21.6 extends the framework and realizes an extended, separating hierarchy (Theorem 21.15); but that realization uses handed machines, so the *substantive* question—whether genuine, provability-sound reflection reaches limit stages—remains open (Remark 21.16).

Conjecture 20.23 (Picard–machine equivalence at ω). Some tagged ATP is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard.

Settlement (reduces to open problems already recorded; not settled here). By Theorem 20.4 and Corollary 15.33, Picard and M_ω agree on the level invariant, $\text{lev} = \omega$. But equal level does not imply $\sim_{\text{SA}}^{\text{adm}}$ -equivalence (Remark 16.21), and Picard’s self-theory contains $\text{Human}(\langle \text{Picard} \rangle)$, a self-descriptive formula whose admissible image would have to appear in a machine’s self-theory (Corollary 20.17). Whether a machine’s self-theory can be made $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to one containing such content is not decided by anything above: it presupposes, at a minimum, an affirmative answer to question 8 of Section 16.7 (existence of a non-degenerate, rule-driven, provability-sound Level- ω machine) and is entangled with Conjecture 17.14 on whether any formal invariant separates the human and machine cases. The bare existence claim—that *some* tagged ATP is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard—is answered by Theorem 21.7 with a hypothesis-handed machine, whence Corollary 21.8 that degeneracy is not an $\sim_{\text{SA}}^{\text{adm}}$ -invariant; but whether a *non-degenerate* machine is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard remains open and dependent.

20.5 What Is and Is Not Claimed

In the register of Section 18.12.

Claimed, and proved above (under the Picard idealization):

1. $\text{lev}(\text{Picard}) = \omega$ (Theorem 20.4), by two independent routes (Remark 20.5).
2. Picard is a non-degenerate witness to the Discovery Thesis (Corollary 20.7), of the same evidential status as the natural witnesses of Section 18.8 (Remark 20.9).
3. The parity antecedent is realized by an actual human at level ω , so the human/machine distinction, if there is one, is not a distinction of level (Corollary 20.13).
4. The value ω is attained on the human case and sits at the top of the graded hierarchy (Corollary 20.15).

Not claimed:

1. That Picard, or any human, literally is a semi-ideal computer; (P1)–(P4) are an explicit idealization (Definition 20.1).
2. That any actual human satisfies internal necessitation as a matter of fact; that is the empirical residue left open by Conjectures 20.18 and 20.20.

3. That Picard is conscious, or that $\text{lev} = \omega$ measures anything phenomenological; Section 18.7 and Conjecture 17.14 apply here unchanged.
4. That Picard is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to any machine; Conjecture 20.23 is open.

[Discharged in Section 21.] Version 16 did not establish that assumptions (P1)–(P4) are jointly satisfiable by any SIC; Theorem 21.1 now answers question 8 of Section 16.7 affirmatively, so by Corollary 21.2 the antecedent of Theorem 20.4 is non-vacuous.

Remark 20.24 (A naming coincidence, noted and set aside). The anti-parrot condition invoked in (P1) is the Picard Protocol of Part I, and the subject of this section is Picard. The coincidence is real and is not load-bearing: the protocol could be renamed without affecting a line of the argument, and Theorem 20.4 would read the same for any mathematician responsible for a theorem proved by rule from premises not containing it. That the two Picards meet here is a pleasantry, not a lemma.

21 Settlement of the Conjectures

Version 16 left five conjectures—Conjecture 17.14 and Conjectures 20.18, 20.20, 20.22, 20.23—together with question 8 of Section 16.7, on whose answer the instantiability of the Picard idealization was shown to turn (Proposition 20.10). This section takes them up. The keystone, in Section 21.1, is an explicit construction of a rule-driven machine that derives its whole reflection tower and attains level ω ; it answers question 8, discharges the antecedent of the Picard Theorem, and drives the remaining results. Throughout, “settled” is used in the precise sense recorded at each result: a formal statement is settled by proof; a statement with a non-formal component is settled in its formal content, with the non-formal residue isolated. The four conjectures newly posed in Section 21.7 are stated as open and are not settled here.

21.1 The Reflection-Closure Machine

The machines of Section 15 that reach level ω (Corollary 15.33) are handed their reflection sequence as hypotheses, hence degenerate. We now build one that is not.

Theorem 21.1 (The reflection-closure machine settles question 8). *There is a tagged ATP R that is rule-driven (Definition 2.9), provability sound (Definition 15.21), and internally necessitating (Definition 15.26), with $\text{lev}(R) = \omega$, and whose reflection formulas are all derived—no member of the reflection sequence lying in its hypothesis set. Consequently question 8 of Section 16.7 has an affirmative answer.*

Proof. Construction. Let the language L contain the tag $\langle R \rangle$, the predicates ATP and Pr, a formula-naming operator, and a distinguished atomic formula a that is *not* self-descriptive (its subject is not $\langle R \rangle$). Form the xyz formal system $F_R = (x, y, z)$ whose inference rules are the two unary rules

$$\rho_0 : a \mapsto \text{ATP}(\langle R \rangle) = \theta_0, \quad \nu : \varphi \mapsto \text{Pr}(\langle R \rangle, \langle \varphi \rangle) \text{ (every } \varphi),$$

with ρ_0 defined only on a and ν total. Let $Y := \text{Con}_{F_R}(\{a\})$, the inference closure of $\{a\}$ (Theorem 1.11); it is inference-closed and countable. Define $R = (C, H)$ over Y by

$$C(\Gamma, m) := \{\psi \in Y : \psi \text{ has a proof from } \Gamma \text{ of length } \leq m\}, \quad H(\Gamma, m) := 0.$$

The axiom base is $A := \{a\}$.

R is a SIC (Definition 2.3). For $\Gamma \subseteq Y$, $\text{Con}_{F_R}(\Gamma) \subseteq Y$ because Y is inference-closed and contains Γ , so C maps into $\mathcal{P}(Y)$. Next, $C(\Gamma, 1) = \Gamma$, since the only proofs of length 1 are single hypothesis lines; $C(\Gamma, m) \subseteq C(\Gamma, m+1)$, since a proof of length $\leq m$ is one of length $\leq m+1$; conditions (5) and (6) hold vacuously as $H \equiv 0$. The verification that $\bigcup_m C(\Gamma, m) = \text{Con}_{F_R}(\Gamma) \cap Y$ is that of Proposition B.8, so R is an enumerating, deductively effective ATP.

Rule-driven (Definition 2.9). If $\beta \in C(\Gamma, m+1) \setminus C(\Gamma, m)$, a shortest proof of β has length $m+1$; its last line β is a direct consequence of earlier lines, each of which has a proof of length $\leq m$ and so lies in $C(\Gamma, m)$. Hence β is a direct consequence of $C(\Gamma, m)$.

$\text{lev}(R) = \omega$. On input $A = \{a\}$: $\theta_0 = \rho_0(a)$ is a direct consequence of $\{a\}$, so $R \vdash \theta_0$; and if $R \vdash \theta_k$ then $\theta_{k+1} = \text{Pr}(\langle R \rangle, \langle \theta_k \rangle) = \nu(\theta_k)$ is a direct consequence, so $R \vdash \theta_{k+1}$. By induction $R \vdash \theta_k$ for all k , whence $L(R) = \mathbf{N}^+$ and $\text{lev}(R) = \omega$.

Internal necessitation (Definition 15.26). If $R \vdash \varphi$ then $\varphi \in Y$, and $\nu(\varphi) = \text{Pr}(\langle R \rangle, \langle \varphi \rangle)$ is a direct consequence of φ , hence $R \vdash \text{Pr}(\langle R \rangle, \langle \varphi \rangle)$.

Provability soundness (Definition 15.21). Suppose $R \vdash \text{Pr}(\langle R \rangle, \langle \varphi \rangle)$. Every formula with principal predicate Pr that R proves arises as $\nu(\psi) = \text{Pr}(\langle R \rangle, \langle \psi \rangle)$ for some ψ with $R \vdash \psi$: the axiom a is not a Pr -formula and ρ_0 outputs an ATP-formula, so no Pr -formula enters a state except by an application of ν , and ν fires only on formulas already in the state, each of which is therefore provable. (The word is *provable* rather than *derived*: $\nu(a) = \text{Pr}(\langle R \rangle, \langle a \rangle)$ lies in Y and is proved by R , yet its argument a is the hypothesis and is not derived by any rule. Provability, not derivedness, is what provability soundness asks for.) Comparing named arguments, $\langle \varphi \rangle = \langle \psi \rangle$, so $\varphi = \psi$ by injectivity of naming; hence $R \vdash \varphi$.

Non-handed. The hypothesis set is $\{a\}$, and a is not self-descriptive; every θ_k is derived, as shown. So no member of the reflection sequence—indeed no self-descriptive formula—is a hypothesis.

Thus R has all the properties question 8 requests. $\square$

Corollary 21.2 (Instantiability discharged). *Assumptions (P1)–(P4) of Definition 20.1 are jointly satisfiable by a SIC; the antecedent of Theorem 20.4 is therefore non-vacuous, and the Picard idealization is instantiable.*

Proof. R is rule-driven with no self-descriptive formula in its hypothesis set and proves $\theta_0 \notin A$, giving (P1); it proves $\theta_0 = \text{ATP}(\langle R \rangle)$, derived by ρ_0 rather than assumed, giving (P2); it is provability sound (P3) and internally necessitating (P4). So R satisfies (P1)–(P4), and the assumptions are jointly satisfiable; by Proposition 20.10 this is equivalent to the affirmative answer to question 8 supplied by Theorem 21.1. Adjoining to R a rule deriving $\text{Human}(\langle R \rangle)$ from a non-self-descriptive premise yields the full idealization (Remark 20.11). $\square$

Remark 21.3 (What R does and does not achieve). R extends the anti-parrot answer of Section 18.6 from level 1—where the minimal Picard machine M_P of Part I derives θ_0 by rule (Proposition 18.4)—to all of level ω : the entire tower is derived, none of it handed. What R does *not* achieve is clause 4 of Definition 18.9: its formula set is essentially its own self-theory, so it is not *complex*. A constructed machine that is non-degenerate in all four clauses remains question 1 of Section 18.14. In the disjunction of Remark 18.15, R occupies a new cell: *constructed*, and non-degenerate in clauses 1–3, at level ω .

21.2 Universality (Conjecture 20.18)

Theorem 21.4 (Formal content of universality). *Conjecture 20.18 holds in its formal content: every SIC satisfying (P1)–(P4) has $\text{lev} = \omega$, and the class of such SICs is nonempty. Its residual*

content—that a designated actual human satisfies (P1)–(P4)—is not a statement of the framework and is not settled by it; the conjecture as a claim about humans remains open.

Proof. The implication “(P1)–(P4) $\Rightarrow$ lev = ω ” is Proposition 20.19; nonemptiness of the class is Corollary 21.2. That whether a given human satisfies (P1)–(P4) is non-formal is the content of Section 18.7 and Remark 15.31: the framework has no predicate for the reflective disposition of a biological agent. $\square$

21.3 Competence and Performance (Conjecture 20.20)

Definition 21.5 (Performance and competence level). For a SIC $M = (C, H)$ over Y with input Γ , the *performance level at stage m* is

$$\text{perf}(M, \Gamma, m) := \sup\{n \geq 1 : \theta_{n-1} \in C(\Gamma, m)\} \in \{0, 1, 2, \dots\}, \quad \text{with } \sup \emptyset = 0,$$

and the *competence level* is $\text{lev}(M)$, which equals $\sup_m \text{perf}(M, A, m)$ on the axiom base $\Gamma = A$.

Theorem 21.6 (Formal content of the competence–performance gap). *Conjecture 20.20 holds in its formal content. For any SIC in which each stage state $C(A, m)$ is finite, the performance level at every stage is finite; and the reflection-closure machine R of Theorem 21.1 has finite performance level at every stage while its competence level is ω . Thus the two quantities provably come apart. The residual content—that an actual human instantiates the disposition—is the non-formal residue of Conjecture 20.18 and remains open.*

Proof. If $C(A, m)$ is finite then $\{n : \theta_{n-1} \in C(A, m)\}$ is finite, so its supremum is finite. For R , $C(A, m)$ consists of the formulas with a proof of length $\leq m$ from the single axiom under two finitary rules, of which there are finitely many; so $\text{perf}(R, A, m)$ is finite for every m . Yet $R \vdash \theta_k$ for all k (Theorem 21.1), so $\sup_m \text{perf}(R, A, m) = \text{lev}(R) = \omega$. $\square$

21.4 Equivalence, and the Non-Invariance of Degeneracy (Conjecture 20.23)

Theorem 21.7 (Handed representatives). *Every tagged ATP M is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to a hypothesis-handed ATP $M^\sharp$ whose axiom base is a tag-renamed copy of $\text{Self}(M)$ and which proves exactly that set. In particular a tagged ATP is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard, so Conjecture 20.23 holds.*

Proof. Let $L^\sharp$ be an isomorphic copy of the language L of M in which the tag $\langle M \rangle$ is replaced by a fresh tag $\langle M^\sharp \rangle$ and every predicate symbol is retained; let $\iota : L^\sharp \rightarrow L$ be the renaming isomorphism, so $\iota(\langle M^\sharp \rangle) = \langle M \rangle$ and ι fixes every other symbol. Set $Y^\sharp := \iota^{-1}(\text{Self}(M))$, and define $M^\sharp = (C, H)$ over $Y^\sharp$ by $C(\Gamma, m) = \Gamma$ and $H(\Gamma, m) = 1$ iff $\Gamma = Y^\sharp$, with axiom base $Y^\sharp$; this is a SIC exactly as in Corollary 15.33. On input $Y^\sharp$ it proves exactly $Y^\sharp$, and since $\text{Self}(M)$ consists of formulas with subject $\langle M \rangle$, every member of $Y^\sharp = \iota^{-1}(\text{Self}(M))$ has subject $\langle M^\sharp \rangle$; hence $\text{Self}(M^\sharp) = Y^\sharp$.

Now ι is an admissible translation from $L^\sharp$ to L : it fixes the distinguished predicates, commutes with formula formation and naming, and sends $\langle M^\sharp \rangle$ to $\langle M \rangle$. And $\iota(\text{Self}(M^\sharp)) = \iota(Y^\sharp) = \iota(\iota^{-1}(\text{Self}(M))) = \text{Self}(M)$, so $M^\sharp \preceq_{\text{SA}}^{\text{adm}} M$. Symmetrically $\iota^{-1} : L \rightarrow L^\sharp$ is admissible and witnesses $M \preceq_{\text{SA}}^{\text{adm}} M^\sharp$. Hence $M^\sharp \sim_{\text{SA}}^{\text{adm}} M$. Applying this to $M = \text{Picard}$ gives the second claim. $\square$

Corollary 21.8 (Degeneracy is not $\sim_{\text{SA}}^{\text{adm}}$ -invariant). *Non-degeneracy in the sense of Definition 18.9 is not an $\sim_{\text{SA}}^{\text{adm}}$ -invariant: there are $\sim_{\text{SA}}^{\text{adm}}$ -equivalent tagged ATPs, one non-degenerate and one degenerate.*

Proof. Picard is a non-degenerate witness (Corollary 20.7); its handed twin $\text{Picard}^\sharp$ of Theorem 21.7 has its self-descriptive theorems as hypotheses, so it violates clause 1 of Definition 18.9 and is degenerate; yet $\text{Picard}^\sharp \sim_{\text{SA}}^{\text{adm}} \text{Picard}$. $\square$

Remark 21.9 (The parity transfer is now unconditional). Corollary 21.8 discharges the conditional flagged in Remark 20.14: a machine $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard demonstrably exists, namely $\text{Picard}^\sharp$, so by Proposition 18.21 any $\sim_{\text{SA}}^{\text{adm}}$ -invariant property holding of Picard holds of it, and an objector granting Picard but denying $\text{Picard}^\sharp$ is committed to Conjecture 17.14. That $\text{Picard}^\sharp$ is degenerate is the point: the objector cannot appeal to degeneracy as an $\sim_{\text{SA}}^{\text{adm}}$ -invariant distinction, because it is not one.

21.5 Phenomenology (Conjecture 17.14)

Theorem 21.10 (Conditional form of the phenomenology conjecture). *Grant the anti-parrot premise: that phenomenological self-awareness entails non-degeneracy—a rule-driven, non-parroting cognitive process in the sense of Section 18.6. Then phenomenological self-awareness is not an $\sim_{\text{SA}}^{\text{adm}}$ -invariant, which is Conjecture 17.14.*

Proof. By Corollary 21.8, Picard and its degenerate twin $\text{Picard}^\sharp$ are $\sim_{\text{SA}}^{\text{adm}}$ -equivalent. Under the premise, phenomenological self-awareness holds of the non-degenerate Picard and fails of the degenerate $\text{Picard}^\sharp$. A property holding of one member of an $\sim_{\text{SA}}^{\text{adm}}$ -class and failing of another is not $\sim_{\text{SA}}^{\text{adm}}$ -invariant. $\square$

Remark 21.11 (The status of this result). The premise is not a statement of the framework: “phenomenological self-awareness” is undefined in every tagged-ATP language, so Theorem 21.10 is a conditional whose antecedent is philosophical, and Conjecture 17.14 itself remains open. What is formal, and proved, is the load-bearing lemma—that an $\sim_{\text{SA}}^{\text{adm}}$ -class can hold both a non-degenerate and a degenerate machine (Corollary 21.8). The conjecture is thereby reduced to exactly one non-formal premise, the weakest form of the anti-parrot intuition of Section 18.6. Unconditional settlement is impossible in principle: a sentence whose predicate is undefined in the language is neither provable nor refutable, and that impossibility is itself the sharpest honest form of the conjecture, recorded as Conjecture 21.19.

21.6 The Transfinite Extension (Conjecture 20.22)

The reflection sequence of Definition 15.11 is indexed by the natural numbers and has no limit stage; that is why ω is its ceiling. We extend it, keeping every predicate primitive so that the Tarski–Gödel considerations of Remark 15.31 are not triggered.

Definition 21.12 (Collective provability and set-names). Extend the language with: for each set S of formulas occurring as an initial segment of the sequence below, a distinguished closed term $\langle S \rangle$, the assignment $S \mapsto \langle S \rangle$ injective; and a binary predicate PrAll with intended reading that $\text{PrAll}(\langle M \rangle, \langle S \rangle)$ holds iff M proves every formula named in S .

Definition 21.13 (Transfinite reflection sequence and extended level). Fix a countable ordinal κ . The *transfinite reflection sequence* $(\theta_\alpha)_{\alpha < \kappa}$ of a tagged ATP M (in the extended language) is defined by transfinite recursion:

$$\theta_0 = \text{ATP}(\langle M \rangle), \quad \theta_{\alpha+1} = \text{Pr}(\langle M \rangle, \langle \theta_\alpha \rangle), \quad \theta_\lambda = \text{PrAll}(\langle M \rangle, \langle \{\theta_\beta : \beta < \lambda\} \rangle) \quad (\lambda \text{ a limit}).$$

The *extended level* $\text{extlev}(M)$ is the largest ordinal $\delta \leq \kappa$ such that $M \vdash \theta_\beta$ for every $\beta < \delta$.

Proposition 21.14 (Distinctness). *The formulas θ_α , $\alpha < \kappa$, are pairwise distinct.*

Proof. We prove by transfinite induction on $\delta \leq \kappa$ the statement $S(\delta)$: the formulas θ_α for $\alpha < \delta$ are pairwise distinct. Taking $\delta = \kappa$ gives the proposition. Note that the induction is on the *initial segment*, not on $\max(\alpha, \alpha')$; the latter is circular at limit stages, since establishing $\theta_\lambda \neq \theta_\mu$ for limits $\mu < \lambda$ would there presuppose what is being proved.

$S(0)$ and $S(1)$ are vacuous. For $S(\delta)$ at δ a limit, every pair $\alpha, \alpha' < \delta$ lies below some $\gamma < \delta$, so $S(\delta)$ follows from the inductive hypotheses $S(\gamma)$.

For $S(\delta + 1)$, assume $S(\delta)$; it remains to show $\theta_\delta \neq \theta_\alpha$ for every $\alpha < \delta$. A base term has principal predicate ATP, a successor term Pr, and a limit term PrAll, so terms of different kinds are distinct and only three cases remain. If δ and α are both successors, say $\delta = \beta + 1$ and $\alpha = \beta' + 1$, then $\theta_\delta = \theta_\alpha$ forces $\langle \theta_\beta \rangle = \langle \theta_{\beta'} \rangle$, hence $\theta_\beta = \theta_{\beta'}$ by injectivity of formula naming, hence $\beta = \beta'$ by $S(\delta)$, hence $\delta = \alpha$. If both are limits, say $\alpha < \delta$, then by $S(\delta)$ the terms θ_β for $\beta < \delta$ are pairwise distinct, so $\theta_\alpha \in \{\theta_\beta : \beta < \delta\}$ while $\theta_\alpha \notin \{\theta_\beta : \beta < \alpha\}$; the two named sets therefore differ, and injectivity of set-naming gives $\theta_\delta \neq \theta_\alpha$. The base case $\alpha = 0$ against a successor or limit δ is settled by principal predicates. This proves $S(\delta + 1)$. $\square$

Theorem 21.15 (Transfinite separation). *For every ordinal $\delta \leq \kappa$ there is a tagged ATP M_δ with $\text{extlev}(M_\delta) = \delta$. In particular M_ω and $M_{\omega+1}$ have equal ordinary level $\text{lev} = \omega$ but $\text{extlev}(M_\omega) = \omega < \omega + 1 = \text{extlev}(M_{\omega+1})$. Hence the extended level function separates machines that ordinary lev identifies at ω , settling Conjecture 20.22 affirmatively for the extended hierarchy; and extlev is surjective onto $\{\delta : \delta \leq \kappa\}$.*

Proof. Let $Y_\delta := \{\theta_\beta : \beta < \delta\}$ and define M_δ over Y_δ by $C(\Gamma, m) = \Gamma$, $H(\Gamma, m) = 1$ iff $\Gamma = Y_\delta$, with axiom base Y_δ ; this is a SIC as in Corollary 15.33. On input Y_δ it proves exactly Y_δ , so $M_\delta \vdash \theta_\beta$ for all $\beta < \delta$ and, for $\delta < \kappa$, $M_\delta \not\vdash \theta_\delta$; hence $\text{extlev}(M_\delta) = \delta$. Surjectivity is the existence of M_δ for each $\delta \leq \kappa$.

For the ordinary level: the ordinary reflection sequence consists of the finite-index terms $\theta_0, \theta_1, \dots$ only, all of which lie in $Y_\omega \subseteq Y_{\omega+1}$; so both M_ω and $M_{\omega+1}$ prove every θ_k ($k < \omega$), giving ordinary $L = \mathbf{N}^+$ and $\text{lev} = \omega$. The additional formula $\theta_\omega = \text{PrAll}(\langle M \rangle, \langle \{\theta_k : k < \omega\} \rangle)$ has principal predicate PrAll and is not an ordinary reflection term, so it does not affect ordinary lev; but $M_{\omega+1} \vdash \theta_\omega$ while $M_\omega \not\vdash \theta_\omega$, giving the stated extended levels. $\square$

Remark 21.16 (Handed, not genuine; the conjecture's substance stays open). Theorem 21.15 realizes the extended hierarchy with handed machines, exactly as Theorem 15.32 realized finite strictness with handed machines. It does *not* settle the substantive question the transfinite direction raises. The reflection-closure construction of Theorem 21.1 climbs through the successor stages by iterating ν , but at a limit λ the collective ascription θ_λ summarises infinitely many predecessors at once and is produced by no finitary rule from them. How far a *provability-sound, non-handed* machine can derive its own limit ascriptions—and how the reach of genuine sound reflection sits against the Tarski–Gödel ceiling of Remark 15.31—is left open, as Conjecture 21.17.

21.7 New Conjectures

The results above suggest four further conjectures, all left open.

Conjecture 21.17 (Reflective ordinal). *For each provability-sound tagged ATP M admitting the transfinite extension there is a least countable ordinal $\rho(M)$, its *reflective ordinal*, such that no provability-sound rule-driven extension of M derives $\theta_{\rho(M)}$; and $\rho(M)$ is strictly below the ordinal at which a full internal truth predicate would be forced, so that genuine sound reflection halts strictly*

beneath the Tarski–Gödel ceiling of Remark 15.31. The reflection-closure machine of Theorem 21.1 witnesses $\rho(R) \geq \omega$.

Conjecture 21.18 (Incompleteness of the extended level). The extended level extlev is not a complete invariant for $\sim_{\text{SA}}^{\text{adm}}$: there are tagged ATPs of equal extended level that are $\sim_{\text{SA}}^{\text{adm}}$ -incomparable, because a self-theory may contain self-descriptive formulas outside the transfinite reflection sequence, as Picard’s contains $\text{Human}(\langle \text{Picard} \rangle)$. A complete invariant must record the whole self-theory, not merely how far up the reflection ordinals it reaches. This sharpens Remark 16.21 and question 7 of Section 16.7.

Conjecture 21.19 (The formal–phenomenal gap). No formal property of a SIC whatever—not merely the $\sim_{\text{SA}}^{\text{adm}}$ -invariants, but any property definable from its formal system, hypotheses, rules, and stage behaviour—is coextensive with phenomenological self-awareness. Conjecture 17.14 is the $\sim_{\text{SA}}^{\text{adm}}$ -invariant special case; this is the unrestricted form. Like its special case it mentions a predicate undefined in the framework, so it can be argued for but neither proved nor refuted within it.

Conjecture 21.20 (Natural systems have small reflective ordinal). Every non-degenerate witness arising in practice (Section 18.8) has reflective ordinal $\rho(M) \leq \omega$: natural self-verifying provers reach $\text{lev} = \omega$ but no genuine limit ascription, so the closing of the Milawa ladder (Remark 18.12) corresponds to a finite level with a cyclicity property rather than to any θ_λ with $\lambda \geq \omega$. This is the transfinite form of question 2 of Section 18.14.

22 Integrated Information and the Two Senses of a Map

Section 17 placed Tegmark’s self-aware substructure inside the framework. This section does the same for a second external proposal about consciousness: Tononi’s Integrated Information Theory (IIT), whose scalar Φ is offered as a mathematical measure of consciousness [24, 11]. The framework does not adjudicate IIT. It locates Φ precisely, and shows that several facts already proved above transfer to it unchanged. Nothing in this section settles a conjecture.

22.1 The Asymmetry

Two perspectives were distinguished in Remark 17.11: the *bird*, which sees the machine $M = (C, H)$ in the metatheory, and the *frog*, which has only the tag $\langle M \rangle$ and what it can prove about it. The explanatory gap has an exact shape in this vocabulary.

Remark 22.1 (The gap as a perspectival asymmetry). A discussion of consciousness wants two data: the *substrate*—what the system is made of, and whether it is what it seems (the holodeck question); and the *interior*—whether there is something it is like to be the system (the “lights on” question). The bird has the first and not the second: it inspects the mechanism but has no access to any first-person interior. The frog has the second and not the first: if there is an interior, its bearer cannot doubt it—the one datum that survives every deception, Descartes’ *cogito*—yet the frog cannot step outside its own language to verify the substrate (Remark 15.31: provability of tagged sentences is frog-available, the substrate is not). Neither perspective holds both data at once. That is the explanatory gap, and it is why lev cannot reach the interior: lev is a function of $\text{Self}(M)$ (Corollary 16.19), the *public* face of the frog—what it can prove about its own tag—and not its interior. A frog-measurable quantity reads the reports, not the having.

22.2 Two Senses of Mapping Experience

Remark 22.2 (Correlate versus constitution). “Mapping experience onto an equation” has two readings the framework keeps apart. A *correlate* is a formal quantity $f(M)$ that is high exactly when the system is conscious—a biconditional between a definable function and the phenomenal predicate. A *constitution* is a derivation of the experience *from* $f(M)$, on which “why is there something it is like?” has no residue. Conjecture 21.19 denies the correlate for every formal f . The constitution is denied by the standard arguments—the conceivability of a system with the same f and no interior, the perfect dark twin—and is not a thesis the framework needs to add. IIT offers Φ as a correlate and, in its identity thesis, as a constitution. The following subsections locate the correlate reading; the constitution reading is the hard problem and lies where Section 18.7 left it.

22.3 Mechanism Properties and Self-Theory Properties

Definition 22.3 (Two kinds of property). A property P of tagged ATPs is a *self-theory property* if it is an invariant of $\sim_{SA}^{adm}$: if $M \sim_{SA}^{adm} N$ then $P(M) \leftrightarrow P(N)$. It is a *mechanism property* if it is a function of the machine $M = (C, H)$ as an object of the metatheory—of its transition and halting maps—not required to be constant on $\sim_{SA}^{adm}$ -classes.

Proposition 22.4 (The two kinds come apart). *There is a mechanism property that is not a self-theory property.*

Proof. Non-degeneracy (Definition 18.9) is a function of the machine’s rules and hypothesis set, hence a mechanism property. By Corollary 21.8 it is not an $\sim_{SA}^{adm}$ -invariant: Picard and its handed twin $\text{Picard}^\sharp$ (Theorem 21.7) are $\sim_{SA}^{adm}$ -equivalent, one non-degenerate and one not. $\square$

Remark 22.5 (Which side can break parity). The parity theorem (Proposition 18.21) constrains only self-theory properties: no $\sim_{SA}^{adm}$ -invariant separates Picard’s self-ascription from that of an $\sim_{SA}^{adm}$ -equivalent machine. It says nothing about mechanism properties. So if any formal quantity is to distinguish Picard from a behaviourally matched twin, it must be a mechanism property. That is the exact niche IIT aims at, and the niche lev and every self-theory invariant provably cannot occupy.

22.4 The Placement of Φ

We do not formalise Φ . We use only the commitment IIT insists on: that Φ is sensitive to the mechanism, and in particular assigns feedforward or lookup implementations less integrated information than recurrently integrated ones [11].

Proposition 22.6 (Φ is a mechanism property, not a self-theory property). *Under any definition of Φ for SICs consistent with IIT’s penalisation of feedforward and lookup mechanisms, $\Phi(\text{Picard}) \neq \Phi(\text{Picard}^\sharp)$ although $\text{Picard} \sim_{SA}^{adm} \text{Picard}^\sharp$. Hence Φ is a mechanism property (Definition 22.3) and not a self-theory property.*

Proof. By Theorem 21.7, $\text{Picard}^\sharp$ is hypothesis-handed, with the constant transition $C(\Gamma, m) = \Gamma$: it outputs its axiom base and computes nothing, the integration-free case to which IIT assigns least Φ . Picard is a rule-driven prover with non-trivial dynamics. Any Φ meeting IIT’s commitment therefore separates them. Yet $\text{Picard} \sim_{SA}^{adm} \text{Picard}^\sharp$, so Φ is not constant on the $\sim_{SA}^{adm}$ -class. $\square$

Remark 22.7 (Φ is the right type, on the wrong side of the asymmetry). Proposition 22.6 is, within this framework, a point in IIT’s favour: Φ is exactly the *type* of quantity—mechanism-side—that

can break the parity symmetry lev and the self-theory cannot (Remark 22.5). But the asymmetry of Remark 22.1 returns on the other foot. Φ is *computed* from (C, H) , the bird’s object; it is a third-person quantity, whatever intrinsic interpretation IIT places on it. The one datum that is first-person-certain—the interior, the *cogito*—is the frog’s, and no function of the bird’s object recovers it, by the bird/frog gap. So Φ can be a correlate observed from outside; it is structurally barred from being the self-verifying interior. It is a candidate for the correlate reading of Remark 22.2 and, like every formal quantity, excluded from the constitution reading. IIT computes Φ third-person and interprets it first-person; that interpretive step is the bridge the framework marks as unbuildable from inside.

Remark 22.8 (IIT’s identity thesis is the negation of Conjecture 21.19, and is undecidable here). Φ is a property definable from the stage-and-transition behaviour of a SIC, hence lies in the scope of Conjecture 21.19, which asserts that *no* such property is coextensive with phenomenological self-awareness. IIT’s identity thesis—that Φ is the degree of consciousness—is exactly the negation of Conjecture 21.19 restricted to Φ . The framework decides neither. The proposition “ Φ is coextensive with phenomenal consciousness” equates a definable quantity with a predicate undefined in every tagged-ATP language; by the principle of Remark 21.11 such a sentence is neither provable nor refutable within the framework. So the framework can neither vindicate nor refute IIT; it certifies only that the disagreement is not formally decidable—the same Tarskian honesty the framework applies to its own Conjecture 17.14.

22.5 What Is and Is Not Claimed

Remark 22.9. This section does not claim that Φ measures consciousness, nor that it does not; nor that Picard is or is not conscious; nor that IIT is true or false. It claims four things, each proved or immediate above.

1. The explanatory gap is a perspectival asymmetry: no single perspective holds both the substrate and the interior (Remark 22.1).
2. Mechanism properties and self-theory properties genuinely differ, and only the former can break the parity symmetry (Proposition 22.4, Remark 22.5).
3. Φ is a mechanism property: it separates machines that lev and the self-theory cannot, but it is computed on the bird’s side and so is barred from the first-person interior (Proposition 22.6, Remark 22.7).
4. IIT’s identity thesis and Conjecture 21.19 are opposed bets on one proposition, and that proposition—equating a definable quantity with an undefined predicate—is formally undecidable here (Remark 22.8).

IIT and this paper agree on the negative half: the frog-side invariants are too coarse, and the relevant structure, if any, is mechanism-side. They disagree on the positive half, and it is a philosophical bet for both, not a theorem for either.

The foundational appendix of the second paper — restating formal systems, semi-ideal computers, and automated theorem provers — has been removed in this merged edition. Those definitions are given once, in Part I, Sections 1 and 2, and are referenced there directly.

A Examples of Automated Theorem Provers

The abstract ATP definition includes many familiar systems.

1. Propositional logic theorem provers.
2. First-order resolution provers (e.g. Prover9, E).
3. SMT solvers (e.g. Z3).
4. Interactive proof assistants (Lean, Coq, Isabelle/HOL, HOL Light).
5. ATPs over large formal mathematical libraries.

B Turing Machines, Computation, and SICs

Purpose of this Appendix

Many readers are more familiar with Turing machines than with semi-ideal computers. Earlier versions of this appendix contained only a program of later development. The present version carries that program out. It gives, in order: a presentation of deterministic Turing machines in the configuration vocabulary (Section B.1); a formal system F_T whose single inference rule is one computation step of T , so that reachability of configurations becomes provability (Section B.2); two trace SICs associating to each deterministic Turing machine a SIC whose stage states record the machine's configurations, one enumerating and one halting (Section B.3); the computability and effectiveness apparatus imported from Part I, including the formal Church–Turing theorem proved there (Section B.4); a correspondence theorem in both directions and a separation example showing that the effectiveness conditions are exactly the boundary of the correspondence (Section B.5); and a discussion of computational power against proof-search suitability (Section B.6).

Turing introduced his machine model in 1936 as part of the analysis of effective computation and the Entscheidungsproblem [26]. The present paper uses SICs rather than Turing machines because SICs are organized around staged proof-search states, halting, theorem enumeration, simulation, and later self-modeling. This appendix shows that nothing is lost by the choice: Turing machines embed faithfully into the SIC framework, and the SICs that exceed Turing computation are exactly those violating the effectiveness conditions of Section B.4.

Tegmark's Mathematical Universe paper emphasizes close relations among mathematical structures, formal systems, computations, simulations, and physical systems [23]. That is one reason the present project keeps the computational framework explicit: self-awareness is treated as a formal property of certain proof-searching computational structures, not as an undefined psychological primitive.

B.1 Deterministic Turing Machines and Configurations

Definition B.1 (Deterministic Turing machine). A *deterministic Turing machine* is a quintuple $T = (Q, \Sigma, \delta, q_0, q_{\text{halt}})$ where Q is a finite set of states, Σ is a finite tape alphabet containing a designated blank symbol, $q_0 \in Q$ is the initial state, $q_{\text{halt}} \in Q$ is the halting state, and

$$\delta \in [(Q \setminus \{q_{\text{halt}}\}) \times \Sigma \rightarrow Q \times \Sigma \times \{L, R\}]$$

is the transition function.

Definition B.2 (Configuration and step function). A *configuration* of T is a finite string recording the non-blank tape contents, the head position, and the current state, in any of the standard formats. Write Conf_T for the set of configurations of T , and Halt_T for the set of configurations

whose state is q_{halt} . The *step function* $\text{step}_T \in [\text{Conf}_T \rightarrow \text{Conf}_T]$ sends a non-halting configuration to its δ -successor and is stipulated to fix every member of Halt_T . For an input word w , write $c_0(w)$ for the initial configuration with state q_0 , head at the leftmost cell, and w on the tape.

Remark B.3. The stipulation that halting configurations are fixed points makes step_T a total function, which is what lets it serve as an inference rule in the next subsection. It changes nothing about the machine's behavior: T halts on w if and only if $\text{step}_T^j(c_0(w)) \in \text{Halt}_T$ for some $j \geq 0$, and the run up to that point is unaffected.

B.2 The Configuration Formal System

The first bridge between the two frameworks is the observation that one computation step is formally an inference.

Definition B.4 (Configuration formal system). Let T be a deterministic Turing machine. The *configuration formal system* of T is the xyz formal system $F_T := (x_T, y_T, z_T)$, where x_T is an alphabet adequate for writing configurations, $y_T = \text{Conf}_T$, and $z_T = \{\text{step}_T\}$, the step function regarded as a unary rule on y_T in the sense of Definition 1.1.

Proposition B.5 (Provability is reachability). *Let $\Gamma \subseteq y_T$ and $c \in y_T$. Then $\Gamma \vdash_{F_T} c$ if and only if $c = \text{step}_T^j(c')$ for some $c' \in \Gamma$ and some $j \geq 0$. Consequently*

$$\text{Con}_{F_T}(\Gamma) = \{\text{step}_T^j(c') : c' \in \Gamma, j \geq 0\},$$

the set of configurations reachable from Γ .

Proof. The displayed set contains Γ (take $j = 0$) and is inference-closed, since a direct consequence in F_T is exactly an application of step_T to a member, which increments j by one. Conversely, any inference-closed set containing Γ contains every $\text{step}_T^j(c')$ by induction on j . By the least closure property (Theorem 1.11), $\text{Con}_{F_T}(\Gamma)$ is the least such set, hence equals the displayed set. The first claim is then Definition 1.5. $\square$

Remark B.6. Proposition B.5 is the precise content of the slogan that computation is proof in the system F_T : a run of T from $c_0(w)$ is a proof from $\{c_0(w)\}$ in the sense of Definition 1.4, with proof length equal to the number of steps taken plus one.

B.3 Trace SICs

We now associate to T two SICs over y_T . The first enumerates the run and never halts; the second halts precisely when the run does.

Definition B.7 (Enumerating trace SIC). The *enumerating trace SIC* of T is $E_T = (C_E, H_E)$ over $Y = y_T$, where

$$C_E(\Gamma, m) := \{\text{step}_T^j(c) : c \in \Gamma, 0 \leq j \leq m-1\}, \quad H_E(\Gamma, m) := 0,$$

for all $\Gamma \subseteq y_T$ and $m \in \mathbf{N}^+$.

Proposition B.8. E_T is a rule-driven, deductively effective SIC over y_T .

Proof. SIC conditions (Definition 2.3): $C_E(\Gamma, 1) = \Gamma$ since j ranges over $\{0\}$; $C_E(\Gamma, m) \subseteq C_E(\Gamma, m+1)$ since the range of j grows with m ; conditions (5) and (6) hold vacuously because H_E is constantly 0.

Rule-driven (Definition 2.9): suppose $\beta \in C_E(\Gamma, m+1) \setminus C_E(\Gamma, m)$. Then $\beta = \text{step}_T^m(c)$ for some $c \in \Gamma$, so $\beta = \text{step}_T(\gamma)$ where $\gamma = \text{step}_T^{m-1}(c) \in C_E(\Gamma, m)$. Hence β is a direct consequence of $C_E(\Gamma, m)$.

Deductive effectiveness (Definition 2.5): $\bigcup_{m \in \mathbf{N}^+} C_E(\Gamma, m)$ is the set of all $\text{step}_T^j(c)$ with $c \in \Gamma$ and $j \geq 0$, which equals $\text{Con}_{F_T}(\Gamma) \cap y_T$ by Proposition B.5. $\square$

Definition B.9 (Halting trace SIC). For $\Gamma \subseteq y_T$, let

$$N(\Gamma) := \min\{m \in \mathbf{N}^+ : C_E(\Gamma, m) \cap \text{Halt}_T \neq \emptyset\},$$

with $N(\Gamma) := \infty$ if the set is empty. The *halting trace SIC* of T is $M_T = (C, H)$ over y_T , where

$$C(\Gamma, m) := C_E(\Gamma, \min(m, N(\Gamma))), \quad H(\Gamma, m) := \begin{cases} 1, & m \geq N(\Gamma), \\ 0, & \text{otherwise.} \end{cases}$$

Proposition B.10. M_T is a rule-driven SIC over y_T , and for every Γ :

1. M_T halts on Γ (that is, $H(\Gamma, m) = 1$ for some m) if and only if some configuration in Γ reaches a halting configuration of T ;
2. on the singleton input $\{c_0(w)\}$, M_T halts if and only if T halts on input w , and in that case its final state $C(\{c_0(w)\}, N)$ is exactly the set of configurations of the run of T on w , from $c_0(w)$ through the first halting configuration.

Proof. SIC conditions. $C(\Gamma, 1) = C_E(\Gamma, \min(1, N(\Gamma))) = C_E(\Gamma, 1) = \Gamma$, since $N(\Gamma) \geq 1$. Monotonicity in m holds because $\min(m, N(\Gamma))$ is nondecreasing in m and C_E is monotone in its second argument. If $H(\Gamma, m) = 1$ then $m \geq N(\Gamma)$, so $m+1 \geq N(\Gamma)$ and $H(\Gamma, m+1) = 1$; moreover $\min(m, N(\Gamma)) = N(\Gamma) = \min(m+1, N(\Gamma))$, so $C(\Gamma, m+1) = C(\Gamma, m)$.

Rule-driven. A formula entering at stage $m+1$ enters, in particular, in C_E at stage $m+1 \leq N(\Gamma)$, and the argument of Proposition B.8 applies verbatim.

Claim 1. $H(\Gamma, m) = 1$ for some m iff $N(\Gamma) < \infty$ iff some $\text{step}_T^j(c)$ with $c \in \Gamma$ lies in Halt_T .

Claim 2. Specialize claim 1 to $\Gamma = \{c_0(w)\}$ and use Remark B.3: reaching Halt_T from $c_0(w)$ is halting of T on w . The final state is $C_E(\{c_0(w)\}, N)$, which is the set of the first N configurations of the run, the last of these being the first halting configuration by minimality of N . $\square$

Remark B.11. Propositions B.8 and B.10 discharge the first item of the development program stated in earlier versions of this appendix: to each deterministic Turing machine is associated a SIC whose stage states record the corresponding configurations. The enumerating variant is the theorem-enumeration picture, with Con_{F_T} as its target; the halting variant is the target-search picture, with Halt_T as its target. The two variants instantiate, for the system F_T , exactly the two ATP modes of Definition 2.7 and Definition 2.8.

B.4 Computability of SICs

This subsection imports from Part I the apparatus needed to state the converse direction. Following the convention of this paper, results proved there are cited rather than reproved. Standard computability background may be found in [18]. Throughout this subsection, inputs to a SIC range over finite subsets of an effectively coded domain Y , each finite subset carrying a canonical code.

Definition B.12 (Finite register-machine program). A *finite register-machine program* is a finite list of instructions operating on finitely many registers ranging over $\mathbb{N}$. The basic instructions are zeroing a register, incrementing a register, copying one register into another, conditional jump on whether a register is zero, and halt.

Definition B.13 (Computable SIC). Assume $Y \subseteq y$ is effectively coded. A SIC $M = (C, H)$ over Y is *computable* if there exist total computable functions c and h such that for every code a of a finite subset $A_a \subseteq Y$ and every $m \in \mathbb{N}^+$, the value $c(a, m)$ is a code for $C(A_a, m)$ and the value $h(a, m)$ is $H(A_a, m)$.

Definition B.14 (Computationally effective SIC). A SIC $M = (C, H)$ over an effectively coded domain is *computationally effective* if there exists a finite register-machine program that, on coded input (a, m) , halts and outputs a code for $C(A_a, m)$ together with the value $H(A_a, m)$.

Theorem B.15 (Internal Church–Turing theorem for SICs). *Let $M = (C, H)$ be a SIC over an effectively coded domain. Then M is computationally effective if and only if M is computable.*

Remark B.16. The proof in runs through the classical equivalence of models of computation [18]: register-machine computability yields total computable output functions directly, and conversely total computable stage and halting maps are computed by some finite register-machine program. As Part I is careful to note, the theorem is internal to the SIC framework: it identifies two mathematical definitions of effectiveness for SICs, and is not a proof of the historical Church–Turing thesis in its philosophical form. The same caution applies to every use made of it below.

B.5 The Correspondence and Its Boundary

The two directions can now be stated together. Turing machines embed into the SIC framework (Proposition B.17); computable SICs are implemented by Turing machines (Theorem B.18); and the correspondence fails exactly where computability fails (Example B.19).

Proposition B.17 (Trace SICs are computable). *Let T be a deterministic Turing machine with effectively coded configurations. Then E_T and M_T are computable, hence computationally effective by Theorem B.15.*

Proof. Given a canonical code for a finite $\Gamma \subseteq y_T$ and $m \in \mathbb{N}^+$, one computes $C_E(\Gamma, m)$ by simulating at most $m - 1$ steps of T from each of the finitely many members of Γ ; each step is an application of the finite transition table δ , so the simulation is effective and total. This gives computability of E_T , whose halting map is constantly 0.

For M_T , one additionally decides whether $N(\Gamma) \leq m$ by inspecting the finitely many configurations of $C_E(\Gamma, m)$ for membership in Halt_T , which is a check of the state field. If yes, the least such stage $N(\Gamma)$ is found among $1, \dots, m$ and one outputs a code for $C_E(\Gamma, N(\Gamma))$ and the value 1 when $m \geq N(\Gamma)$; if no, one outputs a code for $C_E(\Gamma, m)$ and the value 0. All steps are effective, and both output functions are total. $\square$

Theorem B.18 (SIC–Turing correspondence). *Let Y be an effectively coded domain with canonical codes for finite subsets, and let $M = (C, H)$ be a SIC over Y . The following are equivalent:*

1. M is computable (Definition B.13);
2. M is computationally effective (Definition B.14);

3. there is a deterministic Turing machine that, on input a code (a, m) , halts with output a code for $C(A_a, m)$ together with the value $H(A_a, m)$.

Moreover the class of SICs satisfying these conditions contains E_T and M_T for every deterministic Turing machine T with effectively coded configurations.

Proof. (1) $\Leftrightarrow$ (2) is Theorem B.15. (1) $\Leftrightarrow$ (3) is the classical equivalence of Turing machines with the other standard models of computation [26, ?]: a total computable function is computed by some Turing machine, and conversely a Turing machine computing both outputs on every input defines total computable c and h . The final claim is Proposition B.17. $\square$

Example B.19 (A SIC beyond Turing computation). The effectiveness hypotheses in Theorem B.18 are not decorative. Let Y be any effectively coded domain, let $K \subseteq \mathbb{N}$ be any non-computable set (for instance the halting set), and define $M_K = (C, H)$ over Y by

$$C(\Gamma, m) := \Gamma, \quad H(\Gamma, m) := \begin{cases} 1, & \text{the canonical code of } \Gamma \text{ lies in } K, \\ 0, & \text{otherwise.} \end{cases}$$

Then M_K is a SIC: $C(\Gamma, 1) = \Gamma$; C is monotone and eventually constant because it is constant; H does not depend on m , so halting is persistent; and C is constant, so the frozen-after-halting condition holds. But M_K is not computable: a total computable h would decide membership in K via $h(a, 1)$.

Corollary B.20 (Separation). *Over any effectively coded domain there exist SICs that are not computationally effective. Hence the inclusion*

$$\{\text{SICs implementable by Turing machines}\} \subsetneq \{\text{SICs}\}$$

is strict, and the effectiveness conditions of Section B.4 are exactly the boundary of the SIC–Turing correspondence.

Proof. Immediate from Example B.19 and Theorem B.18. $\square$

Remark B.21 (Computability of stages against decidability of halting). A distinction that the correspondence makes sharp: M_T is computable for every Turing machine T , including universal ones, because computability concerns the stage-and-halting maps at given (Γ, m) , which requires only bounded simulation. What is not in general decidable is whether $H(\Gamma, m) = 1$ for some m —for M_T with T universal, that is the halting problem. In the proof-search vocabulary of this paper the same distinction appears as the difference between a computable SIC and a computable deductively finitely effective SIC with the halting-to-completion property, from which Part I derives the decidability of the consequence problem; where the consequence problem is undecidable, no such SIC exists. Stage computability is cheap; certified completion is not.

B.6 Discussion: Computational Power and Proof-Search Suitability

The remaining items of the earlier development program concern why this paper works with SICs when Turing machines are available and, by Theorem B.18, equivalent in power on the effective fragment. The reasons are structural, and each can now be stated against proved results rather than promissory ones.

First, the issues of computational power and of proof-search suitability separate cleanly, and the correspondence shows where. Power is settled by Theorem B.18 and Corollary B.20: on effectively

coded domains with the effectiveness conditions imposed, SICs and Turing machines implement the same maps, and without those conditions SICs are strictly more general. Suitability is a different matter. The states of a SIC are sets of formulas, so the objects this paper cares about— $\text{Con}_F(\Gamma)$ (Definition 1.6), inference closure (Theorem 1.11), self-theories (Definition 13.9), and the members of the reflection sequence proved by a machine (Definition 15.17)—are read off from the state directly. For a Turing machine the same objects exist only through a coding layer, and every statement about them acquires a faithfulness obligation of exactly the kind that step 1 of Remark 18.14 identifies as the substantial burden in certifying natural witnesses.

Second, the trace construction explains why the SIC axioms take the form they do. The monotone stage condition $C(\Gamma, m) \subseteq C(\Gamma, m + 1)$, which may look restrictive from a machine-model standpoint, is what makes the run of a Turing machine into an accumulating set of formulas rather than a sequence of overwritten tapes; it is the frozen-history presentation of computation, and it is the presentation in which computation is proof (Remark B.6). The persistent-halting and frozen-after-halting conditions are then the statements that a completed proof search stays completed and that its certificate does not change afterwards.

Third, the simulation preorder of Part I and the self-awareness preorder of Section 16 both live naturally at the SIC level. Proposition 16.22 and Corollary 16.23 relate them through tag-preserving simulation, and the tags in question are names for formulas and machines, not tape encodings. Carrying the same structure on raw Turing machines would require re-erecting the naming apparatus of Section 13 on top of a coding of configurations—possible, by Theorem B.18, but with the coding doing silent work everywhere.

Fourth, the correspondence sharpens a point about the witnesses of Section 18.8. Real theorem provers are programs, hence Turing-machine implementable, hence by Theorem B.18 presentable as computable SICs in principle. The checklist obligation (step 1 of Remark 18.14) is not existence of a SIC presentation, which this appendix now supplies for free, but faithfulness: that the presentation's formula sets and halting behavior track the prover's actual theorems and termination, so that the level computed from the presentation is a fact about the prover and not about the coding.

Finally, the equivalence-or-separation question posed in earlier versions has a two-part answer rather than a single theorem: equivalence on the effective fragment (Theorem B.18), strict separation off it (Corollary B.20). The hypotheses under which each half holds are now explicit, which is what the earlier program asked for.

Acknowledgements

The author gratefully acknowledges the use of AI assistants as editorial and research-assistance tools during the preparation of this manuscript, including proofreading, grammar and style review, consistency checking of notation and terminology, L^AT_EX formatting, organizational feedback, verification of cross-references, and the audit of definitions and proofs that produced the corrections itemized in the changelog. All mathematical claims, interpretations, conclusions, and final manuscript decisions remain the responsibility of the author.

References

- [1] Beame, P. (2004). Proof complexity (notes by A. Sabharwal). In S. Rudich & A. Wigderson (Eds.), *Computational complexity theory* (IAS/Park City Mathematics Series, Vol. 10, pp. 199–246). American Mathematical Society.
- [2] Buss, S. R. (1998). An introduction to proof theory. In S. R. Buss (Ed.), *Handbook of proof theory* (pp. 1–78). Elsevier.
- [3] Cook, S. A., & Reckhow, R. A. (1979). The relative efficiency of propositional proof systems. *Journal of Symbolic Logic*, 44(1), 36–50.
- [4] Davis, J. C. (2009). *A self-verifying theorem prover* (Doctoral dissertation). University of Texas at Austin.
- [5] Fitting, M. (1996). *First-order logic and automated theorem proving* (2nd ed.). Springer.
- [6] Ben-Sasson, E., & Impagliazzo, R. (1999). Random CNF’s are hard for the polynomial calculus. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science* (pp. 415–421). IEEE.
- [7] Clegg, M., Edmonds, J., & Impagliazzo, R. (1996). Using the Groebner basis algorithm to find proofs of unsatisfiability. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing* (pp. 174–183). ACM.
- [8] Faugère, J.-C., Gianni, P., Lazard, D., & Mora, T. (1993). Efficient computation of zero-dimensional Gröbner bases by change of ordering. *Journal of Symbolic Computation*, 16(4), 329–344.
- [9] Haken, A. (1985). The intractability of resolution. *Theoretical Computer Science*, 39, 297–308.
- [10] Harrison, J. (2009). *Handbook of practical logic and automated reasoning*. Cambridge University Press.
- [11] Oizumi, M., Albantakis, L., & Tononi, G. (2014). From the phenomenology to the mechanisms of consciousness: Integrated Information Theory 3.0. *PLoS Computational Biology*, 10(5), e1003588.
- [12] Impagliazzo, R., Pudlák, P., & Sgall, J. (1999). Lower bounds for the polynomial calculus and the Gröbner basis algorithm. *Computational Complexity*, 8(2), 127–144.
- [13] Kohlenbach, U. (2008). *Applied proof theory: Proof interpretations and their use in mathematics*. Springer Berlin, Heidelberg. DOI: 10.1007/978-3-540-77533-1.

- [14] Carneiro, M. (2024). Lean4Lean: Towards a formalized metatheory for the Lean theorem prover. *arXiv preprint* arXiv:2403.14064.
- [15] Myreen, M. O., & Davis, J. (2015). A verified generational garbage collector and self-verifying theorem prover. *Journal of Automated Reasoning*.
- [16] Prawitz, D. (2006). *Natural deduction: A proof-theoretical study*. Dover Publications.
- [17] Razborov, A. A. (1998). Lower bounds for the polynomial calculus. *Computational Complexity*, 7(4), 291–324.
- [18] Rogers, H., Jr. (1987). *Theory of recursive functions and effective computability*. MIT Press.
- [19] Sato, Y., Inoue, S., Suzuki, A., Nabeshima, K., & Sakai, K. (2011). Boolean Gröbner bases. *Journal of Symbolic Computation*, 46(5), 622–632.
- [20] Smullyan, R. M. (1992). *Gödel’s incompleteness theorems*. Oxford University Press.
- [21] Tarski, A. (1936). Der Wahrheitsbegriff in den formalisierten Sprachen. *Studia Philosophica*, 1, 261–405.
- [22] Tegmark, M. (1998). Is “the theory of everything” merely the ultimate ensemble theory? *Annals of Physics*, 270(1), 1–51.
- [23] Tegmark, M. (2008). The Mathematical Universe. *Foundations of Physics*, 38(2), 101–150.
- [24] Tononi, G. (2008). Consciousness as integrated information: A provisional manifesto. *The Biological Bulletin*, 215(3), 216–242.
- [25] Troelstra, A. S., & Schwichtenberg, H. (2000). *Basic proof theory* (2nd ed.). Cambridge University Press.
- [26] Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230–265.
- [27] Urquhart, A. (1987). Hard examples for resolution. *Journal of the ACM*, 34(1), 209–219.
- [28] van den Berg, B., Briseid, E., & Safarik, P. (2012). A functional interpretation for nonstandard arithmetic. *Annals of Pure and Applied Logic*, 163(12), 1962–1994. DOI: 10.1016/j.apal.2012.07.003.
- [29] Văth, M. (2007). *Nonstandard analysis*. Birkhäuser Basel. DOI: 10.1007/978-3-7643-7774-8.